
Debian Policy Manual

Release 4.7.3.0

The Debian Policy Mailing List

Dec 23, 2025

CONTENTS

1	About this manual	3
1.1	Scope	3
1.2	New versions of this document	4
1.3	Authors and Maintainers	4
1.3.1	Early history	4
1.3.2	Current process	4
1.3.3	Improvements	4
1.4	Related documents	5
1.5	Definitions	5
1.6	Translations	6
2	The Debian Archive	7
2.1	The Debian Free Software Guidelines	7
2.2	Archive areas	8
2.2.1	The main archive area	8
2.2.2	The non-free-firmware archive area	9
2.2.3	The contrib archive area	9
2.2.4	The non-free archive area	9
2.3	Copyright considerations	10
2.4	Sections	11
2.5	Priorities	11
3	Binary packages	13
3.1	The package name	13
3.1.1	Packages with potentially offensive content	13
3.2	The version of a package	13
3.2.1	Version numbers based on dates	14
3.2.2	Uniqueness of version numbers	14
3.3	The maintainer of a package	14
3.4	The description of a package	15
3.4.1	The single line synopsis	15
3.4.2	The extended description	15
3.5	Dependencies	15
3.6	Virtual packages	16
3.7	Base system	16
3.8	Essential packages	16
3.9	Maintainer Scripts	17
3.9.1	Prompting in maintainer scripts	17
4	Source packages	19

4.1	Standards conformance	19
4.2	Package relationships	20
4.3	Changes to the upstream sources	20
4.4	Debian changelog: <code>debian/changelog</code>	21
4.5	Copyright: <code>debian/copyright</code>	22
4.6	Error trapping in makefiles	22
4.7	Time Stamps	23
4.8	Restrictions on objects in source packages	23
4.9	Main building script: <code>debian/rules</code>	23
4.9.1	<code>debian/rules</code> and <code>DEB_BUILD_OPTIONS</code>	26
4.9.2	<code>debian/rules</code> and <code>Rules-Requires-Root</code>	27
4.10	Variable substitutions: <code>debian/substvars</code>	28
4.11	Upstream source location: <code>debian/watch</code>	28
4.12	Generated files list: <code>debian/files</code>	28
4.13	Embedded code copies	29
4.14	Source package handling: <code>debian/README.source</code>	29
4.15	Reproducibility	29
4.16	Missing sources: <code>debian/missing-sources</code>	30
4.17	Vendor-specific patch series	30
5	Control files and their fields	31
5.1	Syntax of control files	31
5.2	Debian source package template control files – <code>debian/control</code>	32
5.3	Debian binary package control files – <code>DEBIAN/control</code>	33
5.4	Debian source package control files – <code>.dsc</code>	34
5.5	Debian upload changes control files – <code>.changes</code>	34
5.6	List of fields	35
5.6.1	Source	35
5.6.2	Maintainer	35
5.6.3	Uploaders	35
5.6.4	Changed-By	36
5.6.5	Section	36
5.6.6	Priority	36
5.6.7	Package	36
5.6.8	Architecture	36
5.6.9	Essential	37
5.6.10	Package interrelationship fields: <code>Depends</code> , <code>Pre-Depends</code> , <code>Recommends</code> , <code>Suggests</code> , <code>Breaks</code> , <code>Conflicts</code> , <code>Provides</code> , <code>Replaces</code> , <code>Enhances</code>	37
5.6.11	<code>Standards-Version</code>	37
5.6.12	<code>Version</code>	37
5.6.12.1	Epochs should be used sparingly	38
5.6.12.2	Special version conventions	39
5.6.13	<code>Description</code>	40
5.6.14	<code>Distribution</code>	41
5.6.15	<code>Date</code>	41
5.6.16	<code>Format</code>	41
5.6.17	<code>Urgency</code>	41
5.6.18	<code>Changes</code>	42
5.6.19	<code>Binary</code>	42
5.6.20	<code>Installed-Size</code>	42
5.6.21	<code>Files</code>	42
5.6.22	<code>Closes</code>	43
5.6.23	<code>Homepage</code>	43
5.6.24	<code>Checksums-Sha1</code> and <code>Checksums-Sha256</code>	43

5.6.25	DM-Upload-Allowed	44
5.6.26	Version Control System (VCS) fields	44
5.6.27	Package-List	45
5.6.28	Package-Type	45
5.6.29	Dgit	45
5.6.30	Testsuite	45
5.6.31	Rules-Requires-Root	46
5.6.31.1	Remarks	46
5.6.31.2	Definition of the keywords	46
5.6.31.3	Provided keywords	47
5.6.32	Git-Tag-Tagger	47
5.6.33	Git-Tag-Info	47
5.7	User-defined fields	47
5.8	Obsolete fields	48
5.8.1	DM-Upload-Allowed	48
6	Package maintainer scripts and installation procedure	49
6.1	Introduction to package maintainer scripts	49
6.2	Maintainer scripts idempotency	49
6.3	Controlling terminal for maintainer scripts	50
6.4	Exit status	50
6.5	Summary of ways maintainer scripts are called	50
6.6	Details of unpack phase of installation or upgrade	52
6.7	Details of configuration	55
6.8	Details of removal and/or configuration purging	55
7	Declaring relationships between packages	57
7.1	Syntax of relationship fields	57
7.2	Binary Dependencies - Depends, Recommends, Suggests, Enhances, Pre-Depends	58
7.3	Packages which break other packages - Breaks	60
7.4	Conflicting binary packages - Conflicts	61
7.5	Virtual packages - Provides	62
7.6	Overwriting files and replacing packages - Replaces	63
7.6.1	Overwriting files in other packages	63
7.6.2	Replacing whole packages, forcing their removal	63
7.7	Relationships between source and binary packages - Build-Depends, Build-Depends-Indep, Build-Depends-Arch, Build-Conflicts, Build-Conflicts-Indep, Build-Conflicts-Arch	64
7.8	Additional source packages used to build the binary - Built-Using	65
8	Shared libraries	67
8.1	Run-time shared libraries	67
8.1.1	ldconfig	68
8.2	Shared library support files	69
8.3	Static libraries	69
8.4	Development files	69
8.5	Dependencies between the packages of the same library	70
8.6	Dependencies between the library and other packages	70
8.6.1	Generating dependencies on shared libraries	70
8.6.2	Shared library ABI changes	71
8.6.3	The symbols system	72
8.6.3.1	The symbols files present on the system	72
8.6.3.2	The symbols File Format	73
8.6.3.3	Providing a symbols file	74
8.6.4	The shlibs system	75

8.6.4.1	The <code>shlibs</code> files present on the system	75
8.6.4.2	The <code>shlibs</code> File Format	75
8.6.4.3	Providing a <code>shlibs</code> file	76
9	The Operating System	77
9.1	File system hierarchy	77
9.1.1	File System Structure	77
9.1.2	Site-specific programs	78
9.1.3	The system-wide mail directory	79
9.1.4	<code>/run</code> and <code>/run/lock</code>	79
9.2	Users and groups	79
9.2.1	Introduction	79
9.2.2	UID and GID classes	80
9.2.3	Non-existent home directories	80
9.3	Starting system services	81
9.3.1	Introduction	81
9.3.2	Writing the scripts	81
9.3.3	Interfacing with <code>init</code> systems	82
9.3.3.1	Managing the links	83
9.3.3.2	Running <code>init</code> scripts	83
9.3.4	Boot-time initialization	84
9.3.5	Example	84
9.4	Console messages from <code>init.d</code> scripts	84
9.5	Cron jobs	84
9.5.1	Cron job file names	85
9.6	Menus	85
9.7	Multimedia handlers	86
9.7.1	Registration of media type handlers with desktop entries	86
9.7.2	Registration of media type handlers with mailcap entries	86
9.7.3	Providing media types to files	86
9.8	Keyboard configuration	87
9.9	Environment variables	88
9.10	Registering Documents using <code>doc-base</code>	88
9.11	Alternate <code>init</code> systems	88
9.11.1	Event-based boot with <code>upstart</code>	88
9.12	Signaling that a reboot is required	88
10	Files	89
10.1	Binaries	89
10.2	Libraries	90
10.3	Shared libraries	91
10.4	Scripts	91
10.5	Symbolic links	92
10.6	Device files	93
10.7	Configuration files	93
10.7.1	Definitions	93
10.7.2	Location	94
10.7.3	Behavior	94
10.7.4	Sharing configuration files	94
10.7.5	User configuration files (“dotfiles”)	95
10.8	Log files	95
10.9	Locale files	96
10.10	Permissions and owners	96
10.10.1	The use of <code>dpkg-statoverride</code>	97

10.11	File names	98
11	Customized programs	99
11.1	Architecture specification strings	99
11.1.1	Architecture wildcards	99
11.2	Daemons	99
11.3	Using pseudo-ttys and modifying wtmp, utmp and lastlog	99
11.4	Editors and pagers	100
11.5	Web servers and applications	100
11.6	Mail transport, delivery and user agents	101
11.7	News system configuration	102
11.8	Programs for the X Window System	102
11.8.1	Providing X support and package priorities	102
11.8.2	Packages providing an X server	102
11.8.3	Packages providing a terminal emulator	103
11.8.4	Packages providing a window manager	103
11.8.5	Packages providing fonts	103
11.8.6	Application defaults files	104
11.8.7	Installation directory issues	105
11.9	Perl programs and modules	105
11.10	Emacs lisp programs	105
11.11	Games	105
12	Documentation	107
12.1	Manual pages	107
12.2	Info documents	108
12.3	Additional documentation	108
12.4	Preferred documentation formats	109
12.5	Copyright information	109
12.5.1	Machine-readable copyright information	110
12.6	Examples	110
12.7	Changelog files and release notes	110
13	Introduction and scope of these appendices	113
14	Binary packages (from old Packaging Manual)	115
14.1	Creating package files - dpkg-deb	115
14.2	Binary package metadata files	116
14.3	The binary package control file: control	116
14.4	Time Stamps	116
15	Source packages (from old Packaging Manual)	117
15.1	Tools for processing source packages	117
15.1.1	dpkg-source - packs and unpacks Debian source packages	117
15.1.2	dpkg-buildpackage - overall package-building control script	117
15.1.3	dpkg-gencontrol - generates binary package control files	118
15.1.4	dpkg-shlibdeps - calculates shared library dependencies	118
15.1.5	dpkg-distaddfile - adds a file to debian/files	118
15.1.6	dpkg-genchanges - generates a .changes upload control file	118
15.1.7	dpkg-parsechangelog - produces parsed representation of a changelog	118
15.1.8	dpkg-architecture - information about the build and host system	119
15.2	The Debian package source tree	119
15.2.1	debian/rules - the main building script	119
15.2.2	debian/substvars and variable substitutions	119
15.2.3	debian/files	119

15.2.4	debian/tmp	119
15.3	Source packages as archives	119
15.4	Unpacking a Debian source package without <code>dpkg-source</code>	120
15.4.1	Restrictions on objects in source packages	120
16	Control files and their fields (from old Packaging Manual)	123
16.1	Syntax of control files	123
16.2	List of fields	123
16.2.1	Filename	123
16.2.2	Size and MD5sum	123
16.2.3	Status	123
16.2.4	Config-Version	124
16.2.5	Conffiles	124
16.2.6	Obsolete fields	124
17	Configuration file handling (from old Packaging Manual)	125
17.1	Automatic handling of configuration files by <code>dpkg</code>	125
17.2	Fully-featured maintainer script configuration handling	126
18	Alternative versions of an interface - <code>update-alternatives</code> (from old Packaging Manual)	127
19	Diversions - overriding a package's version of a file (from old Packaging Manual)	129
20	Debian Policy changes process	131
20.1	Introduction	131
20.2	Change Goals	131
20.3	Current Process	131
20.3.1	State A: More information required	132
20.3.2	State B: Discussion	132
20.3.3	State C: Proposal	132
20.3.4	State D: Wording proposed	132
20.3.5	State E: Seconded	132
20.3.6	State F: Accepted	133
20.3.7	State G: Reject	133
20.4	Other Tags	133
21	Maintainer script flowcharts	135
22	Upgrading checklist	141
22.1	About the checklist	141
22.2	Version 4.7.3	141
22.3	Version 4.7.2	141
22.4	Version 4.7.1	142
22.5	Version 4.7.0	142
22.6	Version 4.6.2	143
22.7	Version 4.6.1	143
22.8	Version 4.6.0	143
22.9	Version 4.5.1	143
22.10	Version 4.5.0	144
22.11	Version 4.4.1	144
22.12	Version 4.4.0	145
22.13	Version 4.3.0	145
22.14	Version 4.2.1	146
22.15	Version 4.2.0	146
22.16	Version 4.1.5	146

22.17	Version 4.1.4	147
22.18	Version 4.1.3	147
22.19	Version 4.1.2	147
22.20	Version 4.1.1	148
22.21	Version 4.1.0	148
22.22	Version 4.0.1	149
22.23	Version 4.0.0	149
22.24	Version 3.9.8	151
22.25	Version 3.9.7	151
22.26	Version 3.9.6	151
22.27	Version 3.9.5	152
22.28	Version 3.9.4	153
22.29	Version 3.9.3	153
22.30	Version 3.9.2	154
22.31	Version 3.9.1	155
22.32	Version 3.9.0	156
22.33	Version 3.8.4	158
22.34	Version 3.8.3	158
22.35	Version 3.8.2	159
22.36	Version 3.8.1	159
22.37	Version 3.8.0	160
22.38	Version 3.7.3	161
22.39	Version 3.7.2.2	162
22.40	Version 3.7.2	162
22.41	Version 3.7.1	162
22.42	Version 3.7.0	162
22.43	Version 3.6.2	163
22.44	Version 3.6.1	163
22.45	Version 3.6.0	163
22.46	Version 3.5.10	164
22.47	Version 3.5.9	164
22.48	Version 3.5.8	164
22.49	Version 3.5.7	165
22.50	Version 3.5.6	165
22.51	Version 3.5.5	165
22.52	Version 3.5.4	166
22.53	Version 3.5.3	166
22.54	Version 3.5.2	166
22.55	Version 3.5.1	167
22.56	Version 3.5.0	167
22.57	Version 3.2.1.1	167
22.58	Version 3.2.1	167
22.59	Version 3.2.0	167
22.60	Version 3.1.1	168
22.61	Version 3.1.0	168
22.62	Version 3.0.1	169
22.63	Version 3.0.0	169
22.64	Version 2.5.0	170
22.65	Version 2.4.1	170
22.66	Version 2.4.0	170
22.67	Version 2.3.0	171
22.68	Version 2.2.0	171
22.69	Version 2.1.3	172
22.70	Version 2.1.2	172

22.71 Version 2.1.1	172
22.72 Version 2.1.0	172
23 License	173
Index	175

This manual describes the policy requirements for the Debian distribution. This includes the structure and contents of the Debian archive and several design issues of the operating system, as well as technical requirements that each package must satisfy to be included in the distribution.

This is Debian Policy version 4.7.3.0, released on 2025-12-23.

ABOUT THIS MANUAL

1.1 Scope

This manual describes the policy requirements for the Debian distribution. This includes the structure and contents of the Debian archive and several design issues of the operating system, as well as technical requirements that each package must satisfy to be included in the distribution.

This manual also describes Debian policy as it relates to creating Debian packages. It is not a tutorial on how to build packages, nor is it exhaustive where it comes to describing the behavior of the packaging system. Instead, this manual attempts to define the interface to the package management system with which the developers must be conversant.¹

This manual cannot and does not prohibit every possible bug or undesirable behaviour. The fact that something is not prohibited by Debian policy does not mean that it is not a bug, let alone that it is desirable. Questions not covered by policy should be evaluated on their merits.

The footnotes present in this manual are merely informative, and are not part of Debian policy itself.

The appendices to this manual are not necessarily normative, either. Please see *Introduction and scope of these appendices* for more information.

In the normative part of this manual, the following terms are used to describe the importance of each statement:²

- The terms *must* and *must not*, and the adjectives *required* and *prohibited*, denote strong requirements. Packages that do not conform to these requirements will generally not be considered acceptable for the Debian distribution. These statements correspond to the *critical*, *grave*, and *serious* bug severities (normally serious). They are collectively called *Policy requirements*.
- The terms *should* and *should not*, and the adjective *recommended*, denote best practices. Non-conformance with these guidelines will generally be considered a bug, but will not necessarily render a package unsuitable for distribution. These statements correspond to bug severities of *important*, *normal*, and *minor*. They are collectively called *Policy recommendations*.
- The adjectives *encouraged* and *discouraged* denote places where Policy offers advice to maintainers, but maintainers are free to follow or not follow that advice. Non-conformance with this advice is normally not considered a bug; if a bug seems worthwhile, normally it would have a severity of *wishlist*. These statements are collectively called *Policy advice*.

¹ Informally, the criteria used for inclusion is that the material meet one of the following requirements:

Standard interfaces

The material presented represents an interface to the packaging system that is mandated for use, and is used by, a significant number of packages, and therefore should not be changed without peer review. Package maintainers can then rely on this interface not changing, and the package management software authors need to ensure compatibility with this interface definition. (Control file and changelog file formats are examples.)

Chosen Convention

If there are a number of technically viable choices that can be made, but one needs to select one of these options for inter-operability. The version number format is one example.

Please note that these are not mutually exclusive; selected conventions often become parts of standard interfaces.

² Compare RFC 2119. Note, however, that these words are used in a different way in this document.

- The term *may* and the adjective *optional* are used to clarify cases where it may otherwise appear that Policy is specifying a requirement or recommendation. In those cases, these words describe decisions that are truly optional and at the maintainer's discretion.

The Release Team can, at their discretion, downgrade a Policy requirement to a Policy recommendation for a given release of the Debian distribution. This may be done for only a specific package or for the archive as a whole. This provision is intended to provide flexibility to balance the quality standards of the distribution against the release schedule and the importance of making a stable release.

Much of the information presented in this manual will be useful even when building a package which is to be distributed in some other way or is intended for local use only.

udebs (stripped-down binary packages used by the Debian Installer) and source packages that produce only udebs do not comply with all of the requirements discussed here. See the [Debian Installer internals manual](#) for more information about them.

1.2 New versions of this document

This manual is distributed via the Debian package [debian-policy](#).

The current version of this document is also available from the Debian web mirrors at <https://www.debian.org/doc/debian-policy/>. Also available from the same directory are several other formats: [policy.epub](#), [policy.txt](#) and [policy.pdf](#). Included in both the same directory and in the [debian-policy](#) package is a standalone copy of *Upgrading checklist*, which indicates policy changes between versions of this document.

1.3 Authors and Maintainers

1.3.1 Early history

Originally called “Debian GNU/Linux Policy Manual”, this manual was initially written in 1996 by Ian Jackson. It was revised on November 27th, 1996 by David A. Morris. Christian Schwarz added new sections on March 15th, 1997, and reworked/restructured it in April-July 1997. Christoph Lameter contributed the “Web Standard”. Julian Gilbey largely restructured it in 2001. Since September 1998, changes to the contents of this document have been co-ordinated by means of the [debian-policy mailing list](#)

1.3.2 Current process

The Policy Editors are DPL delegates with responsibility for the contents of this document (see the Debian Constitution for the meaning of “DPL delegate”). However, the Policy Editors further delegate their editorial power to a process of establishing project member consensus on the [debian-policy mailing list](#), as described in *Debian Policy changes process*. The current Policy Editors are:

1. Russ Allbery
2. Sean Whitton

1.3.3 Improvements

While the authors of this document have tried hard to avoid typos and other errors, these do still occur. If you discover an error in this manual or if you want to give any comments, suggestions, or criticisms please send an email to the Debian Policy Mailing List, debian-policy@lists.debian.org, or submit a bug report against the [debian-policy](#) package.

Please do not try to reach the individual authors or maintainers of the Policy Manual regarding changes to the Policy.

New techniques and functionality are generally implemented in the Debian archive (long) before they are detailed in this document. This is not considered to be a problem: there is a consensus in the Debian Project that the task of keeping this document up-to-date should never block making improvements to Debian. Nevertheless, it is better to submit patches

to this document sooner rather than later. This reduces the amount of work that is needed on the part of others to get themselves up-to-speed on new best practices.

1.4 Related documents

There are several other documents other than this Policy Manual that are necessary to fully understand some Debian policies and procedures.

The external “sub-policy” documents are referred to in:

- *File System Structure*
- *Virtual packages*
- *Menus*
- *Perl programs and modules*
- *Prompting in maintainer scripts*
- *Emacs lisp programs*

In addition to those, which carry the weight of policy, there is the Debian Developer’s Reference. This document describes procedures and resources for Debian developers, but it is *not* normative; rather, it includes things that don’t belong in the Policy, such as best practices for developers.

The Developer’s Reference is available in the developers-reference package. It’s also available from the Debian web mirrors at <https://www.debian.org/doc/developers-reference/>.

Finally, a *specification for machine-readable copyright files* is maintained as part of the debian-policy package using the same procedure as the other policy documents. Use of this format is optional.

1.5 Definitions

The following terms are used in this Policy Manual:

ASCII

The character encoding specified by ANSI X3.4-1986 and its predecessor standards, referred to in MIME as US-ASCII, and corresponding to an encoding in eight bits per character of the first 128 [Unicode](#) characters, with the eighth bit always zero.

upstream

The source of software that is being packaged, or the portion of a software package that originates from outside of Debian. For example, suppose Alice writes and releases a free software package, and then Bob creates a Debian package of that software package. Alice is the *upstream maintainer* (sometimes abbreviated as *upstream*) of the package, Alice’s releases are the *upstream releases*, and the version number she puts on a release is the *upstream version*. Bob may make Debian-specific modifications to the package, and then later send those modifications *upstream* to be incorporated in Alice’s releases.

The packager and upstream developer may be the same person. For example, Alice may choose to package her own software for Debian. However, this manual still distinguishes between the role of upstream and the role of Debian packager, even when the same person is filling both of those roles, since they have some implications for the details of packaging.

UTF-8

The transformation format (sometimes called encoding) of [Unicode](#) defined by [RFC 3629](#). UTF-8 has the useful property of having ASCII as a subset, so any text encoded in ASCII is trivially also valid UTF-8.

1.6 Translations

When translations of this document into languages other than English disagree with the English text, the English text takes precedence.

THE DEBIAN ARCHIVE

The Debian system is maintained and distributed as a collection of *packages*. Since there are so many of them (currently well over 15000), they are split into *sections* and given *priorities* to simplify the handling of them.

The effort of the Debian project is to build a free operating system, but not every package we want to make accessible is *free* in our sense (see the Debian Free Software Guidelines, below), or may be imported/exported without restrictions. Thus, the archive is split into areas¹ based on their licenses and other restrictions.

The aims of this are:

- to allow us to make as much software available as we can
- to allow us to encourage everyone to write free software, and
- to allow us to make it easy for people to produce CD-ROMs of our system without violating any licenses, import/export restrictions, or any other laws.

The *main* archive area forms the *Debian distribution*.

Packages in the other archive areas (*non-free-firmware*, *contrib*, *non-free*) are not considered to be part of the Debian distribution, although we support their use and provide infrastructure for them (such as our bug-tracking system and mailing lists). This Debian Policy Manual applies to these packages as well.

2.1 The Debian Free Software Guidelines

The Debian Free Software Guidelines (DFSG) form our definition of “free software”. These are:

1. Free Redistribution

The license of a Debian component may not restrict any party from selling or giving away the software as a component of an aggregate software distribution containing programs from several different sources. The license may not require a royalty or other fee for such sale.

2. Source Code

The program must include source code, and must allow distribution in source code as well as compiled form.

3. Derived Works

The license must allow modifications and derived works, and must allow them to be distributed under the same terms as the license of the original software.

4. Integrity of The Author's Source Code

The license may restrict source-code from being distributed in modified form *only* if the license allows the distribution of “patch files” with the source code for the purpose of modifying the program at build time. The license must explicitly permit distribution of software built from modified source code. The license may require derived works to carry a different name or version number from the original software. (This is a

¹ The Debian archive software uses the term “component” internally and in the Release file format to refer to the division of an archive. The Debian Social Contract simply refers to “areas.” This document uses terminology similar to the Social Contract.

compromise. The Debian Project encourages all authors to not restrict any files, source or binary, from being modified.)

5. No Discrimination Against Persons or Groups

The license must not discriminate against any person or group of persons.

6. No Discrimination Against Fields of Endeavor

The license must not restrict anyone from making use of the program in a specific field of endeavor. For example, it may not restrict the program from being used in a business, or from being used for genetic research.

7. Distribution of License

The rights attached to the program must apply to all to whom the program is redistributed without the need for execution of an additional license by those parties.

8. License Must Not Be Specific to Debian

The rights attached to the program must not depend on the program's being part of a Debian system. If the program is extracted from Debian and used or distributed without Debian but otherwise within the terms of the program's license, all parties to whom the program is redistributed must have the same rights as those that are granted in conjunction with the Debian system.

9. License Must Not Contaminate Other Software

The license must not place restrictions on other software that is distributed along with the licensed software. For example, the license must not insist that all other programs distributed on the same medium must be free software.

10. Example Licenses

The “GPL,” “BSD,” and “Artistic” licenses are examples of licenses that we consider *free*.

2.2 Archive areas

2.2.1 The main archive area

The *main* archive area comprises the Debian distribution. Only the packages in this area are considered part of the distribution. None of the packages in the *main* archive area require software outside of that area to function. Anyone may use, share, modify and redistribute the packages in this archive area freely².

Every package in *main* must comply with the DFSG (Debian Free Software Guidelines).³

In addition, the packages in *main*

- must not require or recommend a package outside of *main* for compilation or execution (thus, the package must not declare a `Pre-Depends`, `Depends`, `Recommends`, `Build-Depends`, `Build-Depends-Indep`, or `Build-Depends-Arch` relationship on a non-*main* package unless that package is only listed as a non-default alternative for a package in *main*),
- must not be so buggy that we refuse to support them, and
- must meet all policy requirements presented in this manual.

If a source package is in the *main* archive area, then at least one of its binary packages must be in the *main* archive area, and each of the remaining packages must be in either the *main* or *contrib* archive area. Each binary package's archive area is indicated by its `Section` field: see [Sections](#).

Source packages in *main* with a mixture of *main* and *contrib* binary packages are more complex for archive tooling to handle, and therefore should be limited to situations where it would be inconvenient to split the source package. If it is straightforward to split the source package into a *main* part and a *contrib* part that are built separately, then those parts should be represented as separate source packages.

² See [What Does Free Mean?](#) for more about what we mean by free software.

³ Debian's FTP Masters publish a [REJECT-FAQ](#) which details the project's current working interpretation of the DFSG.

When a *main* source package has a mixture of *main* and *contrib* binary packages, the source package and the *main* binary packages must follow the requirements for *main* packages, but the *contrib* binary packages may follow the weaker requirements for *contrib* packages. In particular, source packages in *main* must not have build dependencies outside *main*, but the *contrib* binary packages may have runtime dependencies outside *main*.

2.2.2 The non-free-firmware archive area

The *non-free-firmware* archive area contains packages providing firmware needed to initialize, use or keep updated hardware required by our users, typically necessary for important functions to be available (i.e. wireless network connectivity) or for fixing security defects in hardware (i.e. CPU microcode updates). Packages in this archive may not comply with all of the policy requirements in this manual due to lack of source code availability, restrictions on modification or other limitations.

Packages in *non-free-firmware*

- must not be so buggy that we refuse to support them, and
- must meet all policy requirements presented in this manual that it is possible for them to meet.⁴

2.2.3 The contrib archive area

The *contrib* archive area contains supplemental packages intended to work with the Debian distribution, but which require software outside of the distribution to either build or function.

Every package in *contrib* must comply with the DFSG.

In addition, the packages in *contrib*

- must not be so buggy that we refuse to support them, and
- must meet all policy requirements presented in this manual.

Examples of packages which would be included in *contrib* are:

- free packages which require *contrib*, *non-free* packages or packages which are not in our archive at all for compilation or execution, and
- wrapper packages or other sorts of free accessories for non-free programs.

If a source package is in the *contrib* archive area, then each of the binary packages that it produces must also be in the *contrib* archive area.

2.2.4 The non-free archive area

The *non-free* archive area contains supplemental packages intended to work with the Debian distribution that do not comply with the DFSG or have other problems that make their distribution problematic. They may not comply with all of the policy requirements in this manual due to restrictions on modifications or other limitations.

Packages must be placed in *non-free* if they are not compliant with the DFSG or are encumbered by patents or other legal issues that make their distribution problematic.

In addition, the packages in *non-free*

- must not be so buggy that we refuse to support them, and
- must meet all policy requirements presented in this manual that it is possible for them to meet.⁴

If a source package is in the *non-free* archive area, then each of the binary packages that it produces must also be in the *non-free* archive area.

⁴ It is possible that there are policy requirements which the package is unable to meet, for example, if the source is unavailable. These situations will need to be handled on a case-by-case basis.

2.3 Copyright considerations

Every package must be accompanied by a verbatim copy of its distribution license(s) in the file `/usr/share/doc/PACKAGE/copyright`.

The copyright information for files in a package must be copied verbatim into `/usr/share/doc/PACKAGE/copyright`, when all of the following hold:

1. the distribution license for those files requires that copyright information be included in all copies and/or binary distributions;
2. the files are shipped in the binary package, either in source or compiled form; and
3. the form in which the files are present in the binary package does not include a plain text version of their copyright notices.

Thus, the copyright information for files in the source package which are only part of its build process, such as autotools files, need not be included in `/usr/share/doc/PACKAGE/copyright`, because those files do not get installed into the binary package. Similarly, plain text files which include their own copyright information and are installed into the binary package unmodified need not have that copyright information copied into `/usr/share/doc/PACKAGE/copyright`.

However, the copyright notices for any files which are compiled into the object code shipped in the binary package must all be included in `/usr/share/doc/PACKAGE/copyright` when the license requires that copyright information be included in all copies and/or binary distributions, as most do.⁵

See [Copyright information](#) for further details.

We reserve the right to restrict files from being included anywhere in our archives if

- their use or distribution would break a law,
- there is an ethical conflict in their distribution or use,
- we would have to sign a license for them, or
- their distribution would conflict with other project policies.

Programs whose authors encourage the user to make donations are fine for the main distribution, provided that the authors do not claim that not donating is immoral, unethical, illegal or something similar; in such a case they must go in *non-free*.

Packages whose copyright permission notices (or patent problems) do not even allow redistribution of binaries only, and where no special permission has been obtained, must not be placed on the Debian FTP site and its mirrors at all.

Note that under international copyright law (this applies in the United States, too), *no* distribution or modification of a work is allowed without an explicit notice saying so. Therefore a program without a copyright notice *is* copyrighted and you may not do anything to it without risking being sued! Likewise if a program has a copyright notice but no statement saying what is permitted then nothing is permitted.

Many authors are unaware of the problems that restrictive copyrights (or lack of copyright notices) can cause for the users of their supposedly-free software. It is often worthwhile contacting such authors diplomatically to ask them to modify their license terms. However, this can be a politically difficult thing to do and you should ask for advice on the `debian-legal` mailing list first, as explained below.

When in doubt about a copyright, send mail to debian-legal@lists.debian.org. Be prepared to provide us with the copyright statement. Software covered by the GPL, public domain software and BSD-like copyrights are safe; be wary of the phrases “commercial use prohibited” and “distribution restricted”.

⁵ Licenses that are not thought to require the copying of all copyright notices into Debian’s copyright file include Apache-2.0 and the Boost Software License, version 1.0. Final determination as to whether a package’s copyright file is sufficient lies with the FTP team.

To help find copyright notices you need to copy, you might try `grep --color=always -Eir '(copyright|©)' ' * | less -R`

2.4 Sections

The packages in the archive areas *main*, *non-free-firmware*, *contrib* and *non-free* are grouped further into *sections* to simplify handling.

The archive area and section for each package should be specified in the package's `Section` control field (see [Section](#)). However, the maintainer of the Debian archive may override this selection to ensure the consistency of the Debian distribution. The `Section` field should be of the form:

- *section* if the package is in the *main* archive area,
- *area/section* if the package is in the *non-free-firmware*, *contrib* or *non-free* archive areas.

The Debian archive maintainers provide the authoritative list of sections. At present, they are: `admin`, `cli-mono`, `comm`, `database`, `debug`, `devel`, `doc`, `editors`, `education`, `electronics`, `embedded`, `fonts`, `games`, `gnome`, `gnu-r`, `gnustep`, `graphics`, `hamradio`, `haskell`, `httpd`, `interpreters`, `introspection`, `java`, `javascript`, `kde`, `kernel`, `libdevel`, `libs`, `lisp`, `localization`, `mail`, `math`, `metapackages`, `misc`, `net`, `news`, `ocaml`, `oldlibs`, `otherosfs`, `perl`, `php`, `python`, `ruby`, `rust`, `science`, `shells`, `sound`, `tasks`, `tex`, `text`, `utils`, `vcs`, `video`, `web`, `x11`, `xfce`, `zope`. The additional section *debian-installer* contains special packages used by the installer and is not used for normal Debian packages.

For more information about the sections and their definitions, see the [list of sections in unstable](#).

2.5 Priorities

Each package must have a *priority* value, which is set in the metadata for the Debian archive and is also included in the package's binary control (see [Priority](#)), `.dsc` and `.changes` files. This information is used to control which packages are included in standard or minimal Debian installations.

Starting with `dpkg` version 1.22.13, the *Priority* of a binary package is set to `optional` unless specified otherwise in the source or its binary stanza in the source package template control file. Therefore, explicitly specifying *Priority* as `optional` may be omitted.

Most Debian packages will have a priority of `optional`. Priority levels other than `optional` are only used for packages that should be included by default in a standard installation of Debian.

The priority of a package is determined solely by the functionality it provides directly to the user. The priority of a package should not be increased merely because another higher-priority package depends on it; instead, the tools used to construct Debian installations will correctly handle package dependencies. In particular, this means that C-like libraries will almost never have a priority above `optional`, since they do not provide functionality directly to users. However, as an exception, the maintainers of Debian installers may request an increase of the priority of a package to resolve installation issues and ensure that the correct set of packages is included in a standard or minimal install.

The following *priority levels* are recognized by the Debian package management tools.

required

Packages which are necessary for the proper functioning of the system (usually, this means that `dpkg` functionality depends on these packages). Removing a `required` package may cause your system to become totally broken and you may not even be able to use `dpkg` to put things back, so only do so if you know what you are doing.

Systems with only the `required` packages installed have at least enough functionality for the sysadmin to boot the system and install more software.

important

Important programs, including those which one would expect to find on any Unix-like system. If the expectation is that an experienced Unix person who found it missing would say “What on earth is going on, where is `foo`?”, it must be an `important` package.⁶ Other packages without which the system will not run well or be usable must also have priority `important`. This does *not* include Emacs, the X Window System, TeX or any other large applications. The `important` packages are just a bare minimum of commonly-expected and necessary tools.

⁶ This is an important criterion because we are trying to produce, amongst other things, a free Unix.

standard

These packages provide a reasonably small but not too limited character-mode system. This is what will be installed by default if the user doesn't select anything else. It doesn't include many large applications.

Two packages that both have a priority of `standard` or higher must not conflict with each other.

optional

This is the default priority for the majority of the archive. Unless a package should be installed by default on standard Debian systems, it should have a priority of `optional`. Packages with a priority of `optional` may conflict with each other.

extra

This priority is deprecated. Use the `optional` priority instead. This priority should be treated as equivalent to `optional`.

The `extra` priority was previously used for packages that conflicted with other packages and packages that were only likely to be useful to people with specialized requirements. However, this distinction was somewhat arbitrary, not consistently followed, and not useful enough to warrant the maintenance effort.

BINARY PACKAGES

The Debian distribution is based on the Debian package management system, called `dpkg`. Thus, all packages in the Debian distribution must be provided in the `.deb` file format.

A `.deb` package contains two sets of files: a set of files to install on the system when the package is installed, and a set of files that provide additional metadata about the package or which are executed when the package is installed or removed. This second set of files is called *package metadata files*. Among those files are the package maintainer scripts and `control`, the *binary package control file* that contains the control fields for the package. Other package metadata files include *symbols* or *shlibs* used to store shared library dependency information and the `conffiles` file that lists the package's configuration files (described in *Configuration files*).

There is unfortunately a collision of terminology here between control information files and files in the Debian control file format. Throughout this document, a *control file* refers to a file in the Debian control file format. These files are documented in *Control files and their fields*. Only files referred to specifically as *package metadata files* are the files included in the package metadata member (called `control.tar`) of the `.deb` file format used by binary packages. Most package metadata files are not in the Debian control file format.

3.1 The package name

Every package must have a name that's unique within the Debian archive.

The package name is included in the control field `Package`, the format of which is described in *Package*. The package name is also included as a part of the file name of the `.deb` file.

3.1.1 Packages with potentially offensive content

As a maintainer you should make a judgement about whether the contents of a package is appropriate to include, whether it needs any kind of content warning, and whether some parts should be split out into a separate package (so that users who want to avoid certain parts can do so). In making these decisions you should take into account the project's views as expressed in our Diversity Statement.

If you split out (potentially) offensive or disturbing material into a separate package, you should usually mark this in the package name by adding `-offensive`. For example, `cowsay` vs `cowsay-offensive`. In this situation the `-offensive` package can be Suggested by the core package(s), but should not be Recommended or Depended on.

3.2 The version of a package

Every package has a version number recorded in its `Version` control file field, described in *Version*.

The package management system imposes an ordering on version numbers, so that it can tell whether packages are being up- or downgraded and so that package system front end applications can tell whether a package it finds available is newer than the one installed on the system. The version number format has the most significant parts (as far as comparison is concerned) at the beginning.

If an upstream package has problematic version numbers they should be converted to a sane form for use in the `Version` field.

3.2.1 Version numbers based on dates

In general, Debian packages should use the same version numbers as the upstream sources. However, upstream version numbers based on some date formats (sometimes used for development or “snapshot” releases) will not be ordered correctly by the package management software. For example, `dpkg` will consider “96May01” to be greater than “96Dec24”.

To prevent having to use epochs for every new upstream version, the date-based portion of any upstream version number should be given in a way that sorts correctly: four-digit year first, followed by a two-digit numeric month, followed by a two-digit numeric day-of-month, possibly with punctuation between the components.

Native Debian packages (i.e., packages which have been written especially for Debian) whose version numbers include dates should also follow these rules. If punctuation is desired between the date components, remember that hyphen (–) cannot be used in native version numbers. Period (.) is normally a good choice.

3.2.2 Uniqueness of version numbers

The part of the version number after the epoch must not be reused for a version of the package with different contents once the package has been accepted into the archive, even if the version of the package previously using that part of the version number is no longer present in any archive suites.

This uniqueness requirement applies to the version numbers of source packages and of binary packages, even if the source package producing a given binary package changes. Thus the version numbers which a binary package must not reuse includes the version numbers of any versions of the binary package ever accepted into the archive, under any source package.

Additionally, for non-native packages, the upstream version must not be reused for different upstream source code, so that for each source package name and upstream version number there exists exactly one original source archive contents (see *Files*).

The reason for these restrictions is as follows. Epochs are not included in the names of the files that compose source packages, or in the filenames of binary packages, so reusing a version number, even if the epoch differs, results in identically named files with different contents. This can cause various problems.

If you find yourself wanting to reuse the part of a version number after the epoch, you can just increment the Debian revision, which doesn’t need to start at 1 or be consecutive.

3.3 The maintainer of a package

Every package must have a maintainer, except for orphaned packages as described below. The maintainer may be one person or a group of people reachable from a common email address, such as a mailing list. The maintainer is responsible for maintaining the Debian packaging files, evaluating and responding appropriately to reported bugs, uploading new versions of the package (either directly or through a sponsor), ensuring that the package is placed in the appropriate archive area and included in Debian releases as appropriate for the stability and utility of the package, and requesting removal of the package from the Debian distribution if it is no longer useful or maintainable.

The maintainer must be specified in the `Maintainer` control field with their correct name and a working email address. The email address given in the `Maintainer` control field must accept mail from those role accounts in Debian used to send automated mails regarding the package. This includes non-spam mail from the bug-tracking system, all mail from the Debian archive maintenance software, and other role accounts or automated processes that are commonly agreed on by the project.¹ If one person or team maintains several packages, they should use the same form of their name and email address in the `Maintainer` fields of those packages.

¹ A sample implementation of such a whitelist written for the Mailman mailing list management software is used for mailing lists hosted by <https://alioth-lists.debian.net/>.

The format of the `Maintainer` control field is described in [Maintainer](#).

If the maintainer of the package is a team of people with a shared email address, the `Uploaders` control field must be present and must contain at least one human with their personal email address. See [Uploaders](#) for the syntax of that field.

An orphaned package is one with no current maintainer. Orphaned packages should have their `Maintainer` control field set to `Debian QA Group <packages@qa.debian.org>`. These packages are considered maintained by the Debian project as a whole until someone else volunteers to take over maintenance.²

3.4 The description of a package

Every Debian package must have a `Description` control field which contains a synopsis and extended description of the package. Technical information about the format of the `Description` field is in [Description](#).

The description should describe the package (the program) to a user (system administrator) who has never met it before so that they have enough information to decide whether they want to install it. This description should not just be copied verbatim from the program's documentation.

Put important information first, both in the synopsis and extended description. Sometimes only the first part of the synopsis or of the description will be displayed. You can assume that there will usually be a way to see the whole extended description.

The description should also give information about the significant dependencies and conflicts between this package and others, so that the user knows why these dependencies and conflicts have been declared.

Instructions for configuring or using the package should not be included (that is what installation scripts, manual pages, info files, etc., are for). Copyright statements and other administrivia should not be included either (that is what the copyright file is for).

3.4.1 The single line synopsis

The single line synopsis should be kept brief—certainly under 80 characters.

Do not include the package name in the synopsis line. The display software knows how to display this already, and you do not need to state it. Remember that in many situations the user may only see the synopsis line - make it as informative as you can.

3.4.2 The extended description

Do not try to continue the single line synopsis into the extended description. This will not work correctly when the full description is displayed, and makes no sense where only the summary (the single line synopsis) is available.

The extended description should describe what the package does and how it relates to the rest of the system (in terms of, for example, which subsystem it is which part of).

The description field needs to make sense to anyone, even people who have no idea about any of the things the package deals with.³

3.5 Dependencies

Every package must specify the dependency information about other packages that are required for the first to work correctly.

For example, a dependency entry must be provided for any shared libraries required by a dynamically-linked executable binary in a package.

² The detailed procedure for gracefully orphaning a package can be found in the Debian Developer's Reference (see [Related documents](#)).

³ The blurb that comes with a program in its announcements and/or `README` files is rarely suitable for use in a description. It is usually aimed at people who are already in the community where the package is used.

Packages are not required to declare any dependencies they have on other packages which are marked `Essential` (see below), and should not do so unless they depend on a particular version of that package.⁴

Sometimes, unpacking one package requires that another package be first unpacked *and* configured. In this case, the depending package must specify this dependency in the `Pre-Depends` control field.

You should not specify a `Pre-Depends` entry for a package before this has been discussed on the `debian-devel` mailing list and a consensus about doing that has been reached.

The format of the package interrelationship control fields is described in *Declaring relationships between packages*.

3.6 Virtual packages

Sometimes, there are several packages which offer more-or-less the same functionality. In this case, it's useful to define a *virtual package* whose name describes that common functionality. (The virtual packages only exist logically, not physically; that's why they are called *virtual*.) The packages with this particular function will then *provide* the virtual package. Thus, any other package requiring that function can simply depend on the virtual package without having to specify all possible packages individually.

All packages should use virtual package names where appropriate, and arrange to create new ones if necessary. They should not use virtual package names (except privately, amongst a cooperating group of packages) unless they have been agreed upon and appear in the list of virtual package names. (See also *Virtual packages - Provides*)

The latest version of the authoritative list of virtual package names can be found in the `debian-policy` package. It is also available from the Debian web mirrors at <https://www.debian.org/doc/packaging-manuals/virtual-package-names-list.yaml>.

The procedure for updating the list is described in the preface to the list.

3.7 Base system

The `base system` is a minimum subset of the Debian system that is installed before everything else on a new system. Only very few packages are allowed to form part of the base system, in order to keep the required disk usage very small.

The base system consists of all those packages with priority `required` or `important`. Many of them will be tagged `essential` (see below).

3.8 Essential packages

Essential is defined as the minimal set of functionality that must be available and usable on the system at all times, even when packages are in the “Unpacked” state. Packages are tagged `essential` for a system using the `Essential` control field. The format of the `Essential` control field is described in *Essential*.

Since these packages cannot be easily removed (one has to specify an extra *force option* to `dpkg` to do so), this flag must not be used unless absolutely necessary. A shared library package must not be tagged `essential`; dependencies will prevent its premature removal, and we need to be able to remove it when it has been superseded.

Since `dpkg` will not prevent upgrading of other packages while an `essential` package is in an unconfigured state, all `essential` packages must supply all of their core functionality even when unconfigured after being configured at least once. If the package cannot satisfy this requirement it must not be tagged as `essential`, and any packages depending on this package must instead have explicit dependency fields as appropriate.

⁴ Essential is needed in part to avoid unresolvable dependency loops on upgrade. If packages add unnecessary dependencies on packages in this set, the chances that there **will** be an unresolvable dependency loop caused by forcing these Essential packages to be configured first before they need to be is greatly increased. It also increases the chances that frontends will be unable to **calculate** an upgrade path, even if one exists.

Also, functionality is rarely ever removed from the Essential set, but *packages* have been removed from the Essential set when the functionality moved to a different package. So depending on these packages *just in case* they stop being essential does way more harm than good.

Maintainers should take great care in adding any programs, interfaces, or functionality to `essential` packages. Packages may assume that functionality provided by `essential` packages is always available without declaring explicit dependencies, which means that removing functionality from the Essential set is very difficult and is almost never done. Any capability added to an `essential` package therefore creates an obligation to support that capability as part of the Essential set in perpetuity.

You must not tag any packages `essential` before this has been discussed on the `debian-devel` mailing list and a consensus about doing that has been reached.

3.9 Maintainer Scripts

The package installation scripts should avoid producing output which is unnecessary for the user to see and should rely on `dpkg` to stave off boredom on the part of a user installing many packages. This means, amongst other things, not passing the `--verbose` option to `update-alternatives`.

Errors which occur during the execution of an installation script must be checked and the installation must not continue after an error.

Note that in general *Scripts* applies to package maintainer scripts, too.

You should not use `dpkg-divert` on a file belonging to another package without consulting the maintainer of that package first. When adding or removing diversions, package maintainer scripts must provide the `--package` flag to `dpkg-divert` and must not use `--local`.

All packages which supply an instance of a common command name (or, in general, filename) should generally use `update-alternatives` so that they can be installed together. If `update-alternatives` is not used, then each package must use `Conflicts` to ensure that other packages are removed. (In this case, it may be appropriate to specify a conflict against earlier versions of something that previously did not use `update-alternatives`; this is an exception to the usual rule that versioned conflicts should be avoided.)

Diversions are primarily intended as a tool for local administrators and local packages to override the behavior of Debian. While there are some circumstances where one Debian package may need to divert a file installed by another Debian package, such circumstances are rare. Maintainers should strongly prefer using other overriding mechanisms, instead of diversions, whenever those other mechanisms are sufficient to accomplish the same goal. In other words, diversions in packages should be considered a last resort. Diversion of a file in one Debian package by another Debian package should be coordinated between the maintainers of those packages.

One specific case of this rule is that configuration files used by `systemd` components, such as `units`, `udev rules`, `tmpfiles.d`, `modules-load.d`, `sysusers` and other such files, including those specific to `systemd` daemons (e.g.: `/etc/systemd/system.conf`), must not be diverted by any Debian package. Instead, use `masking` and `drop-ins`.

Alternatives must not be used for `systemd` configuration files. The alternatives system does not know how to apply changes to services when updating alternatives, so the resulting behavior would be confusing and unpredictable. Instead, `aliases` can be used to provide alternative implementations of the same named unit.

3.9.1 Prompting in maintainer scripts

Package maintainer scripts may prompt the user if necessary. Prompting must be done by communicating through a program, such as `debconf`, which conforms to the Debian Configuration Management Specification, version 2 or higher.

Packages which are essential, or which are dependencies of essential packages, may fall back on another prompting method if no such interface is available when they are executed.

The Debian Configuration Management Specification is included in the `debconf_specification` files in the `debian-policy` package. It is also available from the Debian web mirrors at https://www.debian.org/doc/packaging-manuals/debconf_specification.html.

Packages which use the Debian Configuration Management Specification may contain the additional package metadata files `config` and `templates`. `config` is an additional maintainer script used for package configuration, and `templates`

contains templates used for user prompting. The `config` script might be run before the `preinst` script and before the package is unpacked or any of its dependencies or pre-dependencies are satisfied. Therefore it must work using only the tools present in *essential* packages.⁵

Packages which use the Debian Configuration Management Specification must allow for translation of their user-visible messages by using a gettext-based system such as the one provided by the `po-debconf` package.

Packages should try to minimize the amount of prompting they need to do, and they should ensure that the user will only ever be asked each question once. This means that packages should try to use appropriate shared configuration files (such as `/etc/papersize` and `/etc/news/server`), and shared debconf variables rather than each prompting for their own list of required pieces of information.

It also means that an upgrade should not ask the same questions again, unless the user has used `dpkg --purge` to remove the package's configuration. The answers to configuration questions should be stored in an appropriate place in `/etc` so that the user can modify them, and how this has been done should be documented.

If a package has a vitally important piece of information to pass to the user (such as “don't run me as I am, you must edit the following configuration files first or you risk your system emitting badly-formatted messages”), it should display this in the `config` or `postinst` script and prompt the user to hit return to acknowledge the message. Copyright messages do not count as vitally important (they belong in `/usr/share/doc/PACKAGE/copyright`); neither do instructions on how to use a program (these should be in on-line documentation, where all the users can see them).

Any necessary prompting should almost always be confined to the `config` or `postinst` script. If it is done in the `postinst`, it should be protected with a conditional so that unnecessary prompting doesn't happen if a package's installation fails and the `postinst` is called with `abort-upgrade`, `abort-remove` or `abort-deconfigure`.

⁵ Debconf or another tool that implements the Debian Configuration Management Specification will also be installed, and any versioned dependencies on it will be satisfied before preconfiguration begins.

SOURCE PACKAGES

A Debian source package contains the source material used to construct one or more *binary packages*. A source package consists of a `.dsc` file (see *Debian source package control files – .dsc*), one or more compressed tar files, and possibly other files depending on the type and format of source package. Binary packages are constructed from the source package via a build process defined by `debian/rules` and other files in the `debian` directory of the unpacked source package.

Debian source packages are classified as *native* or *non-native*.

A native source package is one that does not distinguish between Debian packaging releases and upstream releases. A native source package contains a single tar file of source material, and the versioning does not have a Debian-specific component. Native packages are normally (but not exclusively) used for software that has no independent existence outside of Debian, such as software written specifically to be a Debian package.

A non-native source package separates the upstream release from the Debian packaging and any Debian-specific changes. The source in a non-native source package is divided into one or more upstream tar files plus a collection of Debian-specific files. (Depending on the format of the source package, those Debian-specific files may come in the form of another tar file or in the form of a compressed diff.) The version of a non-native package has an upstream component and a Debian component, and there may be multiple Debian package versions associated with a single upstream release version and sharing the same upstream source tar files.

Most source packages in Debian are non-native.

4.1 Standards conformance

Source packages should specify the most recent version number of this policy document with which your package complied when it was last updated.

The version is specified in the `Standards-Version` control field. The format of the `Standards-Version` field is described in *Standards-Version*.

For a package to have an old `Standards-Version` value is not *itself* a bug, however. It just means that no-one has yet reviewed the package with changes to the standards in mind.

When updating existing packaging, the `Standards-Version` must not be updated except after reviewing the changes between the old and the new versions of the standards and updating your package if necessary (the *Upgrading checklist* can help with this task).

A very old `Standards-Version` can mean that infelicities in the package are likely. It is recommended that each package be reviewed at least once per Debian release, so a `Standards-Version` older than the previous Debian release is indicative of work (if only review work) that needs doing.

4.2 Package relationships

Source packages should specify which binary packages they require to be installed or not to be installed in order to build correctly. For example, if building a package requires a certain compiler, then the compiler should be specified as a build-time dependency.

It is not necessary to explicitly specify build-time relationships on a minimal set of packages that are always needed to compile, link and put in a Debian package a standard “Hello World!” program written in C or C++. The required packages are called *build-essential*, and an informational list can be found in `/usr/share/doc/build-essential/list` (which is contained in the *build-essential* package).¹

When specifying the set of build-time dependencies, one should list only those packages explicitly required by the build. It is not necessary to list packages which are required merely because some other package in the list of build-time dependencies depends on them.²

If build-time dependencies are specified, it must be possible to build the package and produce working binaries on a system with only essential and build-essential packages installed and also those required to satisfy the build-time relationships (including any implied relationships). In particular, this means that version clauses should be used rigorously in build-time relationships so that one cannot produce bad or inconsistently configured packages when the relationships are properly satisfied.

Declaring relationships between packages explains the technical details.

4.3 Changes to the upstream sources

If changes to the source code are made that are not specific to the needs of the Debian system, they should be sent to the upstream authors in whatever form they prefer so as to be included in the upstream version of the package.

If you need to configure the package differently for Debian or for Linux, and the upstream source doesn’t provide a way to do so, you should add such configuration facilities (for example, a new `autoconf` test or `#define`) and send the patch to the upstream authors, with the default set to the way they originally had it. You can then easily override the default in your `debian/rules` or wherever is appropriate.

You should make sure that the `configure` utility detects the correct architecture specification string (refer to *Architecture specification strings* for details).

If your package includes the scripts `config.sub` and `config.guess`, you should arrange for the versions provided by the package `autotools-dev` be used instead (see `autotools-dev` documentation for details how to achieve that). This ensures that these files can be updated distribution-wide at build time when introducing new architectures.

If you need to edit a `Makefile` where GNU-style `configure` scripts are used, you should edit the `.in` files rather than editing the `Makefile` directly. This allows the user to reconfigure the package if necessary. You should *not* configure the package and edit the generated `Makefile`! This makes it impossible for someone else to later reconfigure the package without losing the changes you made.

¹ Rationale:

- This allows maintaining the list separately from the policy documents (the list does not need the kind of control that the policy documents do).
- Having a separate package allows one to install the build-essential packages on a machine, as well as allowing other packages such as tasks to require installation of the build-essential packages using the `depends` relation.
- The separate package allows bug reports against the list to be categorized separately from the policy management process in the BTS.

² The reason for this is that dependencies change, and you should list all those packages, and *only* those packages that *you* need directly. What others need is their business. For example, if you only link against `libimlib`, you will need to `build-depend` on `libimlib2-dev` but not against any `libjpeg*` packages, even though `libimlib2-dev` currently depends on them: installation of `libimlib2-dev` will automatically ensure that all of its run-time dependencies are satisfied.

4.4 Debian changelog: `debian/changelog`

Every source package must include the Debian changelog file, `debian/changelog`. Changes in the Debian version of the package should be briefly explained in this file.³ This includes modifications made in the Debian package compared to the upstream one as well as other changes and updates to the package.⁴

The format of the `debian/changelog` allows the package building tools to discover which version of the package is being built and find out other release-specific information.

That format is a series of entries like this:

```
package (version) distribution(s); urgency=urgency
[optional blank line(s), stripped]
* change details
more change details
[blank line(s), included in output of dpkg-parsechangelog]
* even more change details
[optional blank line(s), stripped]
-- maintainer name <email address>[two spaces]  date
```

`package` and `version` are the source package name and version number.

`distribution(s)` lists the distributions where this version should be installed when it is uploaded - it is copied to the `Distribution` field in the `.changes` file. See [Distribution](#).

`urgency` is the value for the `Urgency` field in the `.changes` file for the upload (see [Urgency](#)). It is not possible to specify an urgency containing commas; commas are used to separate `keyword=value` settings in the `dpkg` changelog format (though there is currently only one useful keyword, `urgency`).

The change details may in fact be any series of lines starting with at least two spaces, but conventionally each change starts with an asterisk and a separating space and continuation lines are indented so as to bring them in line with the start of the text above. Blank lines may be used here to separate groups of changes, if desired.

If this upload resolves bugs recorded in the Bug Tracking System (BTS), they may be automatically closed on the inclusion of this package into the Debian archive by including the string: `closes: Bug#nnnnn` in the change details, where `#nnnnn` is the bug number.⁵ This information is conveyed via the `Closes` field in the `.changes` file (see [Closes](#)).

The maintainer name and email address used in the changelog should be the details of the person who prepared this release of the package. They are *not* necessarily those of the uploader or usual package maintainer.⁶ The information here will be copied to the `Changed-By` field in the `.changes` file (see [Changed-By](#)), and then later used to send an acknowledgement when the upload has been installed.

The date has the following format⁷ (compatible and with the same semantics of RFC 2822 and RFC 5322):

³ Mistakes in changelogs are usually best rectified by making a new changelog entry rather than “rewriting history” by editing old changelog entries.

⁴ Although there is nothing stopping an author who is also the Debian maintainer from using this changelog for all their changes, it will have to be renamed if the Debian and upstream maintainers become different people. In such a case, however, it might be better to maintain the package as a non-native package.

⁵ To be precise, the string should match the following Perl regular expression:

```
/closes:\s*(?:bug)?\#\s?\d+(?:,\s*(?:bug)?\#\s?\d+)*\s*/i
```

That is: The string should consist of the word `closes:` followed by a comma-separated list of bug numbers. Bug numbers may be preceded by the word `bug` and/or a `#` sign, as in `Closes: 42, bug#43, #44, bug 45`.

The list of bug numbers may span multiple lines.

All of the bug numbers listed will be closed by the archive maintenance software (`dak`) using the version of the changelog entry.

The words `closes:` and `bug` are not case sensitive.

⁶ In the case of a sponsored upload, the uploader signs the files, but the changelog maintainer name and address are those of the person who prepared this release. If the preparer of the release is not one of the usual maintainers of the package (as listed in the [Maintainer](#) or [Uploaders](#) control fields of the package), the first line of the changelog is conventionally used to explain why a non-maintainer is uploading the package. The Debian Developer's Reference (see [Related documents](#)) documents the conventions used.

⁷ This is the same as the format generated by `date -R`.


```
day-of-week, dd month yyyy hh:mm:ss +zzzz
```

where:

- `day-of-week` is one of: Mon, Tue, Wed, Thu, Fri, Sat, Sun
- `dd` is a one- or two-digit day of the month (01-31)
- `month` is one of: Jan, Feb, Mar, Apr, May, Jun, Jul, Aug, Sep, Oct, Nov, Dec
- `yyyy` is the four-digit year (e.g. 2010)
- `hh` is the two-digit hour (00-23)
- `mm` is the two-digit minutes (00-59)
- `ss` is the two-digit seconds (00-60)
- **+zzzz or -zzzz is the time zone offset from Coordinated Universal Time (UTC).** “+” indicates that the time is ahead of (i.e., east of) UTC and “-” indicates that the time is behind (i.e., west of) UTC. The first two digits indicate the hour difference from UTC and the last two digits indicate the number of additional minutes difference from UTC. The last two digits must be in the range 00-59.

The first “title” line with the package name must start at the left hand margin. The “trailer” line with the maintainer and date details must be preceded by exactly one space. The maintainer details and the date must be separated by exactly two spaces.

The entire changelog must be encoded in UTF-8.

For more information on placement of the changelog files within binary packages, please see [Changelog files and release notes](#).

4.5 Copyright: `debian/copyright`

Every package must be accompanied by a verbatim copy of its distribution license(s) in the file `/usr/share/doc/PACKAGE/copyright`.

This file is often required to contain a verbatim copy of the package’s copyright information, too; see [Copyright information](#) and [Copyright considerations](#) for details, and for further considerations related to copyrights for packages.

4.6 Error trapping in makefiles

When `make` invokes a command in a makefile (including your package’s upstream makefiles and `debian/rules`), it does so using `sh`. This means that `sh`’s usual bad error handling properties apply: if you include a miniature script as one of the commands in your makefile you’ll find that if you don’t do anything about it then errors are not detected and `make` will blithely continue after problems.

Every time you put more than one shell command (this includes using a loop) in a makefile command you must make sure that errors are trapped. For simple compound commands, such as changing directory and then running a program, using `&&` rather than semicolon as a command separator is sufficient. For more complex commands including most loops and conditionals you should include a separate `set -e` command at the start of every makefile command that’s actually one of these miniature shell scripts.

4.7 Time Stamps

Maintainers should preserve the modification times of the upstream source files in a package, as far as is reasonably possible.⁸

4.8 Restrictions on objects in source packages

The source package must not contain device special files, sockets, or setuid or setgid files.⁹

4.9 Main building script: `debian/rules`

This file must be an executable makefile. It contains the package-specific recipes for compiling the source (if required) and constructing one or more binary packages.

`debian/rules` must start with the line `#!/usr/bin/make -f`, so that it can be invoked by saying its name rather than invoking `make` explicitly. That is, invoking either of `make -f debian/rules args...` or `./debian/rules args...` must result in identical behavior.

The recommended way to implement the build process of a Debian package, in the absence of a good reason to use a different approach, is the `dh` tool. This includes the contents of the `debian/rules` building script. `dh` is the most common packaging helper tool in Debian. Using it will usually save effort in complying with the rules in this document, because `dh` will automatically implement many of them without requiring explicit instructions.

There are sometimes good reasons to use a different approach. For example, the standard tools for packaging software written in some languages may use another tool; some rarer packaging patterns, such as multiple builds of the same software with different options, are easier to express with other tools; and a packager working on a different packaging helper might want to use their tool. The recommendation to use `dh` does not always apply, and use of `dh` is not required.

For more information about how to use `dh`, see the documentation in the `debhelper` package, most notably the `dh(1)` manual page.

The following targets are required and must be implemented by `debian/rules`: `clean`, `binary`, `binary-arch`, `binary-indep`, `build`, `build-arch` and `build-indep`. These are the targets called by `dpkg-buildpackage`.

Since an interactive `debian/rules` script makes it impossible to auto-compile that package and also makes it hard for other people to reproduce the same binary package, all required targets must be non-interactive. It also follows that any target that these targets depend on must also be non-interactive.

The package build should be as verbose as reasonably possible, except where the `terse` tag is included in `DEB_BUILD_OPTIONS` (see [debian/rules](#) and [DEB_BUILD_OPTIONS](#)). This makes life easier for porters and bug squashers more generally, who can look at build logs for possible problems. To accomplish this, `debian/rules` should pass to the commands it invokes options that cause them to produce verbose output. For example, the `build` target should pass `--disable-silent-rules` to any configure scripts. See also [Binaries](#).

Except for packages in the non-free archive with the `Autobuild` control field unset or set to `no`, required targets must not attempt network access, except, via the loopback interface, to services on the build host that have been started by the build.

Required targets must not attempt to write outside of the unpacked source package tree. There are two exceptions. Firstly, the `binary` targets may write the binary packages to the parent directory of the unpacked source package tree. Secondly, required targets may write to `/tmp`, `/var/tmp` and to the directory specified by the `TMPDIR` environment variable, but must not depend on the contents of any of these.

⁸ The rationale is that there is some information conveyed by knowing the age of the file, for example, you could recognize that some documentation is very old by looking at the modification time, so it would be nice if the modification time of the upstream source would be preserved.

⁹ Setgid directories are allowed.

This restriction is intended to prevent source package builds creating and depending on state outside of themselves, thus affecting multiple independent rebuilds. In particular, the required targets must not attempt to write into `HOME`.

The targets are as follows:

build (required)

The `build` target should perform all the configuration and compilation of the package. If a package has an interactive pre-build configuration routine, the Debian source package must either be built after this has taken place (so that the binary package can be built without rerunning the configuration) or the configuration routine modified to become non-interactive. (The latter is preferable if there are architecture-specific features detected by the configuration routine.)

For some packages, notably ones where the same source tree is compiled in different ways to produce two binary packages, the `build` target does not make much sense. For these packages it is good enough to provide two (or more) targets (`build-a` and `build-b` or whatever) for each of the ways of building the package, and a `build` target that does nothing. The `binary` target will have to build the package in each of the possible ways and make the binary package out of each.

The `build` target must not do anything that might require root privilege.

The `build` target may need to run the `clean` target first - see below.

When a package has a configuration and build routine which takes a long time, or when the makefiles are poorly designed, or when `build` needs to run `clean` first, it is a good idea to `touch build` when the build process is complete. This will ensure that if `debian/rules build` is run again it will not rebuild the whole program.¹⁰

build-arch (required), build-indep (required)

The `build-arch` target must perform all the configuration and compilation required for producing all architecture-dependent binary packages (those packages for which the body of the `Architecture` field in `debian/control` is not `all`). Similarly, the `build-indep` target must perform all the configuration and compilation required for producing all architecture-independent binary packages (those packages for which the body of the `Architecture` field in `debian/control` is `all`). The `build` target should either depend on those targets or take the same actions as invoking those targets would perform.¹¹

The `build-arch` and `build-indep` targets must not do anything that might require root privilege.

binary (required), binary-arch (required), binary-indep (required)

The `binary` target must be all that is necessary for the user to build the binary package(s) produced from this source package. It is split into two parts: `binary-arch` builds the binary packages which are specific to a particular architecture, and `binary-indep` builds those which are not.

`binary` may be (and commonly is) a target with no commands which simply depends on `binary-arch` and `binary-indep`.

Both `binary-*` targets should depend on the `build` target, or on the appropriate `build-arch` or `build-indep` target, so that the package is built if it has not been already. It should then create the relevant binary package(s), using `dpkg-gencontrol` to make their control files and `dpkg-deb` to build them and place them in the parent of the top level directory.

Both the `binary-arch` and `binary-indep` targets *must* exist. If one of them has nothing to do (which will always be the case if the source generates only a single binary package, whether architecture-dependent or not), it must still exist and must always succeed.

The `binary` targets may need to be invoked as root depending on the value of the *Rules-Requires-Root* field.¹²

¹⁰ Another common way to do this is for `build` to depend on `build-stamp` and to do nothing else, and for the `build-stamp` target to do the building and to `touch build-stamp` on completion. This is especially useful if the build routine creates a file or directory called `build`; in such a case, `build` will need to be listed as a phony target (i.e., as a dependency of the `.PHONY` target). See the documentation of `make` for more information on phony targets.

¹¹ This split allows binary-only builds to not install the dependencies required for the `build-indep` target and skip any resource-intensive build tasks that are only required when building architecture-independent binary packages.

¹² The `fakeroot` package often allows one to build a package correctly even without being root.

clean (required)

This must undo any effects that the `build` and `binary` targets may have had, except that it should leave alone any output files created in the parent directory by a run of a `binary` target.

If a `build` file is touched at the end of the `build` target, as suggested above, it should be removed as the first action that `clean` performs, so that running `build` again after an interrupted `clean` doesn't think that everything is already done.

The `clean` target may need to be invoked as root if `binary` has been invoked since the last `clean`, or if `build` has been invoked as root (since `build` may create directories, for example).

The `clean` target cannot be used to remove files in the source tree that are not compatible with the DFSG. This is because the files would remain in the upstream tarball, and thus in the source package, so the source package would continue to violate DFSG. Instead, the upstream source should be repacked to remove those files.

patch (optional)

This target performs whatever additional actions are required to make the source ready for editing (unpacking additional upstream archives, applying patches, etc.). It is recommended to be implemented for any package where `dpkg-source -x` does not result in source ready for additional modification. See *Source package handling: [debian/README.source](#)*.

The `build`, `binary` and `clean` targets must be invoked with the current directory being the package's top-level directory.

Additional targets may exist in `debian/rules`, either as published or undocumented interfaces or for the package's internal use.

The architectures we build on and build for are determined by `make` variables using the utility `dpkg-architecture`. You can determine the Debian architecture and the GNU style architecture specification string for the build architecture as well as for the host architecture. The build architecture is the architecture on which `debian/rules` is run and the package build is performed. The host architecture is the architecture on which the resulting package will be installed and run. The target architecture is the architecture of the packages that the compiler currently being built will generate. These are normally the same, but may be different in the case of cross-compilation (building packages for one architecture on machines of a different architecture), building a cross-compiler (a compiler package that will generate objects for one architecture, built on a machine of a different architecture) or a Canadian cross-compiler (a compiler that will generate objects for one architecture, built on a machine of a different architecture, that will run on yet a different architecture).

Here is a list of supported `make` variables:

- `DEB_*_ARCH` (the Debian architecture)
- `DEB_*_ARCH_CPU` (the Debian CPU name)
- `DEB_*_ARCH_BITS` (the Debian CPU pointer size in bits)
- `DEB_*_ARCH_ENDIAN` (the Debian CPU endianness)
- `DEB_*_ARCH_OS` (the Debian System name)
- `DEB_*_GNU_TYPE` (the GNU style architecture specification string)
- `DEB_*_GNU_CPU` (the CPU part of `DEB_*_GNU_TYPE`)
- `DEB_*_GNU_SYSTEM` (the System part of `DEB_*_GNU_TYPE`)

where `*` is either `BUILD` for specification of the build architecture, `HOST` for specification of the host architecture or `TARGET` for specification of the target architecture.

Backward compatibility can be provided in the rules file by setting the needed variables to suitable default values; please refer to the documentation of `dpkg-architecture` for details.

It is important to understand that the `DEB_*_ARCH` string only determines which Debian architecture we are building on or for. It should not be used to get the CPU or system information; the `DEB_*_ARCH_CPU` and `DEB_*_ARCH_OS` variables should be used for that. GNU style variables should generally only be used with upstream build systems.

The builder may set `DEB_RULES_REQUIRES_ROOT` environment variable when calling any of the mandatory targets as defined in *Rules-Requires-Root*. If the variable is not set, the package must behave as if it was set to `binary-targets`.

4.9.1 `debian/rules` and `DEB_BUILD_OPTIONS`

Supporting the standardized environment variable `DEB_BUILD_OPTIONS` is recommended. This variable can contain several flags to change how a package is compiled and built. Each flag must be in the form `flag` or `flag=options`. If multiple flags are given, they must be separated by whitespace.¹³ flag must start with a lowercase letter (`a-z`) and consist only of lowercase letters, numbers (`0-9`), and the characters `-` and `_` (hyphen and underscore). options must not contain whitespace. The same tag should not be given multiple times with conflicting values. Package maintainers may assume that `DEB_BUILD_OPTIONS` will not contain conflicting tags.

The meaning of the following tags has been standardized:

nocheck

This tag says to not run any build-time test suite provided by the package.

nodoc

This tag says to skip any build steps that only generate package documentation. Files required by other sections of Debian Policy, such as copyright and changelog files, must still be generated and put in the package, but other generated documentation such as `help2man`-generated pages, Doxygen-generated API documentation, or info pages generated from Texinfo sources should be skipped if possible. This option does not change the set of binary packages generated by the source package, but documentation-only binary packages may be nearly empty when built with this option.

noopt

The presence of this tag means that the package should be compiled with a minimum of optimization. For C programs, it is best to add `-O0` to `CFLAGS` (although this is usually the default). Some programs might fail to build or run at this level of optimization; it may be necessary to use `-O1`, for example.

nostrip

This tag means that the debugging symbols should not be stripped from the binary during installation, so that debugging information may be included in the package.

parallel=n

This tag means that the package should be built using up to `n` parallel processes if the package build system supports this.¹⁴ If the package build system does not support parallel builds, this string must be ignored. If the package build system only supports a lower level of concurrency than `n`, the package should be built using as many parallel processes as the package build system supports. It is up to the package maintainer to decide whether the package build times are long enough and the package build system is robust enough to make supporting parallel builds worthwhile.

terse

This tag means that the package build will be less verbose than default. For example, `debian/rules` might pass options to the package's configure script that cause the compiler to produce less output.

Unknown flags must be ignored by `debian/rules`.

The following makefile snippet is an example of how one may implement the build options; you will probably have to massage this example in order to make it work for your package.

```
CFLAGS = -Wall -g
INSTALL = install
INSTALL_FILE = $(INSTALL) -p -o root -g root -m 644
INSTALL_PROGRAM = $(INSTALL) -p -o root -g root -m 755
```

(continues on next page)

¹³ Some packages support any delimiter, but whitespace is the easiest to parse inside a makefile and avoids ambiguity with flag values that contain commas.

¹⁴ Packages built with `make` can often implement this by passing the `-jn` option to `make`.

(continued from previous page)

```

INSTALL_SCRIPT = $(INSTALL) -p      -o root -g root  -m 755
INSTALL_DIR     = $(INSTALL) -p -d -o root -g root  -m 755

ifneq (,$(filter noopt,$(DEB_BUILD_OPTIONS)))
    CFLAGS += -O0
else
    CFLAGS += -O2
endif
ifeq (,$(filter nostrip,$(DEB_BUILD_OPTIONS)))
    INSTALL_PROGRAM += -s
endif
ifneq (,$(filter parallel=%, $(DEB_BUILD_OPTIONS)))
    NUMJOBS = $(patsubst parallel=%,%, $(filter parallel=%, $(DEB_BUILD_OPTIONS)))
    MAKEFLAGS += -j$(NUMJOBS)
endif

build:
    # ...
ifeq (,$(filter nocheck,$(DEB_BUILD_OPTIONS)))
    # Code to run the package test suite.
endif

```

4.9.2 debian/rules and Rules-Requires-Root

Depending on the value of the *Rules-Requires-Root* field, the package builder (e.g. `dpkg-buildpackage`) may run the `debian/rules` target as an unprivileged user and provide a *gain root command*. This command allows the `debian/rules` target to run particular subcommands under (fake)root.

The *gain root command* is passed to the build script via the `DEB_GAIN_ROOT_CMD` environment variable. The contents of this variable is a space separated list, the first entry of which is the command, and the proceeding entries of which are arguments to the command. The *gain root command* must be available via `PATH`. The *gain root command* must not rely on shell features because it will not necessarily be invoked via a shell.

The *gain root command* must not run interactively, including prompting for any user input. It must be possible to prepend the *gain root command* to an existing command and its arguments, without needing to alter or quote the existing command and its arguments. Furthermore, the *gain root command* must preserve all environment variables without the caller having to explicitly request any preservation.

The following are examples of valid *gain root* commands (in syntax of `sh`), assuming the tools used are available and properly configured:

```

# Command "sudo", with arguments "-nE" and "--"
export DEB_GAIN_ROOT_CMD='sudo -nE --'
# Command "fakeroot" with the single argument "--"
export DEB_GAIN_ROOT_CMD='fakeroot --'

```

Examples of valid use of the *gain root command*:

```

# sh-syntax (assumes set -e semantics for error handling)
$DEB_GAIN_ROOT_CMD some-cmd --which-requires-root

# perl
my @cmd = ('some-cmd', '--which-requires-root');

```

(continues on next page)

(continued from previous page)

```
unshift(@cmd, split(' ', $ENV{DEB_GAIN_ROOT_CMD})) if $ENV{DEB_GAIN_ROOT_CMD};
system(@cmd) == 0 or die("@cmd failed");
```

4.10 Variable substitutions: `debian/substvars`

When `dpkg-gencontrol` generates *binary package control files* (`DEBIAN/control`), it performs variable substitutions on its output just before writing it. Variable substitutions have the form `${variable}`. The optional file `debian/substvars` contains variable substitutions to be used; variables can also be set directly from `debian/rules` using the `-V` option to the source packaging commands, and certain predefined variables are also available.

The `debian/substvars` file is usually generated and modified dynamically by `debian/rules` targets, in which case it must be removed by the `clean` target.

See `deb-substvars(5)` for full details about source variable substitutions, including the format of `debian/substvars`.

4.11 Upstream source location: `debian/watch`

This is a configuration file for the `uscan` utility which defines how to automatically scan ftp or http sites for newly available updates of the package. This is also used by some Debian QA tools to help with quality control and maintenance of the distribution as a whole. If the upstream source of the package is available via a mechanism that `uscan` understands, including this configuration file is recommended.

If the upstream maintainer of the software provides OpenPGP signatures for new releases, including the information required for `uscan` to verify signatures for new upstream releases is also recommended. To do this, use the `pgpsig-urlmangle` option in `debian/watch` to specify the location of the upstream signature, and include the key or keys used to sign upstream releases in the Debian source package as `debian/upstream/signing-key.asc`.

For more information about `uscan` and these options, including how to generate the file containing upstream signing keys, see `uscan(1)`.

4.12 Generated files list: `debian/files`

This file is not a permanent part of the source tree; it is used while building packages to record which files are being generated. `dpkg-genchanges` uses it when it generates a `.changes` file.

It should not exist in a shipped source package, and so it (and any backup files or temporary files such as `files.new`)¹⁵ should be removed by the `clean` target. It may also be wise to ensure a fresh start by emptying or removing it at the start of the `binary` target.

When `dpkg-gencontrol` is run for a binary package, it adds an entry to `debian/files` for the `.deb` file that will be created when `dpkg-deb --build` is run for that binary package. So for most packages all that needs to be done with this file is to delete it in the `clean` target.

If a package upload includes files besides the source package and any binary packages whose control files were made with `dpkg-gencontrol` then they should be placed in the parent of the package's top-level directory and `dpkg-distaddfile` should be called to add the file to the list in `debian/files`.

¹⁵ `files.new` is used as a temporary file by `dpkg-gencontrol` and `dpkg-distaddfile` - they write a new version of `files` here before renaming it, to avoid leaving a corrupted copy if an error occurs.

4.13 Embedded code copies

Some software packages include in their distribution convenience copies of code from other software packages, generally so that users compiling from source don't have to download multiple packages. Debian packages should not make use of these convenience copies unless the included package is explicitly intended to be used in this way.¹⁶ If the included code is already in the Debian archive in the form of a library, the Debian packaging should ensure that binary packages reference the libraries already in Debian and the convenience copy is not used. If the included code is not already in Debian, it should be packaged separately as a prerequisite if possible.¹⁷

4.14 Source package handling: `debian/README.source`

If running `dpkg-source -x` on a source package doesn't produce the source of the package, ready for editing, and allow one to make changes and run `dpkg-buildpackage` to produce a modified package without taking any additional steps, creating a `debian/README.source` documentation file is recommended. This file should explain how to do all of the following:

1. Generate the fully patched source, in a form ready for editing, that would be built to create Debian packages. Doing this with a `patch` target in `debian/rules` is recommended; see *Main building script: `debian/rules`*.
2. Modify the source and save those modifications so that they will be applied when building the package.
3. Remove source modifications that are currently being applied when building the package.
4. Optionally, document what steps are necessary to upgrade the Debian source package to a new upstream version, if applicable.

This explanation should include specific commands and mention any additional required Debian packages. It should not assume familiarity with any specific Debian packaging system or patch management tools.

This explanation may refer to a documentation file installed by one of the package's build dependencies provided that the referenced documentation clearly explains these tasks and is not a general reference manual.

`debian/README.source` may also include any other information that would be helpful to someone modifying the source package. Even if the package doesn't fit the above description, maintainers are encouraged to document in a `debian/README.source` file any source package with a particularly complex or unintuitive source layout or build system (for example, a package that builds the same source multiple times to generate different binary packages).

4.15 Reproducibility

Packages should build reproducibly, which for the purposes of this document¹⁸ means that given

- a version of a source package unpacked at a given path;
- a set of versions of installed build dependencies;
- a set of environment variable values;
- a build architecture; and
- a host architecture,

repeatedly building the source package for the build architecture on any machine of the host architecture with those versions of the build dependencies installed and exactly those environment variable values set will produce bit-for-bit identical binary packages.

¹⁶ For example, parts of the GNU build system work like this.

¹⁷ Having multiple copies of the same code in Debian is inefficient, often creates either static linking or shared library conflicts, and, most importantly, increases the difficulty of handling security vulnerabilities in the duplicated code.

¹⁸ This is Debian's precisification of the reproducible-builds.org definition.

It is recommended that packages produce bit-for-bit identical binaries even if most environment variables and build paths are varied. It is intended for this stricter standard to replace the above when it is easier for packages to meet it.

4.16 Missing sources: `debian/missing-sources`

Sometimes upstream does not include the source code for some files in the upstream tarball. In order to satisfy the DFSG for packages in `main` or `contrib`, you should either:

1. repack the upstream tarball to include those sources; or
2. include a copy of the sources in the `debian/missing-sources` directory.

Package maintainers may optionally use the following convention to organize the contents of `debian/missing-sources`: for a sourceless file `foo` in the subdirectory `bar` of the upstream tarball, where the source of `foo` has extension `baz`, place the source at `debian/missing-sources/bar/foo.baz`. For example, according to this convention, the C source code of an executable `checksum/util` would be located at `debian/missing-sources/checksum/util.c`.

4.17 Vendor-specific patch series

Packages in the Debian archive using the 3.0 (quilt) source package format must not contain a non-default series file. That is, there must not exist a file `debian/patches/foo.series` for any `foo`.

CONTROL FILES AND THEIR FIELDS

The package management system manipulates data represented in a common format, known as *control data*, stored in *control files*. Control files are used for source packages, binary packages and the `.changes` files which control the installation of uploaded files.¹

5.1 Syntax of control files

A control file consists of one or more stanzas of fields.² The stanzas are separated by empty lines. Parsers may accept lines consisting solely of spaces and tabs as stanza separators, but control files should use empty lines. Some control files allow only one stanza; others allow several, in which case each stanza usually refers to a different package. (For example, in source packages, the first stanza refers to the source package, and later stanzas refer to binary packages generated from the source.) The ordering of the stanzas in control files is significant.

Each stanza consists of a series of data fields. Each field consists of the field name followed by a colon and then the data/value associated with that field. The field name is composed of US-ASCII characters excluding control characters, space, and colon (i.e., characters in the ranges U+0021 (!) through U+0039 (9), and U+003B (;) through U+007E (~), inclusive). Field names must not begin with the comment character (U+0023 #), nor with the hyphen character (U+002D -).

The field ends at the end of the line or at the end of the last continuation line (see below). Horizontal whitespace (spaces and tabs) may occur immediately before or after the value and is ignored there; it is conventional to put a single space after the colon. For example, a field might be:

```
Package: libc6
```

the field name is `Package` and the field value `libc6`.

Empty field values are only permitted in source package template control files (`debian/control`). Such fields are ignored.

A stanza must not contain more than one instance of a particular field name.

There are three types of fields:

simple

The field, including its value, must be a single line. Folding of the field is not permitted. This is the default field type if the definition of the field does not specify a different type.

folded

The value of a folded field is a logical line that may span several lines. The lines after the first are called continuation

¹ `dpkg`'s internal databases are in a similar format.

² The stanzas sometimes used to be referred to as paragraphs, but that caused confusion with text paragraphs in prose, so it is now considered a discouraged term.

lines and must start with a space or a tab. Whitespace, including any newlines, is not significant in the field values of folded fields.³

multiline

The value of a multiline field may comprise multiple continuation lines. The first line of the value, the part on the same line as the field name, often has special significance or may have to be empty. Other lines are added following the same syntax as the continuation lines of the folded fields. Whitespace, including newlines, is significant in the values of multiline fields.

Whitespace must not appear inside names (of packages, architectures, files or anything else) or version numbers, or between the characters of multi-character version relationships.

The presence and purpose of a field, and the syntax of its value, may differ between types of control files.

Field names are not case-sensitive, but it is usual to capitalize the field names using mixed case as shown below. Field values are case-sensitive unless the description of the field says otherwise.

Stanza separators (empty lines), and lines consisting only of U+0020 SPACE and U+0009 TAB, are not allowed within field values or between fields. Empty lines in field values are usually escaped by representing them by a U+0020 SPACE followed by a U+002E (.).

Lines starting with U+0023 (#), without any preceding whitespace, are comment lines that are only permitted in source package control files (`debian/control`). These comment lines are ignored, even between two continuation lines. They do not end logical lines.

All control files must be encoded in UTF-8.

5.2 Debian source package template control files – `debian/control`

The `debian/control` file contains the most vital (and version-independent) information about the source package and about the binary packages it creates.

The first stanza of the control file contains information about the source package in general. The subsequent stanzas each describe a binary package that the source tree builds. Each binary package built from this source package has a corresponding stanza, except for any automatically-generated debug packages that do not require one.

The fields in the source package stanza (the first one) are:

- *Source* (mandatory)
- *Maintainer* (mandatory)
- *Uploaders*
- *Section* (recommended)
- *Priority*
- *Build-Depends et al*
- *Standards-Version* (mandatory)
- *Homepage*
- *Version Control System (VCS) fields*
- *Testsuite*
- *Rules-Requires-Root*

The fields in the binary package stanzas are:

³ This folding method is similar to RFC 5322, allowing control files that contain only one stanza and no multiline fields to be read by parsers written for RFC 5322.

- *Package* (mandatory)
- *Architecture* (mandatory)
- *Section* (recommended)
- *Priority*
- *Essential*
- *Depends et al*
- *Description* (mandatory)
- *Homepage*
- *Built-Using*
- *Package-Type*

The syntax and semantics of the fields are described below.

These fields are used by `dpkg-gencontrol` to generate control files for binary packages (see below), by `dpkg-genchanges` to generate the `.changes` file to accompany the upload, and by `dpkg-source` when it creates the `.dsc` source control file as part of a source archive. Some fields are folded in `debian/control`, but not in any other control file. These tools are responsible for removing the line breaks from such fields when using fields from `debian/control` to generate other control files. They are also responsible for discarding empty fields.

The fields here may contain variable references - their values will be substituted by `dpkg-gencontrol`, `dpkg-genchanges` or `dpkg-source` when they generate output control files. See *Variable substitutions: debian/substvars* for details.

5.3 Debian binary package control files – `DEBIAN/control`

The `DEBIAN/control` file contains the most vital (and version-dependent) information about a binary package. It consists of a single stanza.

The fields in this file are:

- *Package* (mandatory)
- *Source*
- *Version* (mandatory)
- *Section* (recommended)
- *Priority* (recommended)
- *Architecture* (mandatory)
- *Essential*
- *Depends et al*
- *Installed-Size*
- *Maintainer* (mandatory)
- *Description* (mandatory)
- *Homepage*
- *Built-Using*

5.4 Debian source package control files – `.dsc`

This file consists of a single stanza, possibly surrounded by an OpenPGP signature. The fields of that stanza are listed below. Their syntax is described above, in *Syntax of control files*.

- *Format* (mandatory)
- *Source* (mandatory)
- *Binary*
- *Architecture*
- *Version* (mandatory)
- *Maintainer* (mandatory)
- *Uploaders*
- *Homepage*
- *Version Control System (VCS) fields*
- *Testsuite*
- *Dgit*
- *Git-Tag-Tagger*
- *Git-Tag-Info*
- *Standards-Version* (mandatory)
- *Build-Depends et al*
- *Package-List* (recommended)
- *Checksums-Sha1 and Checksums-Sha256* (mandatory)
- *Files* (mandatory)

The Debian source package control file is generated by `dpkg-source` when it builds the source archive, from other files in the source package, described above. When unpacking, it is checked against the files and directories in the other parts of the source package.

5.5 Debian upload changes control files – `.changes`

The `.changes` files are used by the Debian archive maintenance software to process updates to packages. They consist of a single stanza, possibly surrounded by an OpenPGP signature. That stanza contains information from the `debian/control` file and other data about the source package gathered via `debian/changelog` and `debian/rules`.

`.changes` files have a format version that is incremented whenever the documented fields or their meaning change. This document describes format 1.8.

The fields in this file are:

- *Format* (mandatory)
- *Date* (mandatory)
- *Source* (mandatory)
- *Binary* (mandatory in some cases)
- *Architecture* (mandatory)

- *Version* (mandatory)
- *Distribution* (mandatory)
- *Urgency* (recommended)
- *Maintainer* (mandatory)
- *Changed-By*
- *Git-Tag-Tagger*
- *Git-Tag-Info*
- *Description* (mandatory in some cases)
- *Closes*
- *Changes* (mandatory)
- *Checksums-Sha1 and Checksums-Sha256* (mandatory)
- *Files* (mandatory)

5.6 List of fields

5.6.1 Source

This field identifies the source package name.

In `debian/control` or a `.dsc` file, this field must contain only the name of the source package.

In a binary package control file or a `.changes` file, the source package name may be followed by a version number in parentheses.⁴ This version number may be omitted (and is, by `dpkg-gencontrol`) if it has the same value as the `Version` field of the binary package in question. The field itself may be omitted from a binary package control file when the source package has the same name and version as the binary package.

Package names (both source and binary, see *Package*) must consist only of lower case letters (a–z), digits (0–9), plus (+) and minus (–) signs, and periods (.). They must be at least two characters long and must start with an alphanumeric character.

5.6.2 Maintainer

The package maintainer's name and email address. The name must come first, then the email address inside angle brackets <> (in RFC822 format).

If the maintainer's name contains a full stop then the whole field will not work directly as an email address due to a misfeature in the syntax specified in RFC822; a program using this field as an address must check for this and correct the problem if necessary (for example by putting the name in round brackets and moving it to the end, and bringing the email address forward).

See *The maintainer of a package* for additional requirements and information about package maintainers.

5.6.3 Uploaders

List of the names and email addresses of co-maintainers of the package, if any. If the package has other maintainers besides the one named in the *Maintainer field*, their names and email addresses should be listed here. The format of each entry is the same as that of the *Maintainer* field, and multiple entries must be comma separated.

This is normally an optional field, but if the *Maintainer* control field names a group of people and a shared email address, the *Uploaders* field must be present and must contain at least one human with their personal email address.

⁴ It is customary to leave a space after the package name if a version number is specified.

The Uploaders field in `debian/control` can be folded.

5.6.4 Changed-By

The name and email address of the person who prepared this version of the package, usually a maintainer. The syntax is the same as for the *Maintainer field*.

5.6.5 Section

This field specifies an application area into which the package has been classified. See *Sections*.

When it appears in the `debian/control` file, it gives the value for the subfield of the same name in the `Files` field of the `.changes` file. It also gives the default for the same field in the binary packages.

5.6.6 Priority

This field represents how important it is that the user have the package installed. See *Priorities*.

When it appears in the `debian/control` file, it gives the value for the subfield of the same name in the `Files` field of the `.changes` file. It also gives the default for the same field in the binary packages.

5.6.7 Package

The name of the binary package.

Binary package names must follow the same syntax and restrictions as source package names. See *Source* for the details.

5.6.8 Architecture

Depending on context and the control file used, the `Architecture` field can include the following sets of values:

- A unique single word identifying a Debian machine architecture as described in *Architecture specification strings*.
- An architecture wildcard identifying a set of Debian machine architectures, see *Architecture wildcards*. `any` matches all Debian machine architectures and is the most frequently used.
- `all`, which indicates an architecture-independent package.
- `source`, which indicates a source package.

In the main `debian/control` file in the source package, this field may contain the special value `all`, the special architecture wildcard `any`, or a list of specific and wildcard architectures separated by spaces. If `all` or `any` appears, that value must be the entire contents of the field. Most packages will use either `all` or `any`.

Specifying a specific list of architectures indicates that the source will build an architecture-dependent package only on architectures included in the list. Specifying a list of architecture wildcards indicates that the source will build an architecture-dependent package on only those architectures that match any of the specified architecture wildcards. Specifying a list of architectures or architecture wildcards other than `any` is for the minority of cases where a program is not portable or is not useful on some architectures. Where possible, the program should be made portable instead.

In the Debian source package control file `.dsc`, this field contains a list of architectures and architecture wildcards separated by spaces. When the list contains the architecture wildcard `any`, the only other value allowed in the list is `all`.

The list may include (or consist solely of) the special value `all`. In other words, in `.dsc` files unlike the `debian/control`, `all` may occur in combination with specific architectures. The `Architecture` field in the Debian source package control file `.dsc` is generally constructed from the `Architecture` fields in the `debian/control` in the source package.

Specifying only `any` indicates that the source package isn't dependent on any particular architecture and should compile fine on any one. The produced binary package(s) will be specific to whatever the current build architecture is.

Specifying only `all` indicates that the source package will only build architecture-independent packages.

Specifying `any all` indicates that the source package isn't dependent on any particular architecture. The set of produced binary packages will include at least one architecture-dependent package and one architecture-independent package.

Specifying a list of architectures or architecture wildcards indicates that the source will build an architecture-dependent package, and will only work correctly on the listed or matching architectures. If the source package also builds at least one architecture-independent package, `all` will also be included in the list.

In a `.changes` file, the `Architecture` field lists the architecture(s) of the package(s) currently being uploaded. This will be a list; if the source for the package is also being uploaded, the special entry `source` is also present. `all` will be present if any architecture-independent packages are being uploaded. Architecture wildcards such as `any` must never occur in the `Architecture` field in the `.changes` file.

See *Main building script: `debian/rules`* for information on how to get the architecture for the build process.

5.6.9 Essential

This is a boolean field which may occur only in the control file of a binary package or in a binary package stanza of a source package template control file.

If set to `yes` then the package management system will refuse to remove the package (upgrading and replacing it is still possible). The other possible value is `no`, which is the same as not having the field at all.

5.6.10 Package interrelationship fields: `Depends`, `Pre-Depends`, `Recommends`, `Suggests`, `Breaks`, `Conflicts`, `Provides`, `Replaces`, `Enhances`

These fields describe the package's relationships with other packages. Their syntax and semantics are described in *Declaring relationships between packages*.

5.6.11 Standards-Version

The most recent version of the standards (the policy manual and associated texts) with which the package complies. See *Standards conformance*.

The version number has four components: major and minor version number and major and minor patch level. When the standards change in a way that requires every package to change the major number will be changed. Significant changes that will require work in many packages will be signaled by a change to the minor number. The major patch level will be changed for any change to the meaning of the standards, however small; the minor patch level will be changed when only cosmetic, typographical or other edits are made which neither change the meaning of the document nor affect the contents of packages.

Thus only the first three components of the policy version are significant in the *Standards-Version* control field, and so either these three components or all four components may be specified.⁵

udebs and source packages that only produce udebs do not use *Standards-Version*.

5.6.12 Version

The version number of a package. The format is: `[epoch:]upstream_version[-debian_revision]`.

The three components here are:

epoch

This is a single (generally small) unsigned integer. It may be omitted, in which case zero is assumed.

⁵ In the past, people specified the full version number in the *Standards-Version* field, for example "2.3.0.0". Since minor patch-level changes don't introduce new policy, it was thought it would be better to relax policy and only require the first 3 components to be specified, in this example "2.3.0". All four components may still be used if someone wishes to do so.

Epochs can help when the upstream version numbering scheme changes, but they must be used with care. You should not change the epoch, even in experimental, without getting consensus on debian-devel first.

upstream_version

This is the main part of the version number. It is usually the version number of the original (“upstream”) package from which the `.deb` file has been made, if this is applicable. Usually this will be in the same format as that specified by the upstream author(s); however, it may need to be reformatted to fit into the package management system’s format and comparison scheme.

The comparison behavior of the package management system with respect to the `upstream_version` is described below. The `upstream_version` portion of the version number is mandatory.

The `upstream_version` must contain only alphanumerics⁶ and the characters `.` `+` `-` `~` (full stop, plus, hyphen, tilde) and should start with a digit. If there is no `debian_revision` then hyphens are not allowed.

debian_revision

This part of the version number specifies the version of the Debian package based on the upstream version. It must contain only alphanumerics and the characters `+` `.` `~` (plus, full stop, tilde) and is compared in the same way as the `upstream_version` is.

It is conventional to restart the `debian_revision` at 1 each time the `upstream_version` is increased.

The package management system will break the version number apart at the last hyphen in the string (if there is one) to determine the `upstream_version` and `debian_revision`. The absence of a `debian_revision` is equivalent to a `debian_revision` of 0.

Presence of the `debian_revision` part indicates this package is a non-native package (see *Source packages*). Absence indicates the package is a native package.

When comparing two version numbers, first the epoch of each are compared, then the `upstream_version` if epoch is equal, and then `debian_revision` if `upstream_version` is also equal. epoch is compared numerically. The `upstream_version` and `debian_revision` parts are compared by the package management system using the following algorithm:

The strings are compared from left to right.

First the initial part of each string consisting entirely of non-digit characters is determined. These two parts (one of which may be empty) are compared lexically. If a difference is found it is returned. The lexical comparison is a comparison of ASCII values modified so that all the letters sort earlier than all the non-letters and so that a tilde sorts before anything, even the end of a part. For example, the following parts are in sorted order from earliest to latest: `~~`, `~~a`, `~`, the empty part, `a`.⁷

Then the initial part of the remainder of each string which consists entirely of digit characters is determined. The numerical values of these two parts are compared, and any difference found is returned as the result of the comparison. For these purposes an empty string (which can only occur at the end of one or both version strings being compared) counts as zero.

These two steps (comparing and removing initial non-digit strings and initial digit strings) are repeated until a difference is found or both strings are exhausted.

5.6.12.1 Epochs should be used sparingly

Note that the purpose of epochs is to cope with situations where the upstream version numbering scheme changes and to allow us to leave behind serious mistakes. If you think that increasing the epoch is the right solution, you should consult debian-devel and get consensus before doing so (even in experimental).

⁶ Alphanumerics are A-Za-z0-9 only.

⁷ One common use of `~` is for upstream pre-releases. For example, `1.0~beta1~svn1245` sorts earlier than `1.0~beta1`, which sorts earlier than `1.0`.

Epochs should not be used when a package needs to be rolled back. In that case, use the `+really` convention: for example, if you uploaded `2.3-3` and now you need to go backwards to upstream `2.2`, call your reverting upload something like `2.3+really2.2-1`. Eventually, when we upload upstream `2.4`, the `+really` part can go away.

Epochs are also not intended to cope with version numbers containing strings of letters which the package management system cannot interpret (such as `ALPHA` or `pre-`), or with silly orderings.⁸

5.6.12.2 Special version conventions

The following special version numbering conventions are used in the Debian archive:

- The absence of `debian_revision`, and therefore of a hyphen in the version number, indicates that the package is native.
- The presence of `+really` in the `upstream_version` component indicates that a newer upstream version has been rolled back to an older upstream version. The part of the `upstream_version` component following `+really` is the true upstream version. See *Epochs should be used sparingly* for an example of when this is used.

Non-maintainer uploads:

- `debian_revision` components ending in `.` (period) followed by a number indicate this version of the non-native package was uploaded by someone other than the maintainer (an NMU or non-maintainer upload). This is used for an upload including a source package; for uploads of only binary packages without source changes, see the binary NMU convention below.
- `upstream_version` components in native packages ending in `+nmu` followed by a number indicate an NMU of a native package. As with the convention for non-native packages, this is used for an upload including a source package, not for uploads of only binary packages without source changes.
- `upstream_version` components in native packages or `debian_revision` components in non-native packages ending in `+b` followed by a number indicate a binary NMU: an upload of a binary package without any source changes and hence without any corresponding source package upload or version change.

Stable updates:

- `debian_revision` components in non-native packages ending in `debNuX` also indicate a stable update. Either `~` or `+` will be used before this string depending on the details of the update. `N` is the major version number of the Debian stable release to which the package was uploaded, and `X` is a number, starting at 1, that is increased for each stable upload of this package.

There are three cases for non-native packages:

1. For stable updates that use the same upstream version, the `debian_revision` component will end in `+debNuX`. The portion of the version before that string is the original package version in the stable release.
2. For stable updates to a new upstream version that is based on a newer unstable package, the `debian_revision` component will end in `~debNuX`. The portion before that string will be the unstable version on which the package is based.
3. If a stable update is based on a new upstream version but is not based on a newer unstable package, the convention is to form the version number by taking the upstream version, appending `-0`, and then appending `+debNuX` (so the `debian_revision` component will be `0+debNuX`).

In all cases, these versions are chosen so that they will sort earlier than a subsequent unstable package of the same upstream version and thus that the stable package will upgrade to a newer version during a subsequent system upgrade.

For example, suppose Debian 10 released with a package with version `1.4-5`. If that package later receives a stable update in Debian 10 that uses the same upstream version, the first update would have the version `1.4-5+deb10u1`. A subsequent update would have version `1.4-5+deb10u2`.

⁸ The author of this manual has heard of a package whose versions went `1.1, 1.2, 1.3, 1, 2.1, 2.2, 2` and so forth.

If instead the package receives a stable update based on a 1.5-1 unstable package, the first such stable update would have the version 1.5-1~deb10u1 and a subsequent update would have the version 1.5-1~deb10u2.

If there were no unstable 1.5-1 package, but there were a stable update to an upstream 1.5 release, the first such stable update would have the version 1.5-0+deb10u1.

- `upstream_version` components in native packages ending in `+debNuX` indicate a stable update. This is a version of the package uploaded directly to a stable release, and the version is chosen to sort before any later version of the package uploaded to Debian's unstable or a later stable distribution. As with non-native packages, `N` is the major version number of the Debian stable release to which the package was uploaded, and `X` is a number, starting at 1, that is increased for each stable upload of this package.

For example, suppose Debian 10 released with a package with version 1.4. The first stable update of that package would have the version 1.4+deb10u1, and a subsequent update would have the version 1.4+deb10u2. These versions are chosen to sort before 1.5 (the next unstable version) or 1.4+deb11u1 (a stable update to a subsequent Debian 11 release).

Backports:

- `upstream_version` components in native packages or `debian_revision` components in non-native packages ending in `~bpoNuX` indicate a backport of a version of the package to an older stable release. The part of the version before `~bpo` is the version of the package being backported, `N` is the major version number of the Debian stable release to which the package was backported, and `X` is a number, starting at 1, that is increased for each revision of the backport of that package version. The rationale is the same as for stable updates, with the additional goal of ensuring a backported version sorts earlier than a stable update with the same upstream version.

Be aware that the stable update and backport conventions can stack. If, for example, Debian 10 contains a package with version 1.4-5+deb10u1 and that package is backported to Debian 9, the version of the Debian 9 backport would be 1.4-5+deb10u1~bpo9u1 (although this scenario is rare).

This list of version conventions is not exhaustive.

5.6.13 Description

In a source template control file or binary control file, the `Description` field contains a description of the binary package, consisting of two parts, the synopsis or the short description, and the long description. It is a multiline field with the following format:

```
Description: single line synopsis
extended description over several lines
```

The lines in the extended description can have these formats:

- Those starting with a single space are part of a paragraph. Successive lines of this form will be word-wrapped when displayed. The leading space will usually be stripped off. The line must contain at least one non-whitespace character.
- Those starting with two or more spaces. These will be displayed verbatim. If the display cannot be panned horizontally, the displaying program will line wrap them “hard” (i.e., without taking account of word breaks). If it can they will be allowed to trail off to the right. None, one or two initial spaces may be deleted, but the number of spaces deleted from each line will be the same (so that you can have indenting work correctly, for example). The line must contain at least one non-whitespace character.
- Those containing a single space followed by a single full stop character. These are rendered as blank lines. This is the *only* way to get a blank line.⁹
- Those containing a space, a full stop and some more characters. These are for future expansion. Do not use them.

⁹ Completely empty lines will not be rendered as blank lines. Instead, they will cause the parser to think you're starting a whole new record in the control file, and will therefore likely abort with an error.

Do not use tab characters. Their effect is not predictable.

See *The description of a package* for further information on this.

In a `.changes` file, the `Description` field contains a summary of the descriptions of the binary packages being uploaded. If no binary packages are being uploaded, this field will not be present.

When used inside a `.changes` file, the `Description` field has a different format than in source or binary control files. It is a multiline field with one line per binary package. The first line of the field value (the part on the same line as `Description:`) is always empty. Each subsequent line is indented by one space and contains the name of a binary package, a space, a hyphen (-), a space, and the short description line from that package.

5.6.14 Distribution

In a `.changes` file or parsed changelog output this contains the (space-separated) name(s) of the distribution(s) where this version of the package should be installed. Valid distributions are determined by the archive maintainers.¹⁰ The Debian archive software only supports listing a single distribution. Migration of packages to other distributions is handled outside of the upload process.

5.6.15 Date

This field includes the date the package was built or last edited. It must be in the same format as the date in a `debian/changelog` entry.

The value of this field is usually extracted from the `debian/changelog` file - see *Debian changelog: debian/changelog*).

5.6.16 Format

In `.changes` files, this field declares the format version of that file. The syntax of the field value is the same as that of a *package version number* except that no epoch or Debian revision is allowed. The format described in this document is 1.8.

In `.dsc` *Debian source control* files, this field declares the format of the source package. The field value is used by programs acting on a source package to interpret the list of files in the source package and determine how to unpack it. The syntax of the field value is a numeric major revision, a period, a numeric minor revision, and then an optional subtype after whitespace, which if specified is an alphanumeric word in parentheses. The subtype is optional in the syntax but may be mandatory for particular source format revisions.¹¹

5.6.17 Urgency

This is a description of how important it is to upgrade to this version from previous ones. It consists of a single keyword taking one of the values `low`, `medium`, `high`, `emergency`, or `critical`¹² (not case-sensitive) followed by an optional commentary (separated by a space) which is usually in parentheses. For example:

¹⁰ Example distribution names in the Debian archive used in `.changes` files are:

unstable

This distribution value refers to the *developmental* part of the Debian distribution tree. Most new packages, new upstream versions of packages and bug fixes go into the *unstable* directory tree.

experimental

The packages with this distribution value are deemed by their maintainers to be high risk. Oftentimes they represent early beta or developmental packages from various sources that the maintainers want people to try, but are not ready to be a part of the other parts of the Debian distribution tree.

Others are used for updating stable releases or for security uploads. More information is available in the Debian Developer's Reference, section "The Debian archive".

¹¹ The source formats currently supported by the Debian archive software are 1.0, 3.0 (native), and 3.0 (quilt).

¹² Other urgency values are supported with configuration changes in the archive software but are not used in Debian. The urgency affects how quickly a package will be considered for inclusion into the `testing` distribution and gives an indication of the importance of any fixes included in the upload. `Emergency` and `critical` are treated as synonymous.

Urgency: low (HIGH for users of diversions)

The value of this field is usually extracted from the `debian/changelog` file - see *Debian changelog: debian/changelog*.

5.6.18 Changes

This multiline field contains the human-readable changes data, describing the differences between the last version and the current one.

The first line of the field value (the part on the same line as `Changes:`) is always empty. The content of the field is expressed as continuation lines, with each line indented by at least one space. Blank lines must be represented by a line consisting only of a space and a full stop (.

The value of this field is usually extracted from the `debian/changelog` file - see *Debian changelog: debian/changelog*.

Each version's change information should be preceded by a "title" line giving at least the version, distribution(s) and urgency, in a human-readable way.

If data from several versions is being returned the entry for the most recent version should be returned first, and entries should be separated by the representation of a blank line (the "title" line may also be followed by the representation of a blank line).

5.6.19 Binary

This folded field is a list of binary packages. Its syntax and meaning varies depending on the control file in which it appears.

When it appears in the `.dsc` file, it lists binary packages which a source package can produce, separated by commas¹³. The source package does not necessarily produce all of these binary packages for every architecture. The source control file doesn't contain details of which architectures are appropriate for which of the binary packages.

When it appears in a `.changes` file, it lists the names of the binary packages being uploaded, separated by whitespace (not commas). If no binary packages are being uploaded, this field will not be present.

5.6.20 Installed-Size

This field appears in the binary package control files, and in the `Packages` files. It gives an estimate of the total amount of disk space required to install the named package. Actual installed size may vary based on block size, file system properties, or actions taken by package maintainer scripts.

The disk space is given as the accumulated size of each regular file and symlink rounded to 1 KiB used units, and a baseline of 1 KiB for any other filesystem object type.

5.6.21 Files

This field contains a list of files with information about each one. The exact information and syntax varies with the context.

In all cases, `Files` is a multiline field. The first line of the field value (the part on the same line as `Files:`) is always empty. The content of the field is expressed as continuation lines, one line per file. Each line must be indented by one space and contain a number of sub-fields, separated by spaces, as described below.

In the `.dsc` file, each line contains the MD5 checksum, size and filename of the tar file and (if applicable) diff file which make up the remainder of the source package.¹⁴ For example:

¹³ A space after each comma is conventional.

¹⁴ That is, the parts which are not the `.dsc`.

Files:

```
c6f698f19f2a2aa07dbb9bbda90a2754 571925 example_1.2.orig.tar.gz
938512f08422f3509ff36f125f5873ba 6220 example_1.2-1.diff.gz
```

The exact forms of the filenames are described in *Source packages as archives*.

In the `.changes` file this contains one line per file being uploaded. Each line contains the MD5 checksum, size, section and priority and the filename. For example:

Files:

```
4c31ab7bfc40d3cf49d7811987390357 1428 text extra example_1.2-1.dsc
c6f698f19f2a2aa07dbb9bbda90a2754 571925 text extra example_1.2.orig.tar.gz
938512f08422f3509ff36f125f5873ba 6220 text extra example_1.2-1.diff.gz
7c98fe853b3bbb47a00e5cd129b6cb56 703542 text extra example_1.2-1_i386.deb
```

The *section* and *priority* are the values of the corresponding fields in the source template control file. If no section or priority is specified then `-` should be used, though section and priority values must be specified for new packages to be installed properly.

The special value `byhand` for the section in a `.changes` file indicates that the file in question is not an ordinary package file and must be installed by hand by the distribution maintainers. If the section is `byhand` the priority should be `-`.

If a new Debian revision of a package is being shipped and no new original source archive is being distributed the `.dsc` must still contain the `Files` field entry for the original source archive `package_upstream-version.orig.tar.gz`, but the `.changes` file should leave it out. In this case the original source archive on the distribution site must match exactly, byte-for-byte, the original source archive which was used to generate the `.dsc` file and diff which are being uploaded.

5.6.22 closes

A space-separated list of bug report numbers that the upload governed by the `.changes` file closes.

5.6.23 Homepage

The URL of the web site for this package, preferably (when applicable) the site from which the original source can be obtained and any additional upstream documentation or information may be found. The content of this field is a simple URL without any surrounding characters such as `<>`.

5.6.24 Checksums-Sha1 and Checksums-Sha256

These multiline fields contain a list of files with a checksum and size for each one. Both `Checksums-Sha1` and `Checksums-Sha256` have the same syntax and differ only in the checksum algorithm used: `SHA-1` for `Checksums-Sha1` and `SHA-256` for `Checksums-Sha256`.

`Checksums-Sha1` and `Checksums-Sha256` are multiline fields. The first line of the field value (the part on the same line as `Checksums-Sha1:` or `Checksums-Sha256:`) is always empty. The content of the field is expressed as continuation lines, one line per file. Each line consists of the checksum, a space, the file size, a space, and the file name. For example (from a `.changes` file):

Checksums-Sha1:

```
1f418afaa01464e63cc1ee8a66a05f0848bd155c 1276 example_1.0-1.dsc
a0ed1456fad61116f868b1855530dbe948e20f06 171602 example_1.0.orig.tar.gz
5e86ecf0671e113b63388dac81dd8d00e00ef298 6137 example_1.0-1.debian.tar.gz
71a0ff7da0faaf608481195f9cf30974b142c183 548402 example_1.0-1_i386.deb
```

Checksums-Sha256:

(continues on next page)

(continued from previous page)

```

ac9d57254f7e835bed299926fd51bf6f534597cc3fcc52db01c4bffedae81272 1276 example_1.0-1.
↪dsc
0d123be7f51e61c4bf15e5c492b484054be7e90f3081608a5517007bfb1fd128 171602 example_1.0.
↪orig.tar.gz
f54ae966a5f580571ae7d9ef5e1df0bd42d63e27cb505b27957351a495bc6288 6137 example_1.0-1.
↪debian.tar.gz
3bec05c03974fdecdd11d020fc2e8250de8404867a8a2ce865160c250eb723664 548402 example_1.0-
↪1_i386.deb

```

In the `.dsc` file, these fields list all files that make up the source package. In the `.changes` file, these fields list all files being uploaded. The list of files in these fields must match the list of files in the `Files` field.

5.6.25 DM-Upload-Allowed

Obsolete, see *below*.

5.6.26 Version Control System (VCS) fields

Debian source packages are increasingly developed using VCSs. The purpose of the following fields is to indicate a publicly accessible repository where the Debian source package is developed.

Vcs-Browser

URL of a web interface for browsing the repository.

Here is an example of the form the two values take for Debian's GitLab instance, Salsa:

```

Vcs-Browser: https://salsa.debian.org/debian/grep
Vcs-Git: https://salsa.debian.org/debian/grep.git

```

Vcs-<type>

The field name identifies the VCS. The field's value uses the version control system's conventional syntax for describing repository locations and should be sufficient to locate the repository used for packaging. Ideally, it also locates the branch used for development of new versions of the Debian package.

The following values for `<type>` are supported, with the corresponding VCS indicated in parentheses if it isn't obvious:

- Arch
- Bzr (Bazaar)
- Cvs (CVS)
- Darcs
- Git
- Hg (Mercurial)
- Mtn (Monotone)
- Svn (Subversion)

In the case of Git, the value must have the following syntax:

```
<url> [ " -b " <branch> ] [ " [" <path> "]" ]
```

where the portions enclosed in brackets are optional and the portions enclosed in double quotes are literal strings. `<url>` indicates the repository. If the `<branch>` portion is present, it names a branch in the indicated repository.

If no branch is specified, the packaging should be on the default branch. If the `<path>` portion is present, it specifies the relative path to the top of the packaging tree (the parent directory of the `debian` directory). If no path is specified, it defaults to `.` (the top level of the indicated repository and branch).

For example:

```
Vcs-Git: https://example.org/repo -b debian [p/package]
```

indicates a subdirectory named `p/package` in the `debian` branch of the repository at `https://example.org/repo`.

In the case of Mercurial, the value must have the following syntax:

```
<url> [ " -b " <branch> ]
```

This is interpreted the same way as the Git syntax except a path within the repository is not supported.

A package control file must not have more than one `Vcs-<type>` field. If the package is maintained in multiple version control systems, the maintainer should specify the one that they would prefer other people to use as the basis for proposing changes to the package.

For both fields, any URLs given should use a scheme that provides confidentiality (`https`, for example, rather than `http` or `git`) if the VCS repository supports it.

5.6.27 Package-List

Multiline field listing all the packages that can be built from the source package, considering every architecture. The first line of the field value is empty. Each one of the next lines describes one binary package, by listing its name, type, section and priority separated by spaces. Fifth and subsequent space-separated items may be present and parsers must allow them. See the *Package-Type* field for a list of package types.

5.6.28 Package-Type

Simple field containing a word indicating the type of package: `deb` for binary packages and `udeb` for micro binary packages. Other types not defined here may be indicated. In source package template control files, the `Package-Type` field should be omitted instead of giving it a value of `deb`, as this value is assumed for stanzas lacking this field.

5.6.29 Dgit

Folded field containing a single git commit hash, presented in full, followed optionally by whitespace and other data to be defined in future extensions.

Declares that the source package corresponds exactly to a referenced commit in a Git repository available at the canonical location called *dgit-repos*, used by `dgit`, a bidirectional gateway between the Debian archive and Git. The commit is reachable from at least one reference whose name matches `refs/dgit/*`. See the manual page of `dgit` for further details.

5.6.30 Testsuite

Simple field containing a comma-separated list of values allowing test execution environments to discover packages which provide tests.

This field is automatically added to Debian source control files (`.dsc`) by `dpkg`, with the value `autopkgtest`, when a `debian/tests/control` file is present in the source package. This field may also be used in source package template control files (`debian/control`) if needed in other situations.

5.6.31 Rules-Requires-Root

Simple field that defines if the source package requires access to root (or fakeroot) during selected targets in the *Main building script: `debian/rules`*.

The field can consist of exactly one of the following three items:

- `no` (default): Declares that neither root nor fakeroot is required. Package builders (e.g. `dpkg-buildpackage`) may choose to invoke any target in `debian/rules` with an unprivileged user.
- `binary-targets` (previous default): Declares that the package will need the root (or fakeroot) when either of the `binary`, `binary-arch` or `binary-indep` targets are called. This is how every tool behaved before this field was defined.
- A space separated list of keywords described below. These keywords must always contain a forward slash, which sets them apart from the other possible values of `Rules-Requires-Root`. When this list is provided, the builder must provide a “gain root command” (as defined in *debian/rules and Rules-Requires-Root*) or pretend that the value was set to `binary-targets`, and both the builder and the package’s `debian/rules` script must downgrade accordingly (see below).

The default depends on the `dpkg-build-api` level (see `dpkg-build-api(7)`). If the `dpkg-build-api` level is undeclared, it is equivalent to level 0.

If the package builder supports the `Rules-Requires-Root` field and wants to enable the feature, then it must set the environment variable `DEB_RULES_REQUIRES_ROOT` when invoking the package building script `debian/rules`. The value of `DEB_RULES_REQUIRES_ROOT` should be one of:

- The value of `Rules-Requires-Root` if the builder can support that value. The builder may trim unnecessary whitespace used to format the field for readability.
- The value `binary-targets` if it cannot support the value of `Rules-Requires-Root`.

A compliant builder may also leave `DEB_RULES_REQUIRES_ROOT` unset or set it to `binary-targets` if it has been requested to test whether the package it builds correctly implements the fall-back for legacy builders.

5.6.31.1 Remarks

All packages and builders must support `binary-targets` as this was the historical behaviour prior to the introduction of this field.

Any tool (particularly older versions of them) may be unaware of this field and behave like the field was set to `binary-targets`. The package build must gracefully cope with this and produce a semantically equivalent result.

This field intentionally does not enable a package to request a true root over fakeroot.

5.6.31.2 Definition of the keywords

The keywords have the format `<namespace>/<case>`, where:

- `<namespace>` must consist entirely of printable ASCII characters except for any whitespace and the forward slash (`/`). It must consist of at least 2 characters.
- `/` (between `<namespace>` and `<case>`) is a single ASCII forward slash.
- `<case>` must consist entirely of printable ASCII characters except for any whitespace. It must consist of at least 2 characters.

These keywords define where the package build script `debian/rules`, or the tools called by that script, will need access to root or fakeroot.

In addition to the keywords defined in the next section, each tool or package may define keywords within a namespace named after that tool or package. The package or tool is considered to own that namespace.

A tool is permitted to use the “gain root command” to do something under (fake)root if and only if the tool defines an appropriate keyword in its namespace, and the package lists that keyword in `Rules-Requires-Root`.

All tools must ignore keywords under namespaces they do not know or own. A tool may emit a warning, or abort with an error, if it finds unknown keywords in namespaces it owns, but it is not required to do this for all keywords in the namespace.

5.6.31.3 Provided keywords

The following keywords are defined:

- `dpkg/target-subcommand`: declares that there exists a command that the `debian/rules` file must run under (fake)root
- `dpkg/target/foo`: declares that the additional, package-specific target `foo` (that is, not one of the targets specified in *Main building script: debian/rules*) must be run under (fake)root

This list is intentionally incomplete. You should consult the documentation of the tool or package in question for which keywords it defines and when they are needed.

5.6.32 Git-Tag-Tagger

Name and e-mail address of the person who made the Git tag from which this upload was generated (and to which it corresponds) in accordance with the tagging protocol described in the `tag2upload(5)` manual page and `TAG2UPLOAD-DESIGN.txt`. The syntax is the same as for the *Maintainer field*. The values for the name and e-mail address originate in the `tagger` line of the raw Git tag; they are transformed as necessary to satisfy the field’s syntax.

Uploads signed by an implementation of the tag2upload service must include this field. Uploads not generated in accordance with the tag2upload protocol must not include this field.

5.6.33 Git-Tag-Info

Other information about the Git tag from which this upload was generated (and to which it corresponds) in accordance with the tagging protocol described in the `tag2upload(5)` manual page and `TAG2UPLOAD-DESIGN.txt`.

The value is of the form `tag=TAGOBJID fp=FINGERPRINT` where `TAGOBJID` is the Git object ID of the Git annotated tag object,¹⁵ and `FINGERPRINT` is the fingerprint (in hexadecimal, without spaces) of the PGP key used to sign the Git tag. Other space-separated `keyword=value` items may be introduced in the future, and users of this field must ignore items with unknown keywords.

`FINGERPRINT` is taken from the first field of the `VALIDSIG` line emitted by `gpgv(1)`, as specified in `/usr/share/doc/gnupg/DETAILS.gz` from the `gnupg` package. This will generally be the fingerprint of the signing subkey, if one was used, and the primary key’s fingerprint otherwise.

The Git annotated tag object is obtainable from the `dggit-repos` server, as described under `Dgit`, above.

Uploads signed by an implementation of the tag2upload service must include this field. Uploads not generated in accordance with the tag2upload protocol must not include this field.

5.7 User-defined fields

Additional user-defined fields may be added to the source package template control file. Such fields will be ignored, and not copied to (for example) binary or Debian source control files or Debian upload changes control files.

If you wish to add additional unsupported fields to these output files you should use the mechanism described here.

¹⁵ Annotated tag objects have distinct IDs from the IDs of the commits to which they point. Many Git commands on operating on tags dereference the tag immediately, and show you the ID of the commit object and not of the tag object, but these are not the same.

Fields in the source package template control file with names starting `x`, followed by one or more of the letters `BCS` and a hyphen `-`, will be copied to the output files. Only the part of the field name after the hyphen will be used in the output file. Where the letter `B` is used the field will appear in binary package control files, where the letter `S` is used in Debian source control files and where `C` is used in Debian upload changes control files.

For example, if the source package template control file contains the field

```
XBS-Comment: I stand between the candle and the star.
```

then the binary and Debian source control files will contain the field

```
Comment: I stand between the candle and the star.
```

5.8 Obsolete fields

The following fields have been obsoleted and may be found in packages conforming with previous versions of the Policy.

5.8.1 `DM-Upload-Allowed`

Indicates that Debian Maintainers may upload this package to the Debian archive. The only valid value is `yes`. This field was used to regulate uploads by Debian Maintainers, See the General Resolution [Endorse the concept of Debian Maintainers](#) for more details.

PACKAGE MAINTAINER SCRIPTS AND INSTALLATION PROCEDURE

6.1 Introduction to package maintainer scripts

It is possible to supply scripts as part of a package which the package management system will run for you when your package is installed, upgraded or removed.

These scripts are the package metadata files `preinst`, `postinst`, `prerm` and `postrm`. They must be proper executable files; if they are scripts (which is recommended), they must start with the usual `#!` convention. They should be readable and executable by anyone, and must not be world-writable.

The package management system looks at the exit status from these scripts. It is important that they exit with a non-zero status if there is an error, so that the package management system can stop its processing. For shell scripts this means that you *almost always* need to use `set -e` (this is usually true when writing shell scripts, in fact). It is also important, of course, that they exit with a zero status if everything went well.

Additionally, packages interacting with users using `debconf` in the `postinst` script should install a `config` script as a package metadata file. See *Prompting in maintainer scripts* for details.

When a package is upgraded a combination of the scripts from the old and new packages is called during the upgrade procedure. If your scripts are going to be at all complicated you need to be aware of this, and may need to check the arguments to your scripts.

Broadly speaking the `preinst` is called before (a particular version of) a package is unpacked, and the `postinst` afterwards; the `prerm` before (a version of) a package is removed and the `postrm` afterwards.

Programs called from maintainer scripts should not normally have a path prepended to them. Before installation is started, the package management system checks to see if the programs `ldconfig`, `start-stop-daemon`, and `update-rc.d` can be found via the `PATH` environment variable. Those programs, and any other program that one would expect to be in the `PATH`, should thus be invoked without an absolute pathname. Maintainer scripts should also not reset the `PATH`, though they might choose to modify it by prepending or appending package-specific directories. These considerations really apply to all shell scripts.

6.2 Maintainer scripts idempotency

It is necessary for the error recovery procedures that the scripts be idempotent. This means that if it is run successfully, and then it is called again, it doesn't bomb out or cause any harm, but just ensures that everything is the way it ought to be. If the first call failed, or aborted half way through for some reason, the second call should merely do the things that were left undone the first time, if any, and exit with a success status if everything is OK.¹

¹ This is so that if an error occurs, the user interrupts `dpkg` or some other unforeseen circumstance happens you don't leave the user with a badly-broken package when `dpkg` attempts to repeat the action.

6.3 Controlling terminal for maintainer scripts

Maintainer scripts are not guaranteed to run with a controlling terminal and may not be able to interact with the user. They must be able to fall back to noninteractive behavior if no controlling terminal is available. Maintainer scripts that prompt via a program conforming to the Debian Configuration Management Specification (see *Prompting in maintainer scripts*) may assume that program will handle falling back to noninteractive behavior.

For high-priority prompts without a reasonable default answer, maintainer scripts may abort if there is no controlling terminal. However, this situation should be avoided if at all possible, since it prevents automated or unattended installs. In most cases, users will consider this to be a bug in the package.

6.4 Exit status

Each script must return a zero exit status for success, or a nonzero one for failure, since the package management system looks for the exit status of these scripts and determines what action to take next based on that datum.

6.5 Summary of ways maintainer scripts are called

What follows is a summary of all the ways in which maintainer scripts may be called along with what facilities those scripts may rely on being available at that time. Script names preceded by `new-` are the scripts from the new version of a package being installed, upgraded to, or downgraded to. Script names preceded by `old-` are the scripts from the old version of a package that is being upgraded from or downgraded from.

The `preinst` script may be called in the following ways:

```
new-preinst install
new-preinst install old-version new-version
new-preinst upgrade old-version new-version
```

The package will not yet be unpacked, so the `preinst` script cannot rely on any files included in its package. Only essential packages and pre-dependencies (*Pre-Depends*) may be assumed to be available. Pre-dependencies will have been configured at least once, but at the time the `preinst` is called they may only be in an “Unpacked” or “Half-Configured” state if a previous version of the pre-dependency was completely configured and has not been removed since then.

old-preinst abort-upgrade *new-version*

Called during error handling of an upgrade that failed after unpacking the new package because the `postrm upgrade` action failed. The unpacked files may be partly from the new version or partly missing, so the script cannot rely on files included in the package. Package dependencies may not be available. Pre-dependencies will be at least “Unpacked” following the same rules as above, except they may be only “Half-Installed” if an upgrade of the pre-dependency failed.²

The `postinst` script may be called in the following ways:

postinst configure *most-recently-configured-version*

The files contained in the package will be unpacked. All package dependencies will at least be “Unpacked”. If there are no circular dependencies involved, all package dependencies will be configured. For behavior in the case of circular dependencies, see the discussion in *Binary Dependencies - Depends, Recommends, Suggests, Enhances, Pre-Depends*.

```
old-postinst abort-upgrade new-version
```

² This can happen if the new version of the package no longer pre-depends on a package that had been partially upgraded.

```

conflictor's-postinst abort-remove in-favour package new-version
postinst abort-remove
deconfigured's-postinst abort-deconfigure in-favour failed-install-package version [ removing
conflicting-package version ]

```

The files contained in the package will be unpacked. All package dependencies will at least be “Half-Installed” and will have previously been configured and not removed. However, dependencies may not be configured or even fully unpacked in some error situations.³ The `postinst` should still attempt any actions for which its dependencies are required, since they will normally be available, but consider the correct error handling approach if those actions fail. Aborting the `postinst` action if commands or facilities from the package dependencies are not available is often the best approach.

The `prerm` script may be called in the following ways:

```

prerm remove
old-prerm upgrade new-version
conflictor's-prerm remove in-favour package new-version
deconfigured's-prerm deconfigure in-favour package-being-installed version [removing conflicting-package
version]

```

The package whose `prerm` is being called will be at least “Half-Installed”. All package dependencies will at least be “Half-Installed” and will have previously been configured and not removed. If there was no error, all dependencies will at least be “Unpacked”, but these actions may be called in various error states where dependencies are only “Half-Installed” due to a partial upgrade.

new-prerm failed-upgrade *old-version new-version*

Called during error handling when `prerm upgrade` fails. The new package will not yet be unpacked, and all the same constraints as for `preinst upgrade` apply.

The `postrm` script may be called in the following ways:

```

postrm remove
postrm purge
old-postrm upgrade new-version
disappearer's-postrm disappear overwriter overwriter-version

```

The `postrm` script is called after the package’s files have been removed or replaced. The package whose `postrm` is being called may have previously been deconfigured and only be “Unpacked”, at which point subsequent package changes do not consider its dependencies. Therefore, all `postrm` actions must only rely on essential packages and must gracefully skip any actions that require the package’s dependencies if those dependencies are unavailable.⁴

new-postrm failed-upgrade *old-version new-version*

Called when the old `postrm upgrade` action fails. The new package will be unpacked, but only essential packages

³ For example, suppose packages `foo` and `bar` are “Installed” with `foo` depending on `bar`. If an upgrade of `bar` were started and then aborted, and then an attempt to remove `foo` failed because its `prerm` script failed, `foo`’s `postinst abort-remove` would be called with `bar` only “Half-Installed”.

⁴ This is often done by checking whether the command or facility the `postrm` intends to call is available before calling it. For example:

```

if [ "$1" = purge ] && [ -e /usr/share/debconf/confmodule ]; then
    . /usr/share/debconf/confmodule db_purge
fi

```

in `postrm` purges the `debconf` configuration for the package if `debconf` is installed.

and pre-dependencies can be relied on. Pre-dependencies will either be configured or will be “Unpacked” or “Half-Configured” but previously had been configured and was never removed.

```
new-postrm abort-install
new-postrm abort-install old-version new-version
new-postrm abort-upgrade old-version new-version
```

Called before unpacking the new package as part of the error handling of `preinst` failures. May assume the same state as `preinst` can assume.

6.6 Details of unpack phase of installation or upgrade

The procedure on installation/upgrade/overwrite/disappear (i.e., when running `dpkg --unpack`, or the unpack stage of `dpkg --install`) is as follows.⁵ In each case, if a major error occurs (unless listed below) the actions are, in general, run backwards - this means that the maintainer scripts are run with different arguments in reverse order. These are the “error unwind” calls listed below.

1. Notify the currently installed package:

- a. If a version of the package is already “Installed”, call

```
old-prerm upgrade new-version
```

- b. If the script runs but exits with a non-zero exit status, `dpkg` will attempt:

```
new-prerm failed-upgrade old-version new-version
```

If this works, the upgrade continues. If this does not work, the error unwind:

```
old-postinst abort-upgrade new-version
```

If this works, then the *old-version* is “Installed”, if not, the old version is in a “Half-Configured” state.

2. If a “conflicting” package is being removed at the same time, or if any package will be broken (due to `Breaks`):

- a. If `--auto-deconfigure` is specified, call, for each package to be deconfigured due to `Breaks`:

```
deconfigured's-prerm deconfigure \
    in-favour package-being-installed version
```

Error unwind:

```
deconfigured's-postinst abort-deconfigure \
    in-favour package-being-installed-but-failed version
```

The deconfigured packages are marked as requiring configuration, so that if `--install` is used they will be configured again if possible.

- b. If any packages depended on a conflicting package being removed and `--auto-deconfigure` is specified, call, for each such package:

```
deconfigured's-prerm deconfigure \
    in-favour package-being-installed version \
    removing conflicting-package version
```

Error unwind:

⁵ See [Maintainer script flowcharts](#) for flowcharts illustrating the processes described here.

```
deconfigured's-postinst abort-deconfigure \
    in-favour package-being-installed-but-failed version \
    removing conflicting-package version
```

The deconfigured packages are marked as requiring configuration, so that if `--install` is used they will be configured again if possible.

- c. To prepare for removal of each conflicting package, call:

```
conflictor's-prerm remove \
    in-favour package new-version
```

Error unwind:

```
conflictor's-postinst abort-remove \
    in-favour package new-version
```

3. Run the `preinst` of the new package:

- a. If the package is being upgraded, call:

```
new-preinst upgrade old-version new-version
```

If this fails, we call:

```
new-postrm abort-upgrade old-version new-version
```

- i. If that works, then

```
old-postinst abort-upgrade new-version
```

is called. If this works, then the old version is in an “Installed” state, or else it is left in an “Unpacked” state.

- ii. If it fails, then the old version is left in an “Half-Installed” state.

- b. Otherwise, if the package had some configuration files from a previous version installed (i.e., it is in the “Config-Files” state):

```
new-preinst install old-version new-version
```

Error unwind:

```
new-postrm abort-install old-version new-version
```

If this fails, the package is left in a “Half-Installed” state, which requires a reinstall. If it works, the package is left in a “Config-Files” state.

- c. Otherwise (i.e., the package was completely purged):

```
new-preinst install
```

Error unwind:

```
new-postrm abort-install
```

If the error-unwind fails, the package is in a “Half-Installed” phase, and requires a reinstall. If the error unwind works, the package is in the “Not-Installed” state.

4. The new package’s files are unpacked, overwriting any that may be on the system already, for example any from the old version of the same package or from another package. Backups of the old files are kept temporarily, and if anything goes wrong the package management system will attempt to put them back as part of the error unwind.

It is an error for a package to contain files which are on the system in another package, unless `Replaces` is used (see *Overwriting files and replacing packages - Replaces*).

It is a more serious error for a package to contain a plain file or other kind of non-directory where another package has a directory (again, unless `Replaces` is used). This error can be overridden if desired using `--force-overwrite-dir`, but this is not advisable.

Packages which overwrite each other's files produce behavior which, though deterministic, is hard for the system administrator to understand. It can easily lead to “missing” programs if, for example, a package is unpacked which overwrites a file from another package, and is then removed again.⁶

A directory will never be replaced by a symbolic link to a directory or vice versa; instead, the existing state (symlink or not) will be left alone and `dpkg` will follow the symlink if there is one.

5. If the package is being upgraded:

a. Call:

```
old-postrm upgrade new-version
```

b. If this fails, `dpkg` will attempt:

```
new-postrm failed-upgrade old-version new-version
```

If this works, installation continues. If not, Error unwind:

```
old-preinst abort-upgrade new-version
```

If this fails, the old version is left in a “Half-Installed” state. If it works, `dpkg` now calls:

```
new-postrm abort-upgrade old-version new-version
```

If this fails, the old version is left in a “Half-Installed” state. If it works, `dpkg` now calls:

```
old-postinst abort-upgrade new-version
```

If this fails, the old version is in an “Unpacked” state.

This is the point of no return. If `dpkg` gets this far, it won't back off past this point if an error occurs. This will leave the package in a fairly bad state, which will require a successful re-installation to clear up, but it's when `dpkg` starts doing things that are irreversible.

6. Any files which were in the old version of the package but not in the new are removed.

7. The new file list replaces the old.

8. The new maintainer scripts replace the old.

9. Any packages all of whose files have been overwritten during the installation, and which aren't required for dependencies, are considered to have been removed. For each such package

a. `dpkg` calls:

```
disappearer's-postrm disappear \  
overwriter overwriter-version
```

b. The package's maintainer scripts are removed.

c. It is noted in the status database as being in a sane state, namely “Not-Installed” (any conffiles it may have are ignored, rather than being removed by `dpkg`). Note that disappearing packages do not have their premm called, because `dpkg` doesn't know in advance that the package is going to vanish.

10. Any files in the package we're unpacking that are also listed in the file lists of other packages are removed from those lists. (This will lobotomize the file list of the “conflicting” package if there is one.)

11. The backup files made during installation, above, are deleted.

⁶ Part of the problem is due to what is arguably a bug in `dpkg`.

12. The new package's status is now sane, and recorded as "Unpacked".

Here is another point of no return: if the conflicting package's removal fails we do not unwind the rest of the installation. The conflicting package is left in a half-removed limbo.

13. If there was a conflicting package we go and do the removal actions (described below), starting with the removal of the conflicting package's files (any that are also in the package being unpacked have already been removed from the conflicting package's file list, and so do not get removed now).

6.7 Details of configuration

When we configure a package (this happens with `dpkg --install` and `dpkg --configure`), we first update any `conffiles` and then call:

```
postinst configure most-recently-configured-version
```

No attempt is made to unwind after errors during configuration. If the configuration fails, the package is in a "Half-Configured" state, and an error message is generated.

If there is no most recently configured version `dpkg` will pass a null argument.⁷

6.8 Details of removal and/or configuration purging

1. `prerm remove`

If `prerm` fails during replacement due to conflict

```
conflictor's-postinst abort-remove \
    in-favour package new-version
```

Or else we call:

```
postinst abort-remove
```

If this fails, the package is in a "Half-Configured" state, or else it remains "Installed".

2. The package's files are removed (except `conffiles`).

3. `postrm remove`

If it fails, there's no error unwind, and the package is in an "Half-Installed" state.

4. All the maintainer scripts except the `postrm` are removed.

If we aren't purging the package we stop here. Note that packages which have no `postrm` and no `conffiles` are automatically purged when removed, as there is no difference except for the `dpkg` status.

5. The `conffiles` and any backup files (`~`-files, `##` files, `%`-files, `.dpkg-{old,new,tmp}`, etc.) are removed.

6. `postrm purge`

If this fails, the package remains in a "Config-Files" state.

7. The package's file list is removed.

⁷ Historical note: Truly ancient (pre-1997) versions of `dpkg` passed `<unknown>` (including the angle brackets) in this case. Even older ones did not pass a second argument at all, under any circumstance. Note that upgrades using such an old `dpkg` version are unlikely to work for other reasons, even if this old argument behavior is handled by your `postinst` script.

DECLARING RELATIONSHIPS BETWEEN PACKAGES

7.1 Syntax of relationship fields

These fields all have a uniform syntax. They are a list of package names separated by commas.

In the `Depends`, `Recommends`, `Suggests`, `Pre-Depends`, `Build-Depends`, `Build-Depends-Indep` and `Build-Depends-Arch` control fields of the package, which declare dependencies on other packages, the package names listed may also include lists of alternative package names, separated by vertical bar (pipe) symbols `|`. In such a case, that part of the dependency can be satisfied by any one of the alternative packages. (Alternative dependencies in `Build-Depends`, `Build-Depends-Indep`, and `Build-Depends-Arch` are interpreted specially by Debian auto-builders. See *Relationships between source and binary packages* for more details.)

All of the fields may restrict their applicability to particular versions of each named package. This is done in parentheses after each individual package name; the parentheses should contain a relation from the list below followed by a version number, in the format described in *Version*.

The relations allowed are `<<`, `<=`, `=`, `>=` and `>>` for strictly earlier, earlier or equal, exactly equal, later or equal and strictly later, respectively. The exception is the `Provides` field, for which only `=` is allowed.¹

Whitespace may appear at any point in the version specification subject to the rules in *Syntax of control files*, and must appear where it's necessary to disambiguate; it is not otherwise significant. All of the relationship fields can only be folded in source package template control files. For consistency and in case of future changes to `dpkg` it is recommended that a single space be used after a version relationship and before a version number; it is also conventional to put a single space after each comma, on either side of each vertical bar, and before each open parenthesis. When opening a continuation line in a relationship field, it is conventional to do so after a comma and before the space following that comma.

For example, a list of dependencies might appear as:

```
Package: mutt
Version: 1.3.17-1
Depends: libc6 (>= 2.2.1), default-mta | mail-transport-agent
```

Relationships may be restricted to a certain set of architectures. This is indicated in brackets after each individual package name and the optional version specification. The brackets enclose a non-empty list of Debian architecture names in the format described in *Architecture specification strings*, separated by whitespace. Exclamation marks may be prepended to each of the names. (It is not permitted for some names to be prepended with exclamation marks while others aren't.)

For build relationship fields (`Build-Depends`, `Build-Depends-Indep`, `Build-Depends-Arch`, `Build-Conflicts`, `Build-Conflicts-Indep` and `Build-Conflicts-Arch`), if the current Debian host architecture is not in this list and there are no exclamation marks in the list, or it is in the list with a prepended exclamation mark, the package name and the associated version specification are ignored completely for the purposes of defining the relationships.

¹ The relations `<` and `>` were previously allowed, but they were confusingly defined to mean earlier/later or equal rather than strictly earlier/later. `dpkg` still supports them with a warning, but they are no longer allowed by Debian Policy.

For example:

```
Source: glibc
Build-Depends-Indep: texinfo
Build-Depends: kernel-headers-2.2.10 [!hurd-i386],
             hurd-dev [hurd-i386], gnumach-dev [hurd-i386]
```

requires `kernel-headers-2.2.10` on all architectures other than `hurd-i386` and requires `hurd-dev` and `gnumach-dev` only on `hurd-i386`. Here is another example showing multiple architectures separated by spaces:

```
Build-Depends:
  liblua5.1-dev [i386 amd64 kfreebsd-i386 armel armhf powerpc mips],
  liblua5.1-dev [hurd-i386 ia64 kfreebsd-amd64 s390x sparc],
```

For binary relationship fields and the `Built-Using` field, the architecture restriction syntax is only supported in the source package template control file `debian/control`. When the corresponding binary package control file is generated, the relationship will either be omitted or included without the architecture restriction based on the architecture of the binary package. This means that architecture restrictions must not be used in binary relationship fields for architecture-independent packages (`Architecture: all`).

For example:

```
Depends: foo [i386], bar [amd64]
```

becomes `Depends: foo` when the package is built on the `i386` architecture, `Depends: bar` when the package is built on the `amd64` architecture, and omitted entirely in binary packages built on all other architectures.

If the architecture-restricted dependency is part of a set of alternatives using `|`, that alternative is ignored completely on architectures that do not match the restriction. For example:

```
Build-Depends: foo [!i386] | bar [!amd64]
```

is equivalent to `bar` on the `i386` architecture, to `foo` on the `amd64` architecture, and to `foo | bar` on all other architectures.

Relationships may also be restricted to a certain set of architectures using architecture wildcards in the format described in [Architecture wildcards](#). The syntax for declaring such restrictions is the same as declaring restrictions using a certain set of architectures without architecture wildcards. For example:

```
Build-Depends: foo [linux-any], bar [any-i386], baz [!linux-any]
```

is equivalent to `foo` on architectures using the Linux kernel and any `cpu`, `bar` on architectures using any kernel and an `i386` `cpu`, and `baz` on any architecture using a kernel other than Linux.

Note that the binary package relationship fields such as `Depends` appear in one of the binary package stanzas of the template control file, whereas the build-time relationships such as `Build-Depends` appear in the source package stanza of the template control file (which is the first section).

7.2 Binary Dependencies - Depends, Recommends, Suggests, Enhances, Pre-Depends

Packages can declare in their control file that they have certain relationships to other packages - for example, that they cannot be installed at the same time as certain other packages, and/or that they depend on the presence of others.

This is done using the `Depends`, `Pre-Depends`, `Recommends`, `Suggests`, `Enhances`, `Breaks` and `Conflicts` control fields. `Breaks` is described in [Packages which break other packages - Breaks](#), and `Conflicts` is described in

Conflicting binary packages - Conflicts. The rest are described below.

These seven fields are used to declare a dependency relationship by one package on another. Except for `Enhances` and `Breaks`, they appear in the depending (binary) package's control file. (`Enhances` appears in the recommending package's control file, and `Breaks` appears in the version of depended-on package which causes the named package to break).

A `Depends` field takes effect *only* when a package is to be configured. It does not prevent a package being on the system in an unconfigured state while its dependencies are unsatisfied, and it is possible to replace a package whose dependencies are satisfied and which is properly installed with a different version whose dependencies are not and cannot be satisfied; when this is done the depending package will be left unconfigured (since attempts to configure it will give errors) and will not function properly. If it is necessary, a `Pre-Depends` field can be used, which has a partial effect even when a package is being unpacked, as explained in detail below. (The other three dependency fields, `Recommends`, `Suggests` and `Enhances`, are only used by the various front-ends to `dpkg` such as `apt-get`, `aptitude`, and `dselect`.)

Since `Depends` only places requirements on the order in which packages are configured, packages in an installation run are usually all unpacked first and all configured later.²

If there is a circular dependency among packages being installed or removed, installation or removal order honoring the dependency order is impossible, requiring the dependency loop be broken at some point and the dependency requirements violated for at least one package. Packages involved in circular dependencies may not be able to rely on their dependencies being configured before they themselves are configured, depending on which side of the break of the circular dependency loop they happen to be on. If one of the packages in the loop has no `postinst` script, then the cycle will be broken at that package; this ensures that all `postinst` scripts are run with their dependencies properly configured if this is possible. Otherwise the breaking point is arbitrary. Packages should therefore avoid circular dependencies where possible, particularly if they have `postinst` scripts.

The meaning of the five dependency fields is as follows:

Depends

This declares an absolute dependency. A package will not be configured unless all of the packages listed in its `Depends` field have been correctly configured (unless there is a circular dependency as described above).

The `Depends` field should be used if the depended-on package is required for the depending package to provide a significant amount of functionality.

The `Depends` field should also be used if the `postinst` or `prerm` scripts require the depended-on package to be unpacked or configured in order to run. In the case of `postinst configure`, the depended-on packages will be unpacked and configured first. (If both packages are involved in a dependency loop, this might not work as expected; see the explanation a few paragraphs back.) In the case of `prerm` or other `postinst` actions, the package dependencies will normally be at least unpacked, but they may be only “Half-Installed” if a previous upgrade of the dependency failed.

Finally, the `Depends` field should be used if the depended-on package is needed by the `postrm` script to fully clean up after the package removal. There is no guarantee that package dependencies will be available when `postrm` is run, but the depended-on package is more likely to be available if the package declares a dependency (particularly in the case of `postrm remove`). The `postrm` script must gracefully skip actions that require a dependency if that dependency isn't available.

Recommends

This declares a strong, but not absolute, dependency.

The `Recommends` field should list packages that would be found together with this one in all but unusual installations.

² This approach makes dependency resolution easier. If two packages A and B are being upgraded, the installed package A depends on exactly the installed package B, and the new package A depends on exactly the new package B (a common situation when upgrading shared libraries and their corresponding development packages), satisfying the dependencies at every stage of the upgrade would be impossible. This relaxed restriction means that both new packages can be unpacked together and then configured in their dependency order.

Suggests

This is used to declare that one package may be more useful with one or more others. Using this field tells the packaging system and the user that the listed packages are related to this one and can perhaps enhance its usefulness, but that installing this one without them is perfectly reasonable.

Enhances

This field is similar to `Suggests` but works in the opposite direction. It is used to declare that a package can enhance the functionality of another package.

Pre-Depends

This field is like `Depends`, except that it also forces `dpkg` to complete installation of the packages named before even starting the installation of the package which declares the pre-dependency, as follows:

When a package declaring a pre-dependency is about to be *unpacked* the pre-dependency can be satisfied if the depended-on package is either fully configured, *or even if* the depended-on package(s) are only in the “Unpacked” or the “Half-Configured” state, provided that they have been configured correctly at some point in the past (and not removed or partially removed since). In this case, both the previously-configured and currently “Unpacked” or “Half-Configured” versions must satisfy any version clause in the `Pre-Depends` field.

When the package declaring a pre-dependency is about to be *configured*, the pre-dependency will be treated as a normal `Depends`. It will be considered satisfied only if the depended-on package has been correctly configured. However, unlike with `Depends`, `Pre-Depends` does not permit circular dependencies to be broken. If a circular dependency is encountered while attempting to honor `Pre-Depends`, the installation will be aborted.

`Pre-Depends` are also required if the `preinst` script depends on the named package. It is best to avoid this situation if possible.

`Pre-Depends` should be used sparingly, preferably only by packages whose premature upgrade or installation would hamper the ability of the system to continue with any upgrade that might be in progress.

You should not specify a `Pre-Depends` entry for a package before this has been discussed on the `debian-devel` mailing list and a consensus about doing that has been reached. See [Dependencies](#).

When selecting which level of dependency to use you should consider how important the depended-on package is to the functionality of the one declaring the dependency. Some packages are composed of components of varying degrees of importance. Such a package should list using `Depends` the package(s) which are required by the more important components. The other components’ requirements may be mentioned as `Suggests` or `Recommends`, as appropriate to the components’ relative importance.

7.3 Packages which break other packages - Breaks

When one binary package declares that it breaks another, `dpkg` will refuse to allow the package which declares `Breaks` to be unpacked unless the broken package is deconfigured first, and it will refuse to allow the broken package to be reconfigured.

A package will not be regarded as causing breakage merely because its configuration files are still installed; it must be at least “Half-Installed”.

A special exception is made for packages which declare that they break their own package name or a virtual package which they provide (see below): this does not count as a real breakage.

Normally a `Breaks` entry will have an “earlier than” version clause; such a `Breaks` is introduced in the version of an (implicit or explicit) dependency which violates an assumption or reveals a bug in earlier versions of the broken package, or which takes over a file from earlier versions of the package named in `Breaks`. This use of `Breaks` will inform higher-level package management tools that the broken package must be upgraded before the new one.

If the breaking package also overwrites some files from the older package, it should use `Replaces` to ensure this goes smoothly. See [Overwriting files and replacing packages - Replaces](#) for a full discussion of taking over files from other packages, including how to use `Breaks` in those cases.

Many of the cases where `Breaks` should be used were previously handled with `Conflicts` because `Breaks` did not yet exist. Many `Conflicts` fields should now be `Breaks`. See *Conflicting binary packages - Conflicts* for more information about the differences.

7.4 Conflicting binary packages - Conflicts

When one binary package declares a conflict with another using a `Conflicts` field, `dpkg` will refuse to allow them to be unpacked on the system at the same time. This is a stronger restriction than `Breaks`, which prevents the broken package from being configured while the breaking package is in the “Unpacked” state but allows both packages to be unpacked at the same time.

If one package is to be unpacked, the other must be removed first. If the package being unpacked is marked as replacing (see *Overwriting files and replacing packages - Replaces*, but note that `Breaks` should normally be used in this case) the one on the system, or the one on the system is marked as deselected, or both packages are marked `Essential`, then `dpkg` will automatically remove the package which is causing the conflict. Otherwise, it will halt the installation of the new package with an error. This mechanism is specifically designed to produce an error when the installed package is `Essential`, but the new package is not.

A package will not cause a conflict merely because its configuration files are still installed; it must be at least “Half-Installed”.

A special exception is made for packages which declare a conflict with their own package name, or with a virtual package which they provide (see below): this does not prevent their installation, and allows a package to conflict with others providing a replacement for it. You use this feature when you want the package in question to be the only package providing some feature.

Normally, `Breaks` should be used instead of `Conflicts` since `Conflicts` imposes a stronger restriction on the ordering of package installation or upgrade and can make it more difficult for the package manager to find a correct solution to an upgrade or installation problem. `Breaks` should be used

- when moving a file from one package to another (see *Overwriting files and replacing packages - Replaces*),
- when splitting a package (a special case of the previous one), or
- when the breaking package exposes a bug in or interacts badly with particular versions of the broken package.

`Conflicts` should be used

- when two packages provide the same file and will continue to do so,
- in conjunction with `Provides` when only one package providing a given virtual facility can be unpacked at a time (see *Virtual packages - Provides*),
- in other cases where one must prevent simultaneous installation of two packages for reasons that are ongoing (not fixed in a later version of one of the packages) or that must prevent both packages from being unpacked at the same time, not just configured.

Be aware that adding `Conflicts` is normally not the best solution when two packages provide the same files. Depending on the reason for that conflict, using alternatives or renaming the files is often a better approach. See, for example, *Binaries*.

Neither `Breaks` nor `Conflicts` should be used unless two packages cannot be installed at the same time or installing them both causes one of them to be broken or unusable. Having similar functionality or performing the same tasks as another package is not sufficient reason to declare `Breaks` or `Conflicts` with that package.

A `Conflicts` entry may have an “earlier than” version clause if the reason for the conflict is corrected in a later version of one of the packages. However, normally the presence of an “earlier than” version clause is a sign that `Breaks` should have been used instead. An “earlier than” version clause in `Conflicts` prevents `dpkg` from upgrading or installing the package which declares such a conflict until the upgrade or removal of the conflicted-with package has been completed, which is a strong restriction.

7.5 Virtual packages - Provides

As well as the names of actual (“concrete”) packages, the package relationship fields `Depends`, `Recommends`, `Suggests`, `Enhances`, `Pre-Depends`, `Breaks`, `Conflicts`, `Build-Depends`, `Build-Depends-Indep`, `Build-Depends-Arch`, `Build-Conflicts`, `Build-Conflicts-Indep` and `Build-Conflicts-Arch` may mention “virtual packages”.

A *virtual package* is one which appears in the `Provides` control field of another package. The effect is as if the package(s) which provide a particular virtual package name had been listed by name everywhere the virtual package name appears. (See also [Virtual packages](#))

If there are both concrete and virtual packages of the same name, then the dependency may be satisfied (or the conflict caused) by either the concrete package with the name in question or any other concrete package which provides the virtual package with the name in question. This is so that, for example, supposing we have

```
Package: foo
Depends: bar
```

and someone else releases an enhanced version of the `bar` package they can say:

```
Package: bar-plus
Provides: bar
```

and the `bar-plus` package will now also satisfy the dependency for the `foo` package.

A `Provides` field may contain version numbers, and such a version number will be considered when considering a dependency on or conflict with the virtual package name. For example, given the following packages:

```
Package: foo
Depends: bar (>= 1.0)

Package: bar
Version: 0.9

Package: bar-plus
Provides: bar (= 1.0)
```

the `bar-plus` package will satisfy the dependency for the `foo` package with the virtual package name, as above. If the `Provides` field does not specify a version number, it will not satisfy versioned dependencies or violate versioned `Conflicts` or `Breaks`. For example, given the following packages:

```
Package: foo
Depends: bar (>= 1.0)

Package: bar
Version: 0.9

Package: bar-plus
Provides: bar (= 1.0)

Package: bar-clone
Provides: bar
```

the `bar-plus` package will satisfy the dependency for the `foo` package, but the `bar-clone` package will not.

To specify which of a set of real packages should be the default to satisfy a particular dependency on a virtual package, list the real package as an alternative before the virtual one.

If the virtual package represents a facility that can only be provided by one real package at a time, such as the mail-transport-agent virtual package that requires installation of a binary that would conflict with all other providers of that virtual package (see *Mail transport, delivery and user agents*), all packages providing that virtual package should also declare a conflict with it using `Conflicts`. This will ensure that at most one provider of that virtual package is unpacked or installed at a time.

7.6 Overwriting files and replacing packages - `Replaces`

Packages can declare in their control file that they should overwrite files in certain other packages, or completely replace other packages. The `Replaces` control field has these two distinct purposes.

7.6.1 Overwriting files in other packages

It is usually an error for a package to contain files which are on the system in another package. However, if the overwriting package declares that it `Replaces` the one containing the file being overwritten, then `dpkg` will replace the file from the old package with that from the new. The file will no longer be listed as “owned” by the old package and will be taken over by the new package. Normally, `Breaks` should be used in conjunction with `Replaces`.³

For example, if a package `foo` is split into `foo` and `foo-data` starting at version 1.2-3, `foo-data` would have the fields

```
Replaces: foo (<< 1.2-3)
Breaks: foo (<< 1.2-3)
```

in its control file. The new version of the package `foo` would normally have the field

```
Depends: foo-data (>= 1.2-3)
```

(or possibly `Recommends` or even `Suggests` if the files moved into `foo-data` are not required for normal operation).

If a package is completely replaced in this way, so that `dpkg` does not know of any files it still contains, it is considered to have “disappeared”. It will be marked as not wanted on the system (selected for removal) and “Not-Installed”. Any `conffiles` details noted for the package will be ignored, as they will have been taken over by the overwriting package. The package’s `postrm` script will be run with a special argument to allow the package to do any final cleanup required. See *Summary of ways maintainer scripts are called*.⁴

For this usage of `Replaces`, virtual packages (see *Virtual packages - Provides*) are not considered when looking at a `Replaces` field. The packages declared as being replaced must be mentioned by their real names.

This usage of `Replaces` only takes effect when both packages are at least partially on the system at once. It is not relevant if the packages conflict unless the conflict has been overridden.

7.6.2 Replacing whole packages, forcing their removal

Second, `Replaces` allows the packaging system to resolve which package should be removed when there is a conflict (see *Conflicting binary packages - Conflicts*). This usage only takes effect when the two packages *do* conflict, so that the two usages of this field do not interfere with each other.

In this situation, the package declared as being replaced can be a virtual package, so for example, all mail transport agents (MTAs) would have the following fields in their control files:

³ To see why `Breaks` is normally needed in addition to `Replaces`, consider the case of a file in the package `foo` being taken over by the package `foo-data`. `Replaces` will allow `foo-data` to be installed and take over that file. However, without `Breaks`, nothing requires `foo` to be upgraded to a newer version that knows it does not include that file and instead depends on `foo-data`. Nothing would prevent the new `foo-data` package from being installed and then removed, removing the file that it took over from `foo`. After that operation, the package manager would think the system was in a consistent state, but the `foo` package would be missing one of its files.

⁴ `Replaces` is a one way relationship. You have to install the replacing package after the replaced package.

```
Provides: mail-transport-agent
Conflicts: mail-transport-agent
Replaces: mail-transport-agent
```

ensuring that only one MTA can be unpacked at any one time. See *Virtual packages - Provides* for more information about this example.

7.7 Relationships between source and binary packages - Build-Depends, Build-Depends-Indep, Build-Depends-Arch, Build-Conflicts, Build-Conflicts-Indep, Build-Conflicts-Arch

Source packages that require certain binary packages to be installed or absent at the time of building the package may declare relationships to those binary packages.

This is done using the `Build-Depends`, `Build-Depends-Indep`, `Build-Depends-Arch`, `Build-Conflicts`, `Build-Conflicts-Indep` and `Build-Conflicts-Arch` control fields.

Build-dependencies on “build-essential” binary packages can be omitted. Please see *Package relationships* for more information.

The dependencies and conflicts they define must be satisfied (as defined earlier for binary packages) in order to invoke the targets in `debian/rules`, as follows:

clean

Only the `Build-Depends` and `Build-Conflicts` fields must be satisfied when this target is invoked.

build-arch, and binary-arch

The `Build-Depends`, `Build-Conflicts`, `Build-Depends-Arch`, and `Build-Conflicts-Arch` fields must be satisfied when these targets are invoked.

build-indep, and binary-indep

The `Build-Depends`, `Build-Conflicts`, `Build-Depends-Indep`, and `Build-Conflicts-Indep` fields must be satisfied when these targets are invoked.

build and binary

The `Build-Depends`, `Build-Conflicts`, `Build-Depends-Indep`, `Build-Conflicts-Indep`, `Build-Depends-Arch`, and `Build-Conflicts-Arch` fields must be satisfied when these targets are invoked.

Alternative dependencies are allowed in the `Build-Depends`, `Build-Depends-Indep`, and `Build-Depends-Arch` fields, but Debian’s autobuilders normally discard the dependencies after the first. This is done to give alternative dependencies a consistent interpretation that reduces the risk of inconsistencies between repeated builds. If, for example, the first-listed dependency would normally be available but is temporarily not installable, the autobuilder fails rather than install a subsequent dependency that may significantly change the behavior of the package.

More specifically, Debian autobuilders perform the following transformation on alternative dependencies in the `Build-Depends`, `Build-Depends-Indep`, and `Build-Depends-Arch` fields:

1. Discard any alternatives that are restricted to architectures that do not match the host architecture.
2. Discard any alternatives specifying different package names than the now-first alternative. (Alternatives specifying the same package name are kept to permit relationships such as `foo (<= x) | foo (>= y).`)

For example, an autobuilder for the `amd64` architecture would treat the following dependency:

```
foo-special [armhf] | foo (<= 4) | foo (>= 4.2) | bar
```

as if it were:

```
foo (<= 4) | foo (>= 4.2)
```

The normal effect is to use only the first alternative that is valid on the relevant architecture and fail if that alternative is not installable.

While this rule for build dependencies may limit the usefulness of alternatives, they can still be used to provide flexibility when building the package outside of Debian’s autobuilders.

The autobuilders for the Debian backports and experimental suites do not perform this transformation and instead use the same dependency resolution rules as normal package installations to choose which alternative dependency to install.

7.8 Additional source packages used to build the binary - *Built-Using*

Some binary packages incorporate parts of other packages when built but do not have to depend on those packages. Examples include linking with static libraries or incorporating source code from another package during the build. In this case, the source packages of those other packages are part of the complete source (the binary package is not reproducible without them).

When the license of either the incorporated parts or the incorporating binary package requires that the full source code of the incorporating binary package be made available, the *Built-Using* field must list the corresponding source package for any affected binary package incorporated during the build,⁵ including an “exactly equal” (“=”) version relation on the version that was used to build that version of the incorporating binary package.⁶

This causes the Debian archive to retain the versions of the source packages that were actually incorporated. In particular, if the versions of the incorporated parts are updated but the incorporating binary package is not rebuilt, the older versions of the incorporated parts will remain in the archive in order to satisfy the license.

A package using the source code from the `gcc-4.6-source` binary package built from the `gcc-4.6` source package would have this field in its control file:

```
Built-Using: gcc-4.6 (= 4.6.0-11)
```

A package including binaries from `grub2` and `loadlin` would have this field in its control file:

```
Built-Using: grub2 (= 1.99-9), loadlin (= 1.6e-1)
```

This field should be used only when there are license or DFSG requirements to retain the referenced source packages. It should not be added solely as a way to locate packages that need to be rebuilt against newer versions of their build dependencies.

⁵ *Build-Depends* in the source package is not adequate since it (rightfully) does not document the exact version used in the build.

⁶ The archive software might reject packages that refer to non-existent sources.

SHARED LIBRARIES

Packages containing shared libraries must be constructed with a little care to make sure that the shared library is always available. This is especially important for packages whose shared libraries are vitally important, such as the C library (currently `libc6`).

This section deals only with public shared libraries: shared libraries that are placed in directories searched by the dynamic linker by default or which are intended to be linked against normally and possibly used by other, independent packages. Shared libraries that are internal to a particular package or that are only loaded as dynamic modules are not covered by this section and are not subject to its requirements.

A shared library is identified by the `SONAME` attribute stored in its dynamic section. When a binary is linked against a shared library, the `SONAME` of the shared library is recorded in the binary's `NEEDED` section so that the dynamic linker knows that library must be loaded at runtime. The shared library file's full name (which usually contains additional version information not needed in the `SONAME`) is therefore normally not referenced directly. Instead, the shared library is loaded by its `SONAME`, which exists on the file system as a symlink pointing to the full name of the shared library. This symlink must be provided by the package. *Run-time shared libraries* describes how to do this.¹

When linking a binary or another shared library against a shared library, the `SONAME` for that shared library is not yet known. Instead, the shared library is found by looking for a file matching the library name with `.so` appended. This file exists on the file system as a symlink pointing to the shared library.

Shared libraries are normally split into several binary packages. The `SONAME` symlink is installed by the runtime shared library package, and the bare `.so` symlink is installed in the development package since it's only used when linking binaries or shared libraries. However, there are some exceptions for unusual shared libraries or for shared libraries that are also loaded as dynamic modules by other programs.

This section is primarily concerned with how the separation of shared libraries into multiple packages should be done and how dependencies on and between shared library binary packages are managed in Debian. *Libraries* should be read in conjunction with this section and contains additional rules for the files contained in the shared library packages.

8.1 Run-time shared libraries

The run-time shared library must be placed in a package whose name changes whenever the `SONAME` of the shared library changes. This allows several versions of the shared library to be installed at the same time, allowing installation of the new version of the shared library without immediately breaking binaries that depend on the old version.²

¹ This is a convention of shared library versioning, but not a requirement. Some libraries use the `SONAME` as the full library file name instead and therefore do not need a symlink. Most, however, encode additional information about backwards-compatible revisions as a minor version number in the file name. The `SONAME` itself only changes when binaries linked with the earlier version of the shared library may no longer work, but the filename may change with each release of the library. See *Run-time shared libraries* for more information.

² There are some exceptional situations in which co-installation of two versions of a shared library is not safe, and the new shared library package has to conflict with the previous shared library package. This is never desirable, since it causes significant disruption during upgrades and potentially breaks unpackaged third-party binaries, but is sometimes unavoidable. These situations are sufficiently rare that they usually warrant project-wide discussion, and are complex enough that the rules for them cannot be codified in Debian Policy.

Normally, the run-time shared library and its SONAME symlink should be placed in a package named `librarynamesoversion`, where *soversion* is the version number in the SONAME of the shared library. Alternatively, if it would be confusing to directly append *soversion* to `libraryname` (if, for example, `libraryname` itself ends in a number), you should use `libraryname-soversion` instead.³

To determine the *soversion*, look at the SONAME of the library, stored in the ELF SONAME attribute. It is usually of the form `name.so.major-version` (for example, `libz.so.1`). The version part is the part which comes after `.so.`, so in that example it is 1. The soname may instead be of the form `name-major-version.so`, such as `libdb-5.1.so`, in which case the name would be `libdb` and the version would be 5.1.

If you have several shared libraries built from the same source tree, you may lump them all together into a single shared library package provided that all of their SONAMES will always change together. Be aware that this is not normally the case, and if the SONAMES do not change together, upgrading such a merged shared library package will be unnecessarily difficult because of file conflicts with the old version of the package. When in doubt, always split shared library packages so that each binary package installs a single shared library.

Every time the shared library ABI changes in a way that could break binaries linked against older versions of the shared library, the SONAME of the library and the corresponding name for the binary package containing the runtime shared library should change. Normally, this means the SONAME should change any time an interface is removed from the shared library or the signature of an interface (the number of parameters or the types of parameters that it takes, for example) is changed. This practice is vital to allowing clean upgrades from older versions of the package and clean transitions between the old ABI and new ABI without having to upgrade every affected package simultaneously.

The SONAME and binary package name need not, and indeed normally should not, change if new interfaces are added but none are removed or changed, since this will not break binaries linked against the old shared library. Correct versioning of dependencies on the newer shared library by binaries that use the new interfaces is handled via the `symbols` or `shlibs` system (see *Dependencies between the library and other packages*).

The package should install the shared libraries under their normal names. For example, the `libgdbm3` package should install `libgdbm.so.3.0.0` as `/usr/lib/libgdbm.so.3.0.0`. The files should not be renamed or re-linked by any `prepm` or `postrm` scripts; `dpkg` will take care of renaming things safely without affecting running programs, and attempts to interfere with this are likely to lead to problems.

Shared libraries should not be installed executable, since the dynamic linker does not require this and trying to execute a shared library usually results in a core dump.

The run-time library package should include the symbolic link for the SONAME that `ldconfig` would create for the shared libraries. For example, the `libgdbm3` package should include a symbolic link from `/usr/lib/libgdbm.so.3` to `libgdbm.so.3.0.0`. This is needed so that the dynamic linker (for example `ld.so` or `ld-linux.so.*`) can find the library between the time that `dpkg` installs it and the time that `ldconfig` is run in the `postinst` script.⁴

8.1.1 ldconfig

Any package installing shared libraries in one of the default library directories of the dynamic linker (which are currently `/usr/lib` and `/lib`) or a directory that is listed in `/etc/ld.so.conf`⁵ must use `ldconfig` to update the shared library system.

³ The following command, when run on a shared library, will output the name to be used for the Debian package containing that shared library:

```
objdump -p /path/to/libfoo-bar.so.1.2.3 \
| sed -n -e's/^[[:space:]]*SONAME[[:space:]]*//p' \
| LC_ALL=C sed -r -e's/([0-9])\.so\.\1-;/ s/\.so(\.|$)//; y/_/-/; s/(.*)/L&/'
```

⁴ The package management system requires the library to be placed before the symbolic link pointing to it in the `.deb` file. This is so that when `dpkg` comes to install the symlink (overwriting the previous symlink pointing at an older version of the library), the new shared library is already in place. In the past, this was achieved by creating the library in the temporary packaging directory before creating the symlink. Unfortunately, this was not always effective, since the building of the tar file in the `.deb` depended on the behavior of the underlying file system. Some file systems (such as `reiserfs`) reorder the files so that the order of creation is forgotten. Since version 1.7.0, `dpkg` reorders the files itself as necessary when building a package. Thus it is no longer important to concern oneself with the order of file creation.

⁵ These are currently `/usr/local/lib` plus directories under `/lib` and `/usr/lib` matching the multiarch triplet for the system architecture.

Any such package must have the line `activate-noawait ldconfig` in its `triggers` control file (i.e. `DEBIAN/triggers`).

8.2 Shared library support files

If your package contains files whose names do not change with each change in the library shared object version, you must not put them in the shared library package. Otherwise, several versions of the shared library cannot be installed at the same time without filename clashes, making upgrades and transitions unnecessarily difficult.

It is recommended that supporting files and run-time support programs that do not need to be invoked manually by users, but are nevertheless required for the package to function, be placed (if they are binary) in a subdirectory of `/usr/lib`, preferably under `/usr/lib/package-name`. If the program or file is architecture independent, the recommendation is for it to be placed in a subdirectory of `/usr/share` instead, preferably under `/usr/share/package-name`. Following the package-name naming convention ensures that the file names change when the shared object version changes.

Run-time support programs that use the shared library but are not required for the library to function or files used by the shared library that can be used by any version of the shared library package should instead be put in a separate package. This package might typically be named `libraryname-tools`; note the absence of the soversion in the package name.

Files and support programs only useful when compiling software against the library should be included in the development package for the library.⁶

8.3 Static libraries

The static library (`libraryname.a`) is usually provided in addition to the shared version. It is placed into the development package (see below).

In some cases, it is acceptable for a library to be available in static form only; these cases include:

- libraries for languages whose shared library support is immature or unstable
- libraries whose interfaces are in flux or under development (commonly the case when the library's major version number is zero, or where the ABI breaks across patchlevels)
- libraries which are explicitly intended to be available only in static form by their upstream author(s)

8.4 Development files

If there are development files associated with a shared library, the source package needs to generate a binary development package named `libraryname-dev`, or if you need to support multiple development versions at a time, `librarynameapiversion-dev`. Installing the development package must result in installation of all the development files necessary for compiling programs against that shared library.⁷

In case several development versions of a library exist, you may need to use `dpkg`'s Conflicts mechanism (see *Conflicting binary packages - Conflicts*) to ensure that the user only installs one development version at a time (as different development versions are likely to have the same header files in them, which would cause a filename clash if both were unpacked).

The development package should contain a symlink for the associated shared library without a version number. For example, the `libgdbm-dev` package should include a symlink from `/usr/lib/libgdbm.so` to `libgdbm.so.3.0.0`. This symlink is needed by the linker (`ld`) when compiling packages, as it will only look for `libgdbm.so` when compiling dynamically.

⁶ For example, a `package-name-config` script or `pkg-config` configuration files.

⁷ This wording allows the development files to be split into several packages, such as a separate architecture-independent `libraryname-headers`, provided that the development package depends on all the required additional packages.

If the package provides Ada Library Information (`*.ali`) files for use with GNAT, these files must be installed read-only (mode 0444) so that GNAT will not attempt to recompile them. This overrides the normal file mode requirements given in *Permissions and owners*.

8.5 Dependencies between the packages of the same library

Typically the development version should have an exact version dependency on the runtime library, to make sure that compilation and linking happens correctly. The `${binary:Version}` substitution variable can be useful for this purpose.⁸

8.6 Dependencies between the library and other packages

If a package contains a binary or library which links to a shared library, we must ensure that, when the package is installed on the system, all of the libraries needed are also installed. These dependencies must be added to the binary package when it is built, since they may change based on which version of a shared library the binary or library was linked with even if there are no changes to the source of the binary (for example, symbol versions change, macros become functions or vice versa, or the binary package may determine at compile-time whether new library interfaces are available and can be called). To allow these dependencies to be constructed, shared libraries must provide either a `symbols` file or a `shlibs` file. These provide information on the package dependencies required to ensure the presence of interfaces provided by this library. Any package with binaries or libraries linking to a shared library must use these files to determine the required dependencies when it is built. Other packages which use a shared library (for example using `dlopen()`) should compute appropriate dependencies using these files at build time as well.

The two mechanisms differ in the degree of detail that they provide. A `symbols` file documents, for each symbol exported by a library, the minimal version of the package any binary using this symbol will need. This is typically the version of the package in which the symbol was introduced. This information permits detailed analysis of the symbols used by a particular package and construction of an accurate dependency, but it requires the package maintainer to track more information about the shared library.

A `shlibs` file, in contrast, only documents the last time the library ABI changed in any way. It only provides information about the library as a whole, not individual symbols. When a package is built using a shared library with only a `shlibs` file, the generated dependency will require a version of the shared library equal to or newer than the version of the last ABI change. This generates unnecessarily restrictive dependencies compared to `symbols` files if none of the symbols used by the package have changed. This, in turn, could make upgrades needlessly complex and unnecessarily restrict use of the package on systems with older versions of the shared libraries.

`shlibs` files also only support a limited range of library SONAMEs, making it difficult to use `shlibs` files in some unusual corner cases.⁹

`symbols` files are therefore recommended for most shared library packages since they provide more accurate dependencies. For most C libraries, the additional detail required by `symbols` files is not too difficult to maintain. However, maintaining exhaustive symbols information for a C++ library can be quite onerous, so `shlibs` files may be more appropriate for most C++ libraries. Libraries with a corresponding `udeb` must also provide a `shlibs` file, since the `udeb` infrastructure does not use `symbols` files.

8.6.1 Generating dependencies on shared libraries

When a package that contains any shared libraries or compiled binaries is built, it must run `dpkg-shlibdeps` on each shared library and compiled binary to determine the libraries used and hence the dependencies needed by the package.¹⁰ To do this, put a call to `dpkg-shlibdeps` into your `debian/rules` file in the source package. List all of the compiled

⁸ Previously, `${Source-Version}` was used, but its name was confusing and it has been deprecated since `dpkg` 1.13.19.

⁹ A `shlibs` file represents an SONAME as a library name and version number, such as `libfoo VERSION`, instead of recording the actual SONAME. If the SONAME doesn't match one of the two expected formats (`libfoo-VERSION.so` or `libfoo.so.VERSION`), it cannot be represented.

¹⁰ `dpkg-shlibdeps` will use a program like `objdump` or `readelf` to find the libraries and the symbols in those libraries directly needed by the binaries or shared libraries in the package.

binaries, libraries, or loadable modules in your package.¹¹ `dpkg-shlibdeps` will use the `symbols` or `shlibs` files installed by the shared libraries to generate dependency information. The package must then provide a substitution variable into which the discovered dependency information can be placed.

If you are creating a `udeb` for use in the Debian Installer, you will need to specify that `dpkg-shlibdeps` should use the dependency line of type `udeb` by adding the `-tudeb` option.¹² If there is no dependency line of type `udeb` in the `shlibs` file, `dpkg-shlibdeps` will fall back to the regular dependency line.

`dpkg-shlibdeps` puts the dependency information into the `debian/substvars` file by default, which is then used by `dpkg-gencontrol`. You will need to place a `${shlibs:Depends}` variable in the `Depends` field in the control file of every binary package built by this source package that contains compiled binaries, libraries, or loadable modules. If you have multiple binary packages, you will need to call `dpkg-shlibdeps` on each one which contains compiled libraries or binaries. For example, you could use the `-T` option to the `dpkg` utilities to specify a different `substvars` file for each binary package.¹³

For more details on `dpkg-shlibdeps`, see its manual page.

We say that a binary `foo` *directly* uses a library `libbar` if it is explicitly linked with that library (that is, the library is listed in the ELF `NEEDED` attribute, caused by adding `-lbar` to the link line when the binary is created). Other libraries that are needed by `libbar` are linked *indirectly* to `foo`, and the dynamic linker will load them automatically when it loads `libbar`. A package should depend on the libraries it directly uses, but not the libraries it only uses indirectly. The dependencies for the libraries used directly will automatically pull in the indirectly-used libraries. `dpkg-shlibdeps` will handle this logic automatically, but package maintainers need to be aware of this distinction between directly and indirectly using a library if they have to override its results for some reason.¹⁴

8.6.2 Shared library ABI changes

Maintaining a shared library package using either `symbols` or `shlibs` files requires being aware of the exposed ABI of the shared library and any changes to it. Both `symbols` and `shlibs` files record every change to the ABI of the shared library; `symbols` files do so per public symbol, whereas `shlibs` files record only the last change for the entire library.

There are two types of ABI changes: ones that are backward-compatible and ones that are not. An ABI change is backward-compatible if any reasonable program or library that was linked with the previous version of the shared library will still work correctly with the new version of the shared library.¹⁵ Adding new symbols to the shared library is a backward-compatible change. Removing symbols from the shared library is not. Changing the behavior of a symbol may or may not be backward-compatible depending on the change; for example, changing a function to accept a new enum constant not previously used by the library is generally backward-compatible, but changing the members of a struct that is passed into library functions is generally not unless the library takes special precautions to accept old versions of the data structure.

ABI changes that are not backward-compatible normally require changing the `SONAME` of the library and therefore the shared library package name, which forces rebuilding all packages using that shared library to update their dependencies and allow them to use the new version of the shared library. For more information, see [Run-time shared libraries](#). The remainder of this section will deal with backward-compatible changes.

¹¹ The easiest way to call `dpkg-shlibdeps` correctly is to use a package helper framework such as `debhelper`. If you are using `debhelper`, the `dh_shlibdeps` program will do this work for you. It will also correctly handle multi-binary packages.

¹² `dh_shlibdeps` from the `debhelper` suite will automatically add this option if it knows it is processing a `udeb`.

¹³ Again, `dh_shlibdeps` and `dh_gencontrol` will handle everything except the addition of the variable to the control file for you if you're using `debhelper`, including generating separate `substvars` files for each binary package and calling `dpkg-gencontrol` with the appropriate flags.

¹⁴ A good example of where this helps is the following: We could update `libimlib` with a new version that supports a new revision of a graphics format called `dgf` (but retaining the same major version number) and depends on a new library package `libdgf4` instead of the older `libdgf3`. If we used `ldd` to add dependencies for every library directly or indirectly linked with a binary, every package that uses `libimlib` would need to be recompiled so it would also depend on `libdgf4` in order to retire the older `libdgf3` package. Since dependencies are only added based on ELF `NEEDED` attribute, packages using `libimlib` can rely on `libimlib` itself having the dependency on an appropriate version of `libdgf` and do not need rebuilding.

¹⁵ An example of an “unreasonable” program is one that uses library interfaces that are documented as internal and unsupported. If the only programs or libraries affected by a change are “unreasonable” ones, other techniques, such as declaring `Breaks` relationships with affected packages or treating their usage of the library as bugs in those packages, may be appropriate instead of changing the `SONAME`. However, the default approach is to change the `SONAME` for any change to the ABI that could break a program.

Backward-compatible changes require either updating or recording the minimal-version for that symbol in `symbols` files or updating the version in the dependencies in `shlibs` files. For more information on how to do this in the two formats, see *The symbols File Format* and *The shlibs File Format*. Below are general rules that apply to both files.

The easy case is when a public symbol is added. Simply add the version at which the symbol was introduced (for `symbols` files) or update the dependency version (for `shlibs` files). But special care should be taken to update dependency versions when the behavior of a public symbol changes. This is easy to neglect, since there is no automated method of determining such changes, but failing to update versions in this case could result in binary packages with too-weak dependencies that will fail at runtime, possibly in ways that can cause security vulnerabilities. If the package maintainer believes that a symbol behavior change could have occurred but isn't sure, it's safer to update the version rather than leave it unmodified. This may result in unnecessarily strict dependencies, but it ensures that packages whose dependencies are satisfied will work properly.

A common example of when a change to the dependency version is required is a function that takes an enum or struct argument that controls what the function does. For example:

```
enum library_op { OP_FOO, OP_BAR };
int library_do_operation(enum library_op);
```

If a new operation, `OP_BAZ`, is added, the minimal-version of `library_do_operation` (for `symbols` files) or the version in the dependency for the shared library (for `shlibs` files) must be increased to the version at which `OP_BAZ` was introduced. Otherwise, a binary built against the new version of the library (having detected at compile-time that the library supports `OP_BAZ`) may be installed with a shared library that doesn't support `OP_BAZ` and will fail at runtime when it tries to pass `OP_BAZ` into this function.

Dependency versions in either `symbols` or `shlibs` files normally should not contain the Debian revision of the package, since the library behavior is normally fixed for a particular upstream version and any Debian packaging of that upstream version will have the same behavior. In the rare case that the library behavior was changed in a particular Debian revision, appending `~` to the end of the version that includes the Debian revision is recommended, since this allows backports of the shared library package using the normal backport versioning convention to satisfy the dependency.

8.6.3 The `symbols` system

In the following sections, we will first describe where the various `symbols` files are to be found, then the `symbols` file format, and finally how to create `symbols` files if your package contains a shared library.

8.6.3.1 The `symbols` files present on the system

`symbols` files for a shared library are normally provided by the shared library package as a control file, but there are several override paths that are checked first in case that information is wrong or missing. The following list gives them in the order in which they are read by `dpkg-shlibdeps`. The first one that contains the required information is used.

`debian/*/DEBIAN/symbols`

During the package build, if the package itself contains shared libraries with `symbols` files, they will be generated in these staging directories by `dpkg-gensymbols` (see *Providing a symbols file*). `symbols` files found in the build tree take precedence over `symbols` files from other binary packages.

These files must exist before `dpkg-shlibdeps` is run or the dependencies of binaries and libraries from a source package on other libraries from that same source package will not be correct. In practice, this means that `dpkg-gensymbols` must be run before `dpkg-shlibdeps` during the package build.¹⁶

¹⁶ An example may clarify. Suppose the source package `foo` generates two binary packages, `libfoo2` and `foo-runtime`. When building the binary packages, the contents of the packages are staged in the directories `debian/libfoo2` and `debian/foo-runtime` respectively. (`debian/tmp` could be used instead of one of these.) Since `libfoo2` provides the `libfoo` shared library, it will contain a `symbols` file, which will be installed in `debian/libfoo2/DEBIAN/symbols`, eventually to be included as a control file in that package. When `dpkg-shlibdeps` is run on the executable `debian/foo-runtime/usr/bin/foo-prog`, it will examine the `debian/libfoo2/DEBIAN/symbols` file to determine whether `foo-prog`'s library dependencies are satisfied by any of the libraries provided by `libfoo2`. Since those binaries were linked against the just-built shared library as part of the build process, the `symbols` file for the newly-built `libfoo2` must take precedence over a `symbols` file for any other `libfoo2` package already installed on the system.

`/etc/dpkg/symbols/package.symbols.arch` and `/etc/dpkg/symbols/package.symbols`

Per-system overrides of shared library dependencies. These files normally do not exist. They are maintained by the local system administrator and must not be created by any Debian package.

symbols control files for packages installed on the system

The `symbols` control files for all the packages currently installed on the system are searched last. This will be the most common source of shared library dependency information. These files can be read with `dpkg-query --control-show package.symbols`.

Be aware that if a `debian/shlibs.local` exists in the source package, it will override any `symbols` files. This is the only case where a `shlibs` is used despite `symbols` files being present. See [The shlibs files present on the system](#) and [The shlibs system](#) for more information.

8.6.3.2 The `symbols` File Format

The following documents the format of the `symbols` control file as included in binary packages. These files are built from template `symbols` files in the source package by `dpkg-gensymbols`. The template files support a richer syntax that allows `dpkg-gensymbols` to do some of the tedious work involved in maintaining `symbols` files, such as handling C++ symbols or optional symbols that may not exist on particular architectures. When writing `symbols` files for a shared library package, refer to `dpkg-gensymbols(1)` for the richer syntax.

A `symbols` may contain one or more entries, one for each shared library contained in the package corresponding to that `symbols`. Each entry has the following format:

```
library-soname main-dependency-template
[| alternative-dependency-template]
[...]
[* field-name: field-value]
[...]
symbol minimal-version[ id-of-dependency-template]
```

To explain this format, we'll use the `zlib1g` package as an example, which (at the time of writing) installs the shared library `/usr/lib/libz.so.1.2.3.4`. Mandatory lines will be described first, followed by optional lines.

`library-soname` must contain exactly the value of the ELF `SONAME` attribute of the shared library. In our example, this is `libz.so.1`.¹⁷

`main-dependency-template` has the same syntax as a dependency field in a binary package control file, except that the string `#MINVER#` is replaced by a version restriction like `(>= version)` or by nothing if an unversioned dependency is deemed sufficient. The version restriction will be based on which symbols from the shared library are referenced and the version at which they were introduced (see below). In nearly all cases, `main-dependency-template` will be `package #MINVER#`, where `package` is the name of the binary package containing the shared library. This adds a simple, possibly-versioned dependency on the shared library package. In some rare cases, such as when multiple packages provide the same shared library ABI, the dependency template may need to be more complex.

In our example, the first line of the `zlib1g` `symbols` file would be:

```
libz.so.1 zlib1g #MINVER#
```

Each public symbol exported by the shared library must have a corresponding symbol line, indented by one space. `symbol` is the exported symbol (which, for C++, means the mangled symbol) followed by `@` and the symbol version, or the string `Base` if there is no symbol version. `minimal-version` is the most recent version of the shared library that changed the behavior of that symbol, whether by adding it, changing its function signature (the parameters, their types, or the return

¹⁷ This can be determined by using the command

```
readelf -d /usr/lib/libz.so.1.2.3.4 | grep SONAME
```

type), or changing its behavior in a way that is visible to a caller. `id-of-dependency-template` is an optional field that references an `alternative-dependency-template`; see below for a full description.

For example, `libz.so.1` contains the symbols `compress` and `compressBound`. `compress` has no symbol version and last changed its behavior in upstream version `1:1.1.4`. `compressBound` has the symbol version `ZLIB_1.2.0`, was introduced in upstream version `1:1.2.0`, and has not changed its behavior. Its `symbols` file therefore contains the lines:

```
compress@Base 1:1.1.4
compressBound@ZLIB_1.2.0 1:1.2.0
```

Packages using only `compress` would then get a dependency on `zlib1g` (`>= 1:1.1.4`), but packages using `compressBound` would get a dependency on `zlib1g` (`>= 1:1.2.0`).

One or more `alternative-dependency-template` lines may be provided. These are used in cases where some symbols in the shared library should use one dependency template while others should use a different template. The alternative dependency templates are used only if a symbol line contains the `id-of-dependency-template` field. The first alternative dependency template is numbered 1, the second 2, and so forth.¹⁸

Finally, the entry for the library may contain one or more metadata fields. Currently, the only supported field-name is `Build-Depends-Package`, whose value lists the *library development package* on which packages using this shared library declare a build dependency. If this field is present, `dpkg-shlibdeps` uses it to ensure that the resulting binary package dependency on the shared library is at least as strict as the source package dependency on the shared library development package.¹⁹ For our example, the `zlib1g` `symbols` file would contain:

```
* Build-Depends-Package: zlib1g-dev
```

Also see `deb-symbols(5)`.

8.6.3.3 Providing a `symbols` file

If your package provides a shared library, you should arrange to include a `symbols` control file following the format described above in that package. You must include either a `symbols` control file or a `shlibs` control file.

Normally, this is done by creating a `symbols` in the source package named `debian/package.symbols` or `debian/symbols`, possibly with `.arch` appended if the symbols information varies by architecture. This file may use the extended syntax documented in `dpkg-gensymbols(1)`. Then, call `dpkg-gensymbols` as part of the package build process. It will create `symbols` files in the package staging area based on the binaries and libraries in the package staging area and the `symbols` files in the source package.²⁰

Packages that provide `symbols` files must keep them up-to-date to ensure correct dependencies in packages that use the shared libraries. This means updating the `symbols` file whenever a new public symbol is added, changing the minimal-version field whenever a symbol changes behavior or signature in a backward-compatible way (see *Shared library ABI changes*), and changing the library-soname and main-dependency-template, and probably all of the minimal-version fields,

¹⁸ An example of where this may be needed is with a library that implements the libGL interface. All GL implementations provide the same set of base interfaces, and then may provide some additional interfaces only used by programs that require that specific GL implementation. So, for example, `libgl-mesa-glx` may use the following `symbols` file:

```
libGL.so.1 libgl1
| libgl1-mesa-glx #MINVER#
publicGLSymbol@Base 6.3-1 [...]
implementationSpecificSymbol@Base 6.5.2-7 1
[...]
```

Binaries or shared libraries using only `publicGLSymbol` would depend only on `libgl1` (which may be provided by multiple packages), but ones using `implementationSpecificSymbol` would get a dependency on `libgl1-mesa-glx` (`>= 6.5.2-7`).

¹⁹ This field should normally not be necessary, since if the behavior of any symbol has changed, the corresponding symbol minimal-version should have been increased. But including it makes the `symbols` system more robust by tightening the dependency in cases where the package using the shared library specifically requires at least a particular version of the shared library development package for some reason.

²⁰ If you are using `debhelper`, `dh_makeshlibs` will take care of calling either `dpkg-gensymbols` or generating a `shlibs` file as appropriate.

when the library changes `SONAME`. Removing a public symbol from the `symbols` file because it's no longer provided by the library normally requires changing the `SONAME` of the library. See *Run-time shared libraries* for more information on `SONAMES`.

8.6.4 The `shlibs` system

The `shlibs` system is a simpler alternative to the `symbols` system for declaring dependencies for shared libraries. It may be more appropriate for C++ libraries and other cases where tracking individual symbols is too difficult. It predated the `symbols` system and is therefore frequently seen in older packages. It is also required for `udebs`, which do not support `symbols`.

In the following sections, we will first describe where the various `shlibs` files are to be found, then how to use `dpkg-shlibdeps`, and finally the `shlibs` file format and how to create them.

8.6.4.1 The `shlibs` files present on the system

There are several places where `shlibs` files are found. The following list gives them in the order in which they are read by `dpkg-shlibdeps`. (The first one which gives the required information is used.)

`debian/shlibs.local`

This lists overrides for this package. This file should normally not be used, but may be needed temporarily in unusual situations to work around bugs in other packages, or in unusual cases where the normally declared dependency information in the installed `shlibs` file for a library cannot be used. This file overrides information obtained from any other source.

`/etc/dpkg/shlibs.override`

This lists global overrides. This list is normally empty. It is maintained by the local system administrator.

`DEBIAN/shlibs` files in the “build directory”

These files are generated as part of the package build process and staged for inclusion as control files in the binary packages being built. They provide details of any shared libraries included in the same package.

`shlibs` control files for packages installed on the system

The `shlibs` control files for all the packages currently installed on the system. These files can be read using `dpkg-query --control-show package shlibs`.

`/etc/dpkg/shlibs.default`

This file lists any shared libraries whose packages have failed to provide correct `shlibs` files. It was used when the `shlibs` setup was first introduced, but it is now normally empty. It is maintained by the `dpkg` maintainer.

If a `symbols` file for a shared library package is available, `dpkg-shlibdeps` will always use it in preference to a `shlibs`, with the exception of `debian/shlibs.local`. The latter overrides any other `shlibs` or `symbols` files.

8.6.4.2 The `shlibs` File Format

Each `shlibs` file has the same format. Lines beginning with `#` are considered to be comments and are ignored. Each line is of the form:

```
[type: ]library-name soname-version dependencies ...
```

We will explain this by reference to the example of the `zlib1g` package, which (at the time of writing) installs the shared library `/usr/lib/libz.so.1.2.3.4`.

`type` is an optional element that indicates the type of package for which the line is valid. The only type currently in use is `udeb`. The colon and space after the type are required.

`library-name` is the name of the shared library, in this case `libz`. (This must match the name part of the soname, see below.)

`soname-version` is the version part of the ELF `SONAME` attribute of the library, determined the same way that the `soversion` component of the recommended shared library package name is determined. See *Run-time shared libraries* for the details.

`dependencies` has the same syntax as a dependency field in a binary package control file. It should give details of which packages are required to satisfy a binary built against the version of the library contained in the package. See *Syntax of relationship fields* for details on the syntax, and *Shared library ABI changes* for details on how to maintain the dependency version constraint.

In our example, if the last change to the `zlib1g` package that could change behavior for a client of that library was in version `1:1.2.3.3.dfsg-1`, then the `shlibs` entry for this library could say:

```
libz 1 zlib1g (>= 1:1.2.3.3.dfsg)
```

This version restriction must be new enough that any binary built against the current version of the library will work with any version of the shared library that satisfies that dependency.

As `zlib1g` also provides a `udeb` containing the shared library, there would also be a second line:

```
udeb: libz 1 zlib1g-udeb (>= 1:1.2.3.3.dfsg)
```

8.6.4.3 Providing a `shlibs` file

To provide a `shlibs` file for a shared library binary package, create a `shlibs` file following the format described above and place it in the `DEBIAN` directory for that package during the build. It will then be included as a control file for that package.²¹

Since `dpkg-shlibdeps` reads the `DEBIAN/shlibs` files in all of the binary packages being built from this source package, all of the `DEBIAN/shlibs` files should be installed before `dpkg-shlibdeps` is called on any of the binary packages.

²¹ This is what `dh_makeshlibs` in the `debhelper` suite does. If your package also has a `udeb` that provides a shared library, `dh_makeshlibs` can automatically generate the `udeb:` lines if you specify the name of the `udeb` with the `--add-udeb` option.

THE OPERATING SYSTEM

9.1 File system hierarchy

9.1.1 File System Structure

The location of all files and directories must comply with the Filesystem Hierarchy Standard (FHS), version 3.0, with the exceptions noted below, and except where doing so would violate other terms of Debian Policy. The following exceptions to the FHS apply:

1. The FHS requirement that architecture-independent application-specific static files be located in `/usr/share` is relaxed to a suggestion. In particular, a subdirectory of `/usr/lib` may be used by a package (or a collection of packages) to hold a mixture of architecture-independent and architecture-dependent files. However, when a directory is entirely composed of architecture-independent files, it should be located in `/usr/share`.
2. The optional rules related to user specific configuration files for applications are stored in the user's home directory are relaxed. It is recommended that such files start with the `'.'` character (a "dot file"), and if an application needs to create more than one dot file then the preferred placement is in a subdirectory with a name starting with a `'.'` character, (a "dot directory"). In this case it is recommended the configuration files not start with the `'.'` character.
3. Only the dynamic linker and `libc` are allowed to install files in `/lib64`.
4. The requirement for object files, internal binaries, and libraries, including `libc.so.*`, to be located directly under `/lib{,32}` and `/usr/lib{,32}` is amended, permitting files to instead be installed to `/lib/triplet` and `/usr/lib/triplet`, where `triplet` is the value returned by `dpkg-architecture -qDEB_HOST_MULTIARCH` for the architecture of the package. Packages must not install files to any triplet path other than the one matching the architecture of that package; for instance, an `Architecture: amd64` package containing 32-bit x86 libraries must not install these libraries to `/usr/lib/i386-linux-gnu`.¹

Packages for 64-bit architectures must not install files in `/usr/lib64` or in a subdirectory of it.

The requirement for C and C++ headers files to be accessible through the search path `/usr/include/` is amended, permitting files to be accessible through the search path `/usr/include/triplet` where `triplet` is as above.²

Applications may also use a single subdirectory under `/usr/lib/triplet`.

The execution time linker/loader, `ld*`, must still be made available in the existing location under `/lib` or `/lib64` since this is part of the ELF ABI for the architecture.

5. The requirement that `/usr/local/share/man` be "synonymous" with `/usr/local/man` is relaxed to a recommendation
6. The requirement that window managers with a single configuration file call it `system.*wmrc` is removed, as is the restriction that the window manager subdirectory be named identically to the window manager name itself.

¹ This is necessary in order to reserve the directories for use in cross-installation of library packages from other architectures, as part of `multiarch`.

² This is necessary for architecture-dependent headers file to coexist in a `multiarch` setup.

7. The requirement that boot manager configuration files live in `/etc`, or at least are symlinked there, is relaxed to a recommendation.
8. `/var/run` is required to be a symbolic link to `/run`, and `/var/lock` is required to be a symbolic link to `/run/lock`.
9. The `/var/www` directory is additionally allowed.
10. The requirement for `/usr/local/share/color` to exist if `/usr/share/color` exists is relaxed to a recommendation.
11. The requirement for `/usr/local/libqual` to exist if `/libqual` or `/usr/libqual` exists (where `libqual` is a variant of `lib` such as `lib32` or `lib64`) is removed.
12. On GNU/Hurd systems, the following additional directories are allowed in the root filesystem: `/hurd` and `/servers`.³
13. As an exception to the requirement for there to be no subdirectories in `/usr/bin`, the `mh` mail-handling suite may create `/usr/bin/mh/`, as was allowed in FHS version 2.3. Other subdirectories are not allowed.

The version of this document referred here can be found in the `debian-policy` package or on [FHS \(Debian copy\)](#) alongside this manual (or, if you have the `debian-policy` installed, you can try [FHS \(local copy\)](#)). The latest version, which may be a more recent version, may be found on [FHS \(upstream\)](#). Specific questions about following the standard may be asked on the `debian-devel` mailing list, or referred to the FHS mailing list (see the [FHS web site](#) for more information).

9.1.2 Site-specific programs

As mandated by the FHS, packages must not place any files in `/usr/local`, either by putting them in the file system archive to be unpacked by `dpkg` or by manipulating them in their maintainer scripts.

However, the package may create empty directories below `/usr/local` so that the system administrator knows where to place site-specific files. These are not directories *in* `/usr/local`, but are children of directories in `/usr/local`. These directories (`/usr/local/*/dir/`) should be removed on package removal if they are empty.

Note that this applies only to directories *below* `/usr/local`, not *in* `/usr/local`. Packages must not create subdirectories in the directory `/usr/local` itself, except those listed in FHS, section 4.9. However, you may create directories below them as you wish. You must not remove any of the directories listed in 4.9, even if you created them.

If `/etc/staff-group-for-usr-local` does not exist, `/usr/local` and all subdirectories created by packages should have permissions `0755` and be owned by `root:root`. If `/etc/staff-group-for-usr-local` exists, `/usr/local` and subdirectories should have permissions `2775` (group-writable and set-group-id) and be owned by `root:staff`.

Since `/usr/local` can be mounted read-only from a remote server, `/usr/local/*/dir/` directories must be created and removed by the `postinst` and `prerm` maintainer scripts and not be included in the `.deb` archive. These scripts must not fail if either of these operations fail.

For example, the `emacsen-common` package could contain something like

```
if [ ! -e /usr/local/share/emacs ]; then
    if mkdir /usr/local/share/emacs 2>/dev/null; then
        if test -e /etc/staff-group-for-usr-local ; then
            if chown root:staff /usr/local/share/emacs; then
                chmod 2775 /usr/local/share/emacs || true
            fi
        elif chown root:root /usr/local/share/emacs; then
```

(continues on next page)

³ These directories are used to store translators and as a set of standard names for mount points, respectively.

(continued from previous page)

```

        chmod 0755 /usr/local/share/emacs || true
    fi
fi
fi

```

in its `postinst` script, and

```

rmdir /usr/local/share/emacs/site-lisp 2>/dev/null || true
rmdir /usr/local/share/emacs 2>/dev/null || true

```

in the `prerm` script. (Note that this form is used to ensure that if the script is interrupted, the directory `/usr/local/share/emacs` will still be removed.)

If you do create a directory in `/usr/local` for local additions to a package, you should ensure that settings in `/usr/local` take precedence over the equivalents in `/usr`.

However, because `/usr/local` and its contents are for exclusive use of the local administrator, a package must not rely on the presence or absence of files or directories in `/usr/local` for normal operation.

9.1.3 The system-wide mail directory

The system-wide mail directory is `/var/mail`. This directory is part of the base system and should not be owned by any particular mail agents. The use of the old location `/var/spool/mail` is deprecated, even though the spool may still be physically located there.

9.1.4 `/run` and `/run/lock`

The directory `/run` is cleared at boot, normally by being a mount point for a temporary file system. Packages therefore must not assume that any files or directories under `/run` other than `/run/lock` exist unless the package has arranged to create those files or directories since the last reboot. Normally, this is done by the package via an init script. See [Writing the scripts](#) for more information.

Packages must not include files or directories under `/run`, or under the older `/var/run` and `/var/lock` paths. The latter paths will normally be symlinks or other redirections to `/run` for backwards compatibility.

9.2 Users and groups

9.2.1 Introduction

The Debian system can be configured to use either plain or shadow passwords.

Some user ids (UIDs) and group ids (GIDs) are reserved globally for use by certain packages. Because some packages need to include files which are owned by these users or groups, or need the ids compiled into binaries, these ids must be used on any Debian system only for the purpose for which they are allocated. This is a serious restriction, and we should avoid getting in the way of local administration policies. In particular, many sites allocate users and/or local system groups starting at 100.

Apart from this we should have dynamically allocated ids, which should by default be arranged in some sensible order, but the behavior should be configurable. When maintainers choose a new hardcoded or dynamically generated username for packages to use, they should start this username with an underscore. This minimizes collisions with locally created user accounts.

Packages other than `base-passwd` must not modify `/etc/passwd`, `/etc/shadow`, `/etc/group` or `/etc/gshadow`.

9.2.2 UID and GID classes

The UID and GID numbers are divided into classes as follows:

0-99:

Globally allocated by the Debian project, the same on every Debian system. These ids will appear in the `passwd` and `group` files of all Debian systems, new ids in this range being added automatically as the `base-passwd` package is updated.

Packages which need a single statically allocated uid or gid should use one of these; their maintainers should ask the `base-passwd` maintainer for ids.

100-999:

Dynamically allocated system users and groups. Packages which need a user or group, but can have this user or group allocated dynamically and differently on each system, should use `adduser --system` to create the group and/or user. `adduser` will check for the existence of the user or group, and if necessary choose an unused id based on the ranges specified in `adduser.conf`.

1000-59999:

Dynamically allocated user accounts. By default `adduser` will choose UIDs and GIDs for user accounts in this range, though `adduser.conf` may be used to modify this behavior.

60000-64999:

Globally allocated by the Debian project, but only created on demand. The ids are allocated centrally and statically, but the actual accounts are only created on users' systems on demand.

These ids are for packages which are obscure or which require many statically-allocated ids. These packages should check for and create the accounts in `/etc/passwd` or `/etc/group` (using `adduser` if it has this facility) if necessary. Packages which are likely to require further allocations should have a "hole" left after them in the allocation, to give them room to grow.

65000-65533:

Reserved.

65534:

User `nobody`. The corresponding gid refers to the group `nogroup`.

65535:

This value *must not* be used, because it was the error return sentinel value when `uid_t` was 16 bits.

65536-99999, 600100001-4294967293:

Dynamically allocated user accounts. By default `adduser` will not allocate UIDs and GIDs in this range, to ease compatibility with legacy systems where `uid_t` is still 16 bits.

100000-600100000:

Dynamically allocated subordinate user ids. See `subuid(5)`. `useradd` in its default configuration (and thus `adduser`) automatically allocates a range of 65536 of these to each new non-system user created.

4294967294:

`(uid_t)(-2) == (gid_t)(-2)` *must not* be used, because it is used as the anonymous, unauthenticated user by some NFS implementations.

4294967295:

`(uid_t)(-1) == (gid_t)(-1)` *must not* be used, because it is the error return sentinel value.

9.2.3 Non-existent home directories

The canonical non-existent home directory is `/nonexistent`. Users who should not have a home directory should have their home directory set to this value.

The Debian autobuilders set HOME to `/nonexistent` so that packages which try to write to a home directory will fail to build.

9.3 Starting system services

Debian packages that provide system services should arrange for those services to be automatically started and stopped by the init system or service manager. This section describes how that is done.

9.3.1 Introduction

The default init system and service manager in Debian is `systemd`. Packages that wish to automatically start and stop system services must include `systemd` service units to do so, unless the service is only intended for use on systems running alternate init systems. See `systemd.service(5)` for details on the syntax of a service unit file.

In the common case that a package includes a single system service, the service unit should have the same name as the package plus the `.service` extension.

Packages including a service unit may optionally include an init script to support other init systems. In this case, the init script should have the same name as the `systemd` service unit so that `systemd` will ignore it and use the service unit instead. Packages may also support other init systems by including configuration in the native format of those init systems.

`systemd` uses dependency and ordering information contained within the enabled unit files to decide which services to run and in which order. The `sysv-rc` runlevel system for `sysvinit` uses symlinks in `/etc/rcn.d` to decide which scripts to run and in which order at boot time and when the init state (or “runlevel”) is changed. See the `README.runlevels` file shipped with `sysv-rc` for implementation details. Other alternatives might exist.

The sections below describe how to write those scripts and configure those symlinks.

9.3.2 Writing the scripts

Init scripts are placed in `/etc/init.d`. In the common case that a package starts a single service, they should be named `/etc/init.d/package`. They should accept one argument, saying what to do:

```
start
    start the service,

stop
    stop the service,

restart
    stop and restart the service if it's already running, otherwise start the service

try-restart
    restart the service if it's already running, otherwise just report success.

reload
    cause the configuration of the service to be reloaded without actually stopping and restarting the service,

force-reload
    cause the configuration to be reloaded if the service supports this, otherwise restart the service.

status
    report the current status of the service
```

The `start`, `stop`, `restart`, and `force-reload` options should be supported by all init scripts. Supporting `status` is encouraged. The `reload` and `try-restart` options are optional.

The `init.d` scripts must ensure that they will behave sensibly (i.e., returning success and not starting multiple copies of a service) if invoked with `start` when the service is already running, or with `stop` when it isn't, and that they don't

kill unfortunately-named user processes. The best way to achieve this is usually to use `start-stop-daemon` with the `--oknodo` option.

Be careful of using `set -e` in `init.d` scripts. Writing correct `init.d` scripts requires accepting various error exit statuses when daemons are already running or already stopped without aborting the `init.d` script, and common `init.d` function libraries are not safe to call with `set -e` in effect.⁴ For `init.d` scripts, it's often easier to not use `set -e` and instead check the result of each command separately.

If a service reloads its configuration automatically (as in the case of `cron`, for example), the `reload` option of the `init.d` script should behave as if the configuration has been reloaded successfully.

The `/etc/init.d` scripts must be treated as configuration files, either (if they are present in the package, that is, in the `.deb` file) by marking them as `conffiles`, or, (if they do not exist in the `.deb`) by managing them correctly in the maintainer scripts (see [Configuration files](#)). This is important since we want to give the local system administrator the chance to adapt the scripts to the local system, e.g., to disable a service without de-installing the package, or to specify some special command line options when starting a service, while making sure their changes aren't lost during the next package upgrade.

These scripts should not fail obscurely when the configuration files remain but the package has been removed, as configuration files remain on the system after the package has been removed. Only when `dpkg` is executed with the `--purge` option will configuration files be removed. In particular, as the `/etc/init.d/package` script itself is usually a `conffile`, it will remain on the system if the package is removed but not purged. Therefore, you should include a `test` statement at the top of the script, like this:

```
test -f program-executed-later-in-script || exit 0
```

Often there are some variables in the `init.d` scripts whose values control the behavior of the scripts, and which a system administrator is likely to want to change. As the scripts themselves are frequently `conffiles`, modifying them requires that the administrator merge in their changes each time the package is upgraded and the `conffile` changes. To ease the burden on the system administrator, such configurable values should not be placed directly in the script. Instead, they should be placed in a file in `/etc/default`, which typically will have the same base name as the `init.d` script. This extra file should be sourced by the script when the script runs. It must contain only variable settings and comments in POSIX.1-2017 `sh` format. It must either be a `conffile` or a configuration file maintained by the package maintainer scripts. See [Configuration files](#) for more details.

To ensure that vital configurable values are always available, the `init.d` script should set default values for each of the shell variables it uses, either before sourcing the `/etc/default/` file or afterwards using something like the `: ${VAR:=default}` syntax. Also, the `init.d` script must behave sensibly and not fail if the `/etc/default` file is deleted.

Files and directories under `/run`, including ones referred to via the compatibility paths `/var/run` and `/var/lock`, are normally stored on a temporary filesystem and are normally not persistent across a reboot. The `init.d` scripts must handle this correctly. This will typically mean creating any required subdirectories dynamically when the `init.d` script is run. See [/run and /run/lock](#) for more information.

9.3.3 Interfacing with init systems

Maintainer scripts for packages including `init` scripts must use `update-rc.d` as described below to interact with the service manager for requests such as enabling or disabling services. They should use `invoke-rc.d` as described below to invoke `init` scripts for requests such as starting and stopping service.

Directly managing the `/etc/rc?.d` links and directly invoking the `/etc/init.d/` `init` scripts should be done only by packages providing the `init` script subsystem (such as `init-system-helpers`).

⁴ `/lib/lsb/init-functions`, which assists in writing LSB-compliant `init` scripts, may fail if `set -e` is in effect and echoing status messages to the console fails, for example.

9.3.3.1 Managing the links

The program `update-rc.d` is provided for package maintainers to arrange for the proper creation and removal of `/etc/rcn.d` symbolic links, or their functional equivalent if another method is being used. It is intended for use in package maintainer scripts.

You must not include any `/etc/rcn.d` symbolic links in the actual archive or manually create or remove the symbolic links in maintainer scripts; you must use the `update-rc.d` program instead. (The former will fail if an alternative method of maintaining runlevel information is being used.) You must not include the `/etc/rcn.d` directories themselves in the archive either. (Only the `init-system-helpers` package is permitted to do so.)

To get the default behavior for your package, put in your `postinst` script:

```
update-rc.d package defaults
```

and in your `postrm`:

```
if [ "$1" = purge ]; then
    update-rc.d package remove
fi
```

The default behaviour is to enable autostarting your package's daemon. The local administrator can override this using the command `update-rc.d package disable`. If, however, the daemon should not be autostarted unless the local administrator has explicitly requested this, instead add to your `postinst` script:

```
update-rc.d package defaults-disabled
```

and add a dependency on `init-system-helpers (>= 1.50)`, which introduced the `defaults-disabled` option. Then the local administrator can enable autostarting the daemon using the command `update-rc.d package enable`.

An older practice, which should not be used, was to include a line like `DISABLED=yes` in the package's `/etc/default` file. The package's init script would not start the service until the local system administrator changed this to `DISABLED=no`, or similar. The problem with this approach was that it hides from the init system whether or not the daemon should actually be started, which leads to inconsistent and confusing behavior: `service <package> start` could return success but not start the service; services with a dependency on this service will be started even though the service isn't running; and init system status commands could incorrectly claim that the service was started.

Note that if your package changes runlevels or priority, you may have to remove and recreate the links, since otherwise the old links may persist. Refer to the documentation of `update-rc.d`.

For more information about using `update-rc.d`, please consult its man page, `update-rc.d(8)`.

It is easiest for packages not to call `update-rc.d` directly, but instead use debhelper programs that add the required `update-rc.d` calls automatically. See `dh_installinit`, `dh_installsystemd`, etc.

9.3.3.2 Running init scripts

The program `invoke-rc.d` is provided to make it easier for package maintainers to properly invoke an init script, obeying runlevel and other locally-defined constraints that might limit a package's right to start, stop and otherwise manage services. This program may be used by maintainers in their packages' scripts.

The package maintainer scripts must use `invoke-rc.d` to invoke the `/etc/init.d/*` init scripts or equivalent instead of calling them directly.

By default, `invoke-rc.d` will pass any action requests (start, stop, reload, restart...) to the `/etc/init.d` script, filtering out requests to start or restart a service out of its intended runlevels.

Most packages will simply use:

`invoke-rc.d package action`

in their `postinst` and `prerm` scripts.

A package should register its init script services using `update-rc.d` before it tries to invoke them using `invoke-rc.d`. Invocation of unregistered services may fail.

For more information about using `invoke-rc.d`, please consult its man page, *invoke-rc.d(8)*.

It is easiest for packages not to call `invoke-rc.d` directly, but instead use debhelper programs that add the required `invoke-rc.d` calls automatically. See `dh_installinit`, `dh_installsystemd`, etc.

9.3.4 Boot-time initialization

This section has been deleted.

9.3.5 Example

Examples on which you can base your `systemd` service units are available in the man page *systemd.unit(5)*. An example on which you can base your init scripts is available in the man page *init-d-script(5)*.

9.4 Console messages from `init.d` scripts

This section has been deleted.

9.5 Cron jobs

Packages must not modify the configuration file `/etc/crontab`, and they must not modify the files in `/var/spool/cron/crontabs`.

If a package wants to install a job that has to be executed via cron, it should place a file named as specified in *Cron job file names* into one or more of the following directories:

- `/etc/cron.hourly`
- `/etc/cron.daily`
- `/etc/cron.weekly`
- `/etc/cron.monthly`

As these directory names imply, the files within them are executed on an hourly, daily, weekly, or monthly basis, respectively. The exact times are listed in `/etc/crontab`.

All files installed in any of these directories must be scripts (e.g., shell scripts or Perl scripts) so that they can easily be modified by the local system administrator. In addition, they must be treated as configuration files.

If a certain job has to be executed at some other frequency or at a specific time, the package should install a file in `/etc/cron.d` with a name as specified in *Cron job file names*. This file uses the same syntax as `/etc/crontab` and is processed by `cron` automatically. The file must also be treated as a configuration file. (Note that entries in the `/etc/cron.d` directory are not handled by `anacron`. Thus, you should only use this directory for jobs which may be skipped if the system is not running.)

Unlike `crontab` files described in the IEEE Std 1003.1-2008 (POSIX.1) available from [The Open Group](#), the files in `/etc/cron.d` and the file `/etc/crontab` have seven fields; namely:

1. Minute [0,59]
2. Hour [0,23]

3. Day of the month [1,31]
4. Month of the year [1,12]
5. Day of the week ([0,6] with 0=Sunday)
6. Username
7. Command to be run

Ranges of numbers are allowed. Ranges are two numbers separated with a hyphen. The specified range is inclusive. Lists are allowed. A list is a set of numbers (or ranges) separated by commas. Step values can be used in conjunction with ranges.

The scripts or `crontab` entries in these directories should check if all necessary programs are installed before they try to execute them. Otherwise, problems will arise when a package was removed but not purged since configuration files are kept on the system in this situation.

Any `cron` daemon must provide `/usr/bin/crontab` and support normal `crontab` entries as specified in POSIX. The daemon must also support names for days and months, ranges, and step values. It has to support `/etc/crontab`, and correctly execute the scripts in `/etc/cron.d`. The daemon must also correctly execute scripts in `/etc/cron.{hourly,daily,weekly,monthly}`.

9.5.1 Cron job file names

The file name of a cron job file should normally match the name of the package from which it comes.

If a package supplies multiple cron job files in the same directory, the file names should all start with the name of the package (possibly modified as described below) followed by a hyphen (-) and a suitable suffix.

A cron job file name must not include any period or plus characters (. or +) characters as this will cause cron to ignore the file. Underscores (_) should be used instead of . and + characters.

9.6 Menus

Packages shipping applications that comply with minimal requirements described below for integration with desktop environments should register these applications in the desktop menu, following the *FreeDesktop* standard, using text files called *desktop entries*. Their format is described in the *Desktop Entry Specification* at <https://standards.freedesktop.org/desktop-entry-spec/latest/> and complementary information can be found in the *Desktop Menu Specification* at <https://standards.freedesktop.org/menu-spec/latest/>.

The desktop entry files are installed by the packages in the directory `/usr/share/applications` and the FreeDesktop menus are refreshed using *dpkg triggers*. It is therefore not necessary to depend on packages providing FreeDesktop menu systems.

Entries displayed in the FreeDesktop menu should conform to the following minima for relevance and visual integration.

- Unless hidden by default, the desktop entry must point to a PNG or SVG icon with a transparent background, providing at least the 22×22 size, and preferably up to 64×64. The icon should be neutral enough to integrate well with the default icon themes. It is encouraged to ship the icon in the default *hicolor* icon theme directories, or to use an existing icon from the *hicolor* theme.
- If the menu entry is not useful in the general case as a standalone application, the desktop entry should set the `NoDisplay` key to true, so that it can be configured to be displayed only by those who need it.
- In doubt, the package maintainer should coordinate with the maintainers of menu implementations through the *debian-desktop* mailing list in order to avoid problems with categories or bad interactions with other icons. Especially for packages which are part of installation tasks, the contents of the `NotShowIn/OnlyShowIn` keys should be validated by the maintainers of the relevant environments.

Since the FreeDesktop menu is a cross-distribution standard, the desktop entries written for Debian should be forwarded upstream, where they will benefit to other users and are more likely to receive extra contributions such as translations.

If a package installs a FreeDesktop desktop entry, it must not also install a Debian menu entry.

9.7 Multimedia handlers

Media types (formerly known as MIME types, Multipurpose Internet Mail Extensions, RFCs 2045-2049) is a mechanism for encoding files and data streams and providing meta-information about them, in particular their type and format (e.g. `image/png`, `text/html`, `audio/ogg`).

Registration of media type handlers allows programs like mail user agents and web browsers to invoke these handlers to view, edit or display media types they don't support directly.

There are two overlapping systems to associate media types to programs which can handle them. The *mailcap* system is found on a large number of Unix systems. The *FreeDesktop* system is aimed at Desktop environments. In Debian, FreeDesktop entries are automatically translated in mailcap entries, therefore packages already using desktop entries should not use the mailcap system directly.

9.7.1 Registration of media type handlers with desktop entries

Packages shipping an application able to view, edit or point to files of a given media type, or open links with a given URI scheme, should list it in the `MimeType` key of the application's *desktop entry*. For URI schemes, the relevant MIME types are `x-scheme-handler/*` (e.g. `x-scheme-handler/https`).

9.7.2 Registration of media type handlers with mailcap entries

Packages that are not using desktop entries for registration should install a file in *mailcap* (5) format (RFC 1524) in the directory `/usr/lib/mime/packages/`. The file name should be the binary package's name.

The mailcap package provides the `update-mime` program, which integrates these registrations in the `/etc/mailcap` file, using `dpkg` triggers.⁵

Packages installing desktop entries should not install mailcap entries for the same program, because the mailcap package already reads desktop entries.

Packages using these facilities *should not* depend on, recommend, or suggest `mailcap`.

9.7.3 Providing media types to files

The media type of a file is discovered by inspecting the file's extension or its *magic* (5) pattern, and interrogating a database associating them with media types.

To support new associations between media types and files, their characteristic file extensions and magic patterns should be registered to the IANA (Internet Assigned Numbers Authority). See <https://www.iana.org/assignments/media-types> and RFC 6838 for details. This information will then propagate to the systems discovering file media types in Debian, provided by the `shared-mime-info`, `media-types` and `file` packages. If registration and propagation can not be waited for, support can be asked to the maintainers of the packages mentioned above.

For files that are produced and read by a single application, it is also possible to declare this association to the *Shared MIME Info* system by installing in the directory `/usr/share/mime/packages` a file in the XML format specified at <https://standards.freedesktop.org/shared-mime-info-spec/latest/>.

⁵ Creating, modifying or removing a file in `/usr/lib/mime/packages/` using maintainer scripts will not activate the trigger. In that case, it can be done by calling `dpkg-trigger --no-await /usr/lib/mime/packages` from the maintainer script after creating, modifying, or removing the file.

9.8 Keyboard configuration

To achieve a consistent keyboard configuration so that all applications interpret a keyboard event the same way, all programs in the Debian distribution must be configured to comply with the following guidelines.

The following keys must have the specified interpretations:

<--

delete the character to the left of the cursor

Delete

delete the character to the right of the cursor

Control+H

emacs: the help prefix

The interpretation of any keyboard events should be independent of the terminal that is used, be it a virtual console, an X terminal emulator, an rlogin/telnet session, etc.

The following list explains how the different programs should be set up to achieve this:

- **<--** generates `KB_BackSpace` in X.
- **Delete** generates `KB_Delete` in X.
- X translations are set up to make `KB_Backspace` generate ASCII DEL, and to make `KB_Delete` generate `ESC [ 3 ~` (this is the vt220 escape code for the “delete character” key). This must be done by loading the X resources using `xrdb` on all local X displays, not using the application defaults, so that the translation resources used correspond to the `xmodmap` settings.
- The Linux console is configured to make **<--** generate DEL, and **Delete** generate `ESC [ 3 ~`.
- X applications are configured so that **<** deletes left, and **Delete** deletes right. Motif applications already work like this.
- Terminals should have `stty erase ^?`.
- The `xterm` terminfo entry should have `ESC [ 3 ~` for `kdch1`, just as for `TERM=linux` and `TERM=vt220`.
- Emacs is programmed to map `KB_Backspace` or the `stty erase` character to `delete-backward-char`, and `KB_Delete` or `kdch1` to `delete-forward-char`, and `^H` to help as always.
- Other applications use the `stty erase` character and `kdch1` for the two delete keys, with ASCII DEL being “delete previous character” and `kdch1` being “delete character under cursor”.

This will solve the problem except for the following cases:

- Some terminals have a **<--** key that cannot be made to produce anything except `^H`. On these terminals Emacs help will be unavailable on `^H` (assuming that the `stty erase` character takes precedence in Emacs, and has been set correctly). `M-x help` or `F1` (if available) can be used instead.
- Some operating systems use `^H` for `stty erase`. However, modern telnet versions and all rlogin versions propagate `stty` settings, and almost all UNIX versions honour `stty erase`. Where the `stty` settings are not propagated correctly, things can be made to work by using `stty` manually.
- Some systems (including previous Debian versions) use `xmodmap` to arrange for both **<--** and **Delete** to generate `KB_Delete`. We can change the behavior of their X clients using the same X resources that we use to do it for our own clients, or configure our clients using their resources when things are the other way around. On displays configured like this **Delete** will not work, but **<--** will.
- Some operating systems have different `kdch1` settings in their `terminfo` database for `xterm` and others. On these systems the **Delete** key will not work correctly when you log in from a system conforming to our policy, but **<--** will.

9.9 Environment variables

Programs installed on the system PATH (`/bin`, `/usr/bin`, `/sbin`, `/usr/sbin`, or similar directories) must not depend on custom environment variable settings to get reasonable defaults. This is because such environment variables would have to be set in a system-wide configuration file such as a file in `/etc/profile.d`, which is not supported by all shells.

If a program usually depends on environment variables for its configuration, the program should be changed to fall back to a reasonable default configuration if these environment variables are not present. If this cannot be done easily (e.g., if the source code of a non-free program is not available), the program must be replaced by a small “wrapper” shell script that sets the environment variables if they are not already defined, and calls the original program.

Here is an example of a wrapper script for this purpose:

```
#!/bin/sh
BAR=${BAR:-/var/lib/fubar}
export BAR
exec /usr/lib/foo/foo "$@"
```

9.10 Registering Documents using doc-base

The doc-base package implements a mechanism for handling and presenting documentation. Debian packages that provides online documentation (other than just manual pages) may register these documents with doc-base by installing a doc-base control file in `/usr/share/doc-base/`.

Please refer to the documentation that comes with the doc-base package for information and details.

9.11 Alternate init systems

This section has been deleted.

9.11.1 Event-based boot with upstart

The `upstart` event-based boot system is no longer maintained in Debian, so this section has been removed.

9.12 Signaling that a reboot is required

Programs can signal that a reboot is required by touching `/run/reboot-required`. It is conventional to add the name of the package(s) requiring the reboot to `/run/reboot-required.pkgs`. Programs should not add a package name to `/run/reboot-required.pkgs` if it is already present there.

The `/run/reboot-required` mechanism is used when a reboot is needed to fully apply the changes introduced by package installation or upgrade. Typically it is the `postinst` maintainer script that touches `/run/reboot-required`, at the end of a successful configuration of the package.

There are no guarantees provided by the `/run/reboot-required` convention as to when or whether the requested reboot will occur.

10.1 Binaries

Two different packages must not install programs with different functionality but with the same filenames. This also applies when they are installed into different directories on the default (user or root) `PATH`. (The case of two programs having the same functionality but different implementations is handled via “alternatives” or the “Conflicts” mechanism. See *Maintainer Scripts* and *Conflicting binary packages - Conflicts* respectively.) If this case happens, one of the programs must be renamed. The maintainers should report this to the `debian-devel` mailing list and try to find a consensus about which program will have to be renamed. If a consensus cannot be reached, *both* programs must be renamed.

There is an exception to the above rules for `/usr/games`: packages that already install programs to `/usr/games`, where another package installs a program of the same name with different functionality to a different directory on the default `PATH`, may continue to do so. However, packages must not install any newly conflicting programs to `/usr/games`, and packages already doing so are encouraged to move these programs to non-conflicting names.

Packages must not install files to paths whose first component is a name directly under the file system root and which is a symbolic link to a directory of the same name under `/usr`. That is, packages must not install files to paths matching the glob patterns `/bin/*`, `/lib/*`, `/lib*/*` and `/sbin/*`. The `base-files` package is an exception, for it installs aliasing symbolic links from `/bin` to `/usr/bin`, `/lib` to `/usr/lib`, et cetera. This restriction is necessary to avoid triggering undefined behaviour in `dpkg`.

Packages may assume that `/bin`, `/lib` and `/sbin` are always symbolic links such that files under `/usr/bin`, `/usr/lib` and `/usr/sbin` are always accessible via these aliases.

Binary executables must not be statically linked with the GNU C library, since this prevents the binary from benefiting from fixes and improvements to the C library without being rebuilt and complicates security updates. This requirement may be relaxed for binary executables whose intended purpose is to diagnose and fix the system in situations where the GNU C library may not be usable (such as system recovery shells or utilities like `ldconfig`) or for binary executables where the security benefits of static linking outweigh the drawbacks.

By default, when a package is being built, any binaries created should include debugging information, as well as being compiled with optimization. You should also turn on as many reasonable compilation warnings as possible (see *Main building script: debian/rules*). For the C programming language, this means the following compilation parameters should be used:

```
CC = gcc
CFLAGS = -O2 -g -Wall # sane warning options vary between programs
LDFLAGS = # none
```

By default all installed binaries should be stripped by calling

```
strip --strip-unneeded --remove-section=.comment --remove-section=.note binaries
```

on the binaries after they have been copied into `debian/tmp` but before the tree is made into a package.

It is not recommended to strip binaries by passing the `-s` flag to `install`, because this fails to remove `.comment` and `.note` sections, and also prevents the automatic creation of `dbgsym` binary packages by tools like `dh_strip`.

Although binaries in the build tree should be compiled with debugging information by default, it can often be difficult to debug programs if they are also subjected to compiler optimization. For this reason, it is recommended to support the standardized environment variable `DEB_BUILD_OPTIONS` (see [debian/rules](#) and [DEB_BUILD_OPTIONS](#)). This variable can contain several flags to change how a package is compiled and built.

It is up to the package maintainer to decide what compilation options are best for the package. Certain binaries (such as computationally-intensive programs) will function better with certain flags (`-O3`, for example); feel free to use them. Please use good judgment here. Don't use flags for the sake of it; only use them if there is good reason to do so. Feel free to override the upstream author's ideas about which compilation options are best: they are often inappropriate for our environment.

10.2 Libraries

If the package is **architecture: any**, then the shared library compilation and linking flags must have `-fPIC`, or the package shall not build on some of the supported architectures.¹ Any exception to this rule must be discussed on the mailing list debian-devel@lists.debian.org, and a rough consensus obtained. The reasons for not compiling with `-fPIC` flag must be recorded in the file `README.Debian`, and care must be taken to either restrict the architecture or arrange for `-fPIC` to be used on architectures where it is required.²

As to the static libraries, the common case is not to have relocatable code, since there is no benefit, unless in specific cases; therefore the static version must not be compiled with the `-fPIC` flag. Any exception to this rule should be discussed on the mailing list debian-devel@lists.debian.org, and the reasons for compiling with the `-fPIC` flag must be recorded in the file `README.Debian`.³

In other words, if both a shared and a static library is being built, each source unit (`*.c`, for example, for C files) will need to be compiled twice, for the normal case.

Libraries should be built with threading support and to be thread-safe if the library supports this.

Although not enforced by the build tools, shared libraries must be linked against all libraries that they use symbols from in the same way that binaries are. This ensures the correct functioning of the [symbols](#) and [shlibs](#) systems and guarantees that all libraries can be safely opened with `dlopen()`. Packagers may wish to use the gcc option `-Wl,-z,defs` when building a shared library. Since this option enforces symbol resolution at build time, a missing library reference will be caught early as a fatal build error.

All installed shared libraries should be stripped with

```
strip --strip-unneeded --remove-section=.comment --remove-section=.note your-lib
```

(The option `--strip-unneeded` makes `strip` remove only the symbols which aren't needed for relocation processing.) Shared libraries can function perfectly well when stripped, since the symbols for dynamic linking are in a separate part of the ELF object file.⁴

¹ If you are using GCC, `-fPIC` produces code with relocatable position independent code, which is required for most architectures to create a shared library, with `i386` and perhaps some others where non position independent code is permitted in a shared library.

Position independent code may have a performance penalty, especially on `i386`. However, in most cases the speed penalty must be measured against the memory wasted on the few architectures where non position independent code is even possible.

² Some of the reasons why this might be required is if the library contains hand crafted assembly code that is not relocatable, the speed penalty is excessive for compute intensive libs, and similar reasons.

³ Some of the reasons for linking static libraries with the `-fPIC` flag are if, for example, one needs a Perl API for a library that is under rapid development, and has an unstable API, so shared libraries are pointless at this phase of the library's development. In that case, since Perl needs a library with relocatable code, it may make sense to create a static library with relocatable code. Another reason cited is if you are distilling various libraries into a common shared library, like `mklibs` does in the Debian installer project.

⁴ You might want to replace `--strip-unneeded` with `--strip-debug` for static libraries, as `dh_strip` does. When stripping static libraries, you should also pass `--enable-deterministic-archives` to ensure that your package build is reproducible.

Note that under some circumstances it may be useful to install a shared library unstripped, for example when building a separate package to support debugging. The debhelper `dh_strip` tool can create such packages automatically.

Shared object files (often `.so` files) that are not public libraries, that is, they are not meant to be linked to by third party executables (binaries of other packages), should be installed in subdirectories of the `/usr/lib` or `/usr/lib/triplet` directories (see the FHS for a definition). Such files are exempt from the rules that govern ordinary shared libraries, except that they must not be installed executable and should be stripped.⁵

Packages that use `libtool` to create and install their shared libraries install a file containing additional metadata (ending in `.la`) alongside the library. For public libraries intended for use by other packages, these files normally should not be included in the Debian package, since the information they include is not necessary to link with the shared library on Debian and can add unnecessary additional dependencies to other programs or libraries.⁶ If the `.la` file is required for that library (if, for instance, it's loaded via `libltdl` in a way that requires that meta-information), the `dependency_libs` setting in the `.la` file should normally be set to the empty string. If the shared library development package has historically included the `.la`, it must be retained in the development package (with `dependency_libs` emptied) until all libraries that depend on it have removed or emptied `dependency_libs` in their `.la` files to prevent linking with those other libraries using `libtool` from failing.

If the `.la` must be included, it should be included in the development (`-dev`) package, unless the library will be loaded by `libtool`'s `libltdl` library. If it is intended for use with `libltdl`, the `.la` files must go in the run-time library package.

These requirements for handling of `.la` files do not apply to loadable modules or libraries not installed in directories searched by default by the dynamic linker. Packages installing loadable modules will frequently need to install the `.la` files alongside the modules so that they can be loaded by `libltdl`. `dependency_libs` does not need to be modified for libraries or modules that are not installed in directories searched by the dynamic linker by default and not intended for use by other packages.

You must make sure that you use only released versions of shared libraries to build your packages; otherwise other users will not be able to run your binaries properly. Producing source packages that depend on unreleased compilers is also usually a bad idea.

10.3 Shared libraries

This section has moved to *Shared libraries*.

10.4 Scripts

All command scripts, including the package maintainer scripts inside the package and used by `dpkg`, should have a `#!` line naming the shell to be used to interpret them.

In the case of Perl scripts this should be `#!/usr/bin/perl`.

When scripts are installed into a directory in the system `PATH`, the script name should not include an extension such as `.sh` or `.pl` that denotes the scripting language currently used to implement it.

Shell scripts (`sh` and `bash`) other than `init.d` scripts should almost certainly start with `set -e` so that errors are detected. `init.d` scripts are something of a special case, due to how frequently they need to call commands that are allowed to fail, and it may instead be easier to check the exit status of commands directly. See *Writing the scripts* for more information about writing `init.d` scripts.

Every script should use `set -e` or check the exit status of *every* command.

⁵ A common example are the so-called “plug-ins”, internal shared objects that are dynamically loaded by programs using `dlopen(3)`.

⁶ These files store, among other things, all libraries on which that shared library depends. Unfortunately, if the `.la` file is present and contains that dependency information, using `libtool` when linking against that library will cause the resulting program or library to be linked against those dependencies as well, even if this is unnecessary. This can create unneeded dependencies on shared library packages that would otherwise be hidden behind the library ABI, and can make library transitions to new SONAMEs unnecessarily complicated and difficult to manage.

Scripts may assume that `/bin/sh` implements the POSIX.1-2017 Shell Command Language⁷ plus the following additional features not mandated by POSIX.1-2017..⁸

- `echo -n`, if implemented as a shell built-in, must not generate a newline.
- `test`, if implemented as a shell built-in, must support `-a` and `-o` as binary logical operators.
- `local` to create a scoped variable must be supported, including listing multiple variables in a single `local` command and assigning a value to a variable at the same time as localizing it. `local` may or may not preserve the variable value from an outer scope if no assignment is present. Uses such as:

```
fname () {
    local a b c=delta d
    # ... use a, b, c, d ...
}
```

must be supported and must set the value of `c` to `delta`.

- The XSI extension to `kill` allowing `kill -signal`, where `signal` is either the name of a signal or one of the numeric signals listed in the XSI extension (0, 1, 2, 3, 6, 9, 14, and 15), must be supported if `kill` is implemented as a shell built-in.
- The XSI extension to `trap` allowing numeric signals must be supported. In addition to the signal numbers listed in the extension, which are the same as for `kill` above, 13 (SIGPIPE) must be allowed.

If a shell script requires non-POSIX.1-2017 features from the shell interpreter other than those listed above, the appropriate shell must be specified in the first line of the script (e.g., `#!/bin/bash`) and the package must depend on the package providing the shell (unless the shell package is marked “Essential”, as in the case of `bash`).

You may wish to restrict your script to POSIX.1-2017 features plus the above set when possible so that it may use `/bin/sh` as its interpreter. Checking your script with `checkbashisms` from the `devscripts` package or running your script with an alternate shell such as `posh` may help uncover violations of the above requirements. If in doubt whether a script complies with these requirements, use `/bin/bash`.

Perl scripts should check for errors when making any system calls, including `open`, `print`, `close`, `rename` and `system`.

`csh` and `tcsh` should be avoided as scripting languages. See *Csh Programming Considered Harmful*, one of the `comp.unix.*` FAQs, which can be found at <http://www.faqs.org/faqs/unix-faq/shell/csh-why-not/>. If an upstream package comes with `csh` scripts then you must make sure that they start with `#!/bin/csh` and make your package depend on the `c-shell` virtual package.

Any scripts which create files in world-writable directories (e.g., in `/tmp`) must use a mechanism which will fail atomically if a file with the same name already exists.

The Debian base system provides the `tempfile` and `mktemp` utilities for use by scripts for this purpose.

10.5 Symbolic links

In general, symbolic links within a top-level directory should be relative, and symbolic links pointing from one top-level directory to or into another should be absolute. (A top-level directory is a sub-directory of the root directory `/`.) For example, a symbolic link from `/usr/lib/foo` to `/usr/share/bar` should be relative (`../share/bar`), but a symbolic link from `/var/run` to `/run` should be absolute.⁹ Symbolic links must not traverse above the root directory.

In addition, symbolic links should be specified as short as possible, i.e., link targets like `foo/./bar` are deprecated.

⁷ The Open Group Base Specifications Issue 7, 2018 Edition, which is also known as POSIX.1-2017 and as IEEE Std 1003.1-2017 and is available on the World Wide Web from [The Open Group](http://www.open-group.org/).

⁸ These features are in widespread use in the Linux community and are implemented in all of `bash`, `dash`, and `ksh`, the most common shells users may wish to use as `/bin/sh`.

⁹ This is necessary to allow top-level directories to be symlinks. If linking `/var/run` to `/run` were done with the relative symbolic link `../run`, but `/var` were a symbolic link to `/srv/disk1`, the symbolic link would point to `/srv/run` rather than the intended target.

Note that when creating a relative link using `ln` it is not necessary for the target of the link to exist relative to the working directory you're running `ln` from, nor is it necessary to change directory to the directory where the link is to be made. Simply include the string that should appear as the target of the link (this will be a pathname relative to the directory in which the link resides) as the first argument to `ln`.

For example, in your `Makefile` or `debian/rules`, you can do things like:

```
ln -fs gcc $(prefix)/bin/cc
ln -fs gcc debian/tmp/usr/bin/cc
ln -fs ../sbin/sendmail $(prefix)/bin/runq
ln -fs ../sbin/sendmail debian/tmp/usr/bin/runq
```

A symbolic link pointing to a compressed file (in the sense that it is meant to be uncompressed with `unzip` or `zless` etc.) should always have the same file extension as the referenced file. (For example, if a file `foo.gz` is referenced by a symbolic link, the filename of the link has to end with “.gz” too, as in `bar.gz`.)

10.6 Device files

Packages must not include device files or named pipes in the package file tree.

Debian packages should assume that device files in `/dev` are dynamically managed by the kernel or some other system facility and do not have to be explicitly created or managed by the package. Debian packages other than those whose purpose is to manage the `/dev` device file tree must not attempt to create or remove device files in `/dev` when a dynamic device management facility is in use.

If named pipes or device files outside of `/dev` are required by a package, they should normally be created when necessary by the programs in the package, by `init` scripts or `systemd` unit files, or by similar on-demand mechanisms. If such files need to be created during package installation, they must be created in the `postinst` maintainer script¹⁰ and removed in either the `prerm` or the `postrm` maintainer script.

10.7 Configuration files

10.7.1 Definitions

configuration file

A file that affects the operation of a program, or provides site- or host-specific information, or otherwise customizes the behavior of a program. Typically, configuration files are intended to be modified by the system administrator (if needed or desired) to conform to local policy or to provide more useful site-specific behavior.

conffile

A file listed in a package's `conffiles` file, and is treated specially by `dpkg` (see [Details of configuration](#)).

The distinction between these two is important; they are not interchangeable concepts. Almost all `conffiles` are configuration files, but many configuration files are not `conffiles`.

As noted elsewhere, `/etc/init.d` scripts, `/etc/default` files, scripts installed in `/etc/cron.{hourly,daily,weekly,monthly}`, and cron configuration installed in `/etc/cron.d` must be treated as configuration files. In general, any script that embeds configuration information is de-facto a configuration file and should be treated as such.

¹⁰ It's better to use `mkfifo` rather than `mknod` to create named pipes to avoid false positives from automated checks for packages incorrectly creating device files.

10.7.2 Location

Any configuration files created or used by your package must reside in `/etc`. If there are several, consider creating a subdirectory of `/etc` named after your package.

If your package creates or uses configuration files outside of `/etc`, and it is not feasible to modify the package to use `/etc` directly, put the files in `/etc` and create symbolic links to those files from the location that the package requires.

10.7.3 Behavior

Configuration file handling must conform to the following behavior:

- local changes must be preserved during a package upgrade, and
- configuration files must be preserved when the package is removed, and only deleted when the package is purged.

Obsolete configuration files without local changes should be removed by the package during upgrade.¹¹

The easy way to achieve this behavior is to make the configuration file a `conffile`. This is appropriate only if it is possible to distribute a default version that will work for most installations, although some system administrators may choose to modify it. This implies that the default version will be part of the package distribution, and must not be modified by the maintainer scripts during installation (or at any other time).

In order to ensure that local changes are preserved correctly, packages must not contain or make hard links to `conffiles`.¹²

The other way to do it is via the maintainer scripts. In this case, the configuration file must not be listed as a `conffile` and must not be part of the package distribution. If the existence of a file is required for the package to be sensibly configured it is the responsibility of the package maintainer to provide maintainer scripts which correctly create, update and maintain the file and remove it on purge. (See *Package maintainer scripts and installation procedure* for more information.) These scripts must be idempotent (i.e., must work correctly if `dpkg` needs to re-run them due to errors during installation or removal), must cope with all the variety of ways `dpkg` can call maintainer scripts, must not overwrite or otherwise mangle the user's configuration without asking, must not ask unnecessary questions (particularly during upgrades), and must otherwise be good citizens.

The scripts are not required to configure every possible option for the package, but only those necessary to get the package running on a given system. Ideally the sysadmin should not have to do any configuration other than that done (semi-)automatically by the `postinst` script.

A common practice is to create a script called `package-configure` and have the package's `postinst` call it if and only if the configuration file does not already exist. In certain cases it is useful for there to be an example or template file which the maintainer scripts use. Such files should be in `/usr/share/package` or `/usr/lib/package` (depending on whether they are architecture-independent or not). There should be symbolic links to them from `/usr/share/doc/package/examples` if they are examples, and should be perfectly ordinary `dpkg`-handled files (*not* configuration files).

These two styles of configuration file handling must not be mixed, for that way lies madness: `dpkg` will ask about overwriting the file every time the package is upgraded.

10.7.4 Sharing configuration files

If two or more packages use the same configuration file and it is reasonable for both to be installed at the same time, one of these packages must be defined as *owner* of the configuration file, i.e., it will be the package which handles that file as a configuration file. Other packages that use the configuration file must depend on the owning package if they require the configuration file to operate. If the other package will use the configuration file if present, but is capable of operating without it, no dependency need be declared.

If it is desirable for two or more related packages to share a configuration file *and* for all of the related packages to be able to modify that configuration file, then the following should be done:

¹¹ The `dpkg-maintscript-helper` tool, available from the `dpkg` package, can help for this task.

¹² Rationale: There are two problems with hard links. The first is that some editors break the link while editing one of the files, so that the two files may unwittingly become unlinked and different. The second is that `dpkg` might break the hard link while upgrading `conffiles`.

1. One of the related packages (the “owning” package) will manage the configuration file with maintainer scripts as described in the previous section.
2. The owning package should also provide a program that the other packages may use to modify the configuration file.
3. The related packages must use the provided program to make any desired modifications to the configuration file. They should either depend on the core package to guarantee that the configuration modifier program is available or accept gracefully that they cannot modify the configuration file if it is not. (This is in addition to the fact that the configuration file may not even be present in the latter scenario.)

Sometimes it's appropriate to create a new package which provides the basic infrastructure for the other packages and which manages the shared configuration files. (The `sgml-base` package is a good example.)

If the configuration file cannot be shared as described above, the packages must be marked as conflicting with each other. Two packages that specify the same file as a `conffile` must conflict. This is an instance of the general rule about not sharing files. Neither alternatives nor diversions are likely to be appropriate in this case; in particular, `dpkg` does not handle diverted `conffiles` well.

When two packages both declare the same `conffile`, they may see left-over configuration files from each other even though they conflict with each other. If a user removes (without purging) one of the packages and installs the other, the new package will take over the `conffile` from the old package. If the file was modified by the user, it will be treated the same as any other locally modified `conffile` during an upgrade.

The maintainer scripts must not alter a `conffile` of *any* package, including the one the scripts belong to.

10.7.5 User configuration files (“dotfiles”)

The files in `/etc/skel` will automatically be copied into new user accounts by `adduser`. No other program should reference the files in `/etc/skel`.

Therefore, if a program needs a dotfile to exist in advance in `$HOME` to work sensibly, that dotfile should be installed in `/etc/skel` and treated as a configuration file.

However, programs that require dotfiles in order to operate sensibly are a bad thing, unless they do create the dotfiles themselves automatically.

Furthermore, programs should be configured by the Debian default installation to behave as closely to the upstream default behavior as possible.

Therefore, if a program in a Debian package needs to be configured in some way in order to operate sensibly, that should be done using a site-wide configuration file placed in `/etc`. Only if the program doesn't support a site-wide default configuration and the package maintainer doesn't have time to add it may a default per-user file be placed in `/etc/skel`.

`/etc/skel` should be as empty as we can make it. This is particularly true because there is no easy (or necessarily desirable) mechanism for ensuring that the appropriate dotfiles are copied into the accounts of existing users when a package is installed.

10.8 Log files

Log files should usually be named `/var/log/package.log`. If you have many log files, or need a separate directory for permission reasons (`/var/log` is writable only by `root`), you should usually create a directory named `/var/log/package` and place your log files there.

Log files must be rotated occasionally so that they don't grow indefinitely. The best way to do this is to install a log rotation configuration file in the directory `/etc/logrotate.d`, normally named `/etc/logrotate.d/package`, and

use the facilities provided by `logrotate`.¹³ Here is a good example for a `logrotate` config file (for more information see `logrotate(8)`):

```
/var/log/foo/*.log {
    rotate 12
    weekly
    compress
    missingok
    postrotate
        start-stop-daemon -K -p /var/run/foo.pid -s HUP -x /usr/sbin/foo -q
    endscript
}
```

This rotates all files under `/var/log/foo`, saves 12 compressed generations, and tells the daemon to reopen its log files after the log rotation. It skips this log rotation (via `missingok`) if no such log file is present, which avoids errors if the package is removed but not purged.

Log files should be removed when the package is purged (but not when it is only removed). This should be done by the `postrm` script when it is called with the argument `purge` (see *Details of removal and/or configuration purging*).

10.9 Locale files

The directory `/usr/share/locale/` contains localization files for different locales.

Packages should not require the existence of any files in `/usr/share/locale/` in order to function in a C or C.UTF-8 locale.

10.10 Permissions and owners

The rules in this section are guidelines for general use. If necessary you may deviate from the details below. However, if you do so you must make sure that what is done is secure and you should try to be as consistent as possible with the rest of the system. You are also encouraged to discuss it on `debian-devel` first.

Files should be owned by `root:root`, and made writable only by the owner and universally readable (and executable, if appropriate), that is mode 644 or 755.

Directories should be mode 755 or (for group-writability) mode 2775. The ownership of the directory should be consistent with its mode: if a directory is mode 2775, it should be owned by the group that needs write access to it.¹⁴

Control information files should be owned by `root:root` and either mode 644 (for most files) or mode 755 (for executables such as *maintainer scripts*).

Setuid and setgid executables should be mode 4755 or 2755 respectively, and owned by the appropriate user or group. They should not be made unreadable (modes like 4711 or 2711 or even 4111); doing so achieves no extra security, because anyone can find the binary in the freely available Debian package; it is merely inconvenient. For the same reason you should not restrict read or execute permissions on non-set-id executables.

¹³ The traditional approach to log files has been to set up *ad hoc* log rotation schemes using simple shell scripts and cron. While this approach is highly customizable, it requires quite a lot of sysadmin work. Even though the original Debian system helped a little by automatically installing a system which can be used as a template, this was deemed not enough.

The use of `logrotate`, a program developed by Red Hat, is better, as it centralizes log management. It has both a configuration file (`/etc/logrotate.conf`) and a directory where packages can drop their individual log rotation configurations (`/etc/logrotate.d`).

¹⁴ When a package is upgraded, and the owner or permissions of a file included in the package has changed, `dpkg` arranges for the ownership and permissions to be correctly set upon installation. However, this does not extend to directories; the permissions and ownership of directories already on the system does not change on install or upgrade of packages. This makes sense, since otherwise common directories like `/usr` would always be in flux. To correctly change permissions of a directory the package owns, explicit action is required, usually in the `postinst` script. Care must be taken to handle downgrades as well, in that case.

Some `setuid` programs need to be restricted to particular sets of users, using file permissions. In this case they should be owned by the uid to which they are set-id, and by the group which should be allowed to execute them. They should have mode 4754; again there is no point in making them unreadable to those users who must not be allowed to execute them.

It is possible to arrange that the system administrator can reconfigure the package to correspond to their local security policy by changing the permissions on a binary: they can do this by using `dpkg-statoverride`, as described below.¹⁵ Another method you should consider is to create a group for people allowed to use the program(s) and make any `setuid` executables executable only by that group.

If you need to create a new user or group for your package there are two possibilities. Firstly, you may need to make some files in the binary package be owned by this user or group, or you may need to compile the user or group id (rather than just the name) into the binary (though this latter should be avoided if possible, as in this case you need a statically allocated id).

If you need a statically allocated id, you must ask for a user or group id from the `base-passwd` maintainer, and must not release the package until you have been allocated one. Once you have been allocated one you must either make the package depend on a version of the `base-passwd` package with the id present in `/etc/passwd` or `/etc/group`, or arrange for your package to create the user or group itself with the correct id (using `adduser`) in its `preinst` or `postinst`. (Doing it in the `postinst` is to be preferred if it is possible, otherwise a pre-dependency will be needed on the `adduser` package.)

On the other hand, the program might be able to determine the uid or gid from the user or group name at runtime, so that a dynamically allocated id can be used. In this case you should choose an appropriate user or group name, discussing this on `debian-devel` and checking that it is unique. When this has been checked you must arrange for your package to create the user or group if necessary using `adduser` in the `preinst` or `postinst` script (again, the latter is to be preferred if it is possible).

Note that changing the numeric value of an id associated with a name is very difficult, and involves searching the file system for all appropriate files. You need to think carefully whether a static or dynamic id is required, since changing your mind later will cause problems.

10.10.1 The use of `dpkg-statoverride`

This section is not intended as policy, but as a description of the use of `dpkg-statoverride`.

If a system administrator wishes to have a file (or directory or other such thing) installed with owner and permissions different from those in the distributed Debian package, they can use the `dpkg-statoverride` program to instruct `dpkg` to use the different settings every time the file is installed. Thus the package maintainer should distribute the files with their normal permissions, and leave it for the system administrator to make any desired changes. For example, a daemon which is normally required to be `setuid root`, but in certain situations could be used without being `setuid`, should be installed `setuid` in the `.deb`. Then the local system administrator can change this if they wish. If there are two standard ways of doing it, the package maintainer can use `debconf` to find out the preference, and call `dpkg-statoverride` in the maintainer script if necessary to accommodate the system administrator's choice. Care must be taken during upgrades to not override an existing setting.

Given the above, `dpkg-statoverride` is essentially a tool for system administrators and would not normally be needed in the maintainer scripts. There is one type of situation, though, where calls to `dpkg-statoverride` would be needed in the maintainer scripts, and that involves packages which use dynamically allocated user or group ids. In such a situation, something like the following idiom can be very helpful in the package's `postinst`, where `sysuser` is a dynamically allocated id:

```
for i in /usr/bin/foo /usr/sbin/bar; do
    # only do something when no setting exists
    if ! dpkg-statoverride --list $i >/dev/null 2>&1; then
        #include: debconf processing, question about foo and bar
```

(continues on next page)

¹⁵ Ordinary files installed by `dpkg` (as opposed to `conffiles` and other similar objects) normally have their permissions reset to the distributed permissions when the package is reinstalled. However, the use of `dpkg-statoverride` overrides this default behavior.

(continued from previous page)

```
        if [ "$RET" = "true" ] ; then
            dpkg-statoverride --update --add sysuser root 4755 $i
        fi
    fi
done
```

The corresponding code to remove the override when the package is purged would be:

```
for i in /usr/bin/foo /usr/sbin/bar; do
    if dpkg-statoverride --list $i >/dev/null 2>&1; then
        dpkg-statoverride --remove $i
    fi
done
```

10.11 File names

The name of the files installed by binary packages in the system PATH (namely /bin, /sbin, /usr/bin, /usr/sbin and /usr/games) must be encoded in ASCII.

The name of the files and directories installed by binary packages outside the system PATH must be encoded in UTF-8 and should be restricted to ASCII when it is possible to do so.

CUSTOMIZED PROGRAMS

11.1 Architecture specification strings

If a program needs to specify an *architecture specification string* in some place, it should select one of the strings provided by `dpkg-architecture -L`. The strings are in the format `os-arch`, though the OS part is sometimes elided, as when the OS is Linux.

Note that we don't want to use `arch-debian-linux` to apply to the rule `architecture-vendor-os` since this would make our programs incompatible with other Linux distributions. We also don't use something like `arch-unknown-linux`, since the `unknown` does not look very good.

11.1.1 Architecture wildcards

A package may specify an architecture wildcard. Architecture wildcards are in the format `any` (which matches every architecture), `os-any`, or `any-cpu`.¹

11.2 Daemons

The configuration files `/etc/services`, `/etc/protocols`, and `/etc/rpc` are managed by the `netbase` package and must not be modified by other packages.

If a package requires a new entry in one of these files, the maintainer should get in contact with the `netbase` maintainer, who will add the entries and release a new version of the `netbase` package.

The configuration file `/etc/inetd.conf` must not be modified by the package's scripts except via the `update-inetd` script or the `DebianNet.pm` Perl module. See their documentation for details on how to add entries.

If a package wants to install an example entry into `/etc/inetd.conf`, the entry must be preceded with exactly one hash character (`#`). Such lines are treated as "commented out by user" by the `update-inetd` script and are not changed or activated during package updates.

11.3 Using pseudo-ttys and modifying `wtmp`, `utmp` and `lastlog`

Some programs need to create pseudo-ttys. This should be done using Unix98 ptys if the C library supports it. The resulting program must not be installed setuid root, unless that is required for other functionality.

The files `/var/run/utmp`, `/var/log/wtmp` and `/var/log/lastlog` must be installed writable by group `utmp`. Programs which need to modify those files must be installed setgid `utmp`.

¹ Internally, the package system normalizes the GNU triplets and the Debian arches into Debian arch triplets (which are kind of inverted GNU triplets), with the first component of the triplet representing the libc and ABI in use, and then does matching against those triplets. However, such triplets are an internal implementation detail that should not be used by packages directly. The libc and ABI portion is handled internally by the package system based on the `os` and `cpu`.

11.4 Editors and pagers

Some programs have the ability to launch an editor or pager program to edit or display a text document. Since there are lots of different editors and pagers available in the Debian distribution, the system administrator and each user should have the possibility to choose their preferred editor and pager.

In addition, every program should choose a good default editor/pager if none is selected by the user or system administrator.

Thus, every program that launches an editor or pager must use the EDITOR or PAGER environment variable to determine the editor or pager the user wishes to use. If these variables are not set, the programs `/usr/bin/editor` and `/usr/bin/pager` should be used, respectively. These commands may be invoked explicitly (e.g., as `/usr/bin/editor`) or via a PATH search (e.g., as `editor`).

These two files are managed through the `dpkg` “alternatives” mechanism. Every package providing an editor or pager must call the `update-alternatives` script to register as an alternative for `/usr/bin/editor` or `/usr/bin/pager` as appropriate. The alternative should have a slave alternative for `/usr/share/man/man1/editor.1.gz` or `/usr/share/man/man1/pager.1.gz` pointing to the corresponding manual page.

If it is very hard to adapt a program to make use of the EDITOR or PAGER variables, that program may be configured to use `/usr/bin/sensible-editor` and `/usr/bin/sensible-pager` as the editor or pager program respectively. These are two scripts provided in the `sensible-utils` package that check the EDITOR and PAGER variables and launch the appropriate program, and fall back to `/usr/bin/editor` and `/usr/bin/pager` if the variable is not set.

A program may also use the VISUAL environment variable to determine the user’s choice of editor. If it exists, it should take precedence over EDITOR. This is in fact what `/usr/bin/sensible-editor` does.

It is not required for a package to depend on `editor` and `pager`, nor is it required for a package to provide such virtual packages.²

11.5 Web servers and applications

This section describes the locations and URLs that should be used by all web servers and web applications in the Debian system.

1. Cgi-bin executable files are installed in the directory

```
/usr/lib/cgi-bin
```

or a subdirectory of that directory, and the script

```
/usr/lib/cgi-bin/.../cgi-bin-name
```

should be referred to as

```
http://localhost/cgi-bin/.../cgi-bin-name
```

2. (Deleted)
3. Access to images

Images for a package should be stored in `/usr/share/images/package` and referred to through an alias `/images/` as:

```
http://localhost/images/package/filename
```

² The Debian base system already provides an editor and a pager program.

4. Web Document Root

Web Applications should try to avoid storing files in the Web Document Root. Instead they should use the `/usr/share/doc/package` directory for documents. If access to the web document root is unavoidable then use

```
/var/www/html
```

as the Document Root. This might be just a symbolic link to the location where the system administrator has put the real document root.

5. Providing httpd and/or httpd-cgi

All web servers should provide the virtual package `httpd`. If a web server has CGI support it should provide `httpd-cgi` additionally.

All web applications which do not contain CGI scripts should depend on `httpd`, all those web applications which do contain CGI scripts, should depend on `httpd-cgi`.

11.6 Mail transport, delivery and user agents

Debian packages which process electronic mail, whether mail user agents (MUAs) or mail transport agents (MTAs), must ensure that they are compatible with the configuration decisions below. Failure to do this may result in lost mail, broken `From:` lines, and other serious brain damage!

The mail spool is `/var/mail` and the interface to send a mail message is `/usr/sbin/sendmail` (as per the FHS). On older systems, the mail spool may be physically located in `/var/spool/mail`, but all access to the mail spool should be via the `/var/mail` symlink. The mail spool is part of the base system and not part of the MTA package.

All Debian MUAs, MTAs, MDAs and other mailbox accessing programs (such as IMAP daemons) must lock the mailbox in an NFS-safe way. This means that `fcntl()` locking must be combined with dot locking. To avoid deadlocks, a program should use `fcntl()` first and dot locking after this, or alternatively implement the two locking methods in a non blocking way.³ Using the functions `maillock` and `mailunlock` provided by the `liblockfile*` packages is the recommended way to accomplish this.

Mailboxes are generally either mode 600 and owned by user or mode 660 and owned by `user:mail`.⁴ The local system administrator may choose a different permission scheme; packages should not make assumptions about the permission and ownership of mailboxes unless required (such as when creating a new mailbox). A MUA may remove a mailbox (unless it has nonstandard permissions) in which case the MTA or another MUA must recreate it if needed.

The mail spool is 2775 `root:mail`, and MUAs should be setgid mail to do the locking mentioned above (and must obviously avoid accessing other users' mailboxes using this privilege).

`/etc/aliases` is the source file for the system mail aliases (e.g., `postmaster`, `usenet`, etc.), it is the one which the `sysadmin` and `postinst` scripts may edit. After `/etc/aliases` is edited the program or human editing it must call `newaliases`. All MTA packages must come with a `newaliases` program, even if it does nothing, but older MTA packages did not do this so programs should not fail if `newaliases` cannot be found. Note that because of this, all MTA packages must have `Provides`, `Conflicts` and `Replaces`: `mail-transport-agent` control fields.

The convention of writing `forward to address` in the mailbox itself is not supported. Use a `.forward` file instead.

The `rmail` program used by UUCP for incoming mail should be `/usr/sbin/rmail`. Likewise, `rsmtpt`, for receiving batch-SMTP-over-UUCP, should be `/usr/sbin/rsmtpt` if it is supported.

³ If it is not possible to establish both locks, the system shouldn't wait for the second lock to be established, but remove the first lock, wait a (random) time, and start over locking again.

⁴ There are two traditional permission schemes for mail spools: mode 600 with all mail delivery done by processes running as the destination user, or mode 660 and owned by group mail with mail delivery done by a process running as a system user in group mail. Historically, Debian required mode 660 mail spools to enable the latter model, but that model has become increasingly uncommon and the principle of least privilege indicates that mail systems that use the first model should use permissions of 600. If delivery to programs is permitted, it's easier to keep the mail system secure if the delivery agent runs as the destination user. Debian Policy therefore permits either scheme.

If your package needs to know what hostname to use on (for example) outgoing news and mail messages which are generated locally, you should use the file `/etc/mailname`. It will contain the portion after the username and @ (at) sign for email addresses of users on the machine (followed by a newline).

Such a package should check for the existence of this file when it is being configured. If it exists, it should be used without comment, although an MTA's configuration script may wish to prompt the user even if it finds that this file exists. If the file does not exist, the package should prompt the user for the value (preferably using `debconf`) and store it in `/etc/mailname` as well as using it in the package's configuration. The prompt should make it clear that the name will not just be used by that package. For example, in this situation the `inn` package could say something like:

```
Please enter the "mail name" of your system. This is the hostname portion
of the address to be shown on outgoing news and mail messages. The
default is syshostname, your system's host name.
```

```
Mail name ["syshostname"]:
```

where `syshostname` is the output of `hostname --fqdn`.

11.7 News system configuration

All the configuration files related to the NNTP (news) servers and clients should be located under `/etc/news`.

There are some configuration issues that apply to a number of news clients and server packages on the machine. These are:

`/etc/news/organization`

A string which should appear as the organization header for all messages posted by NNTP clients on the machine

`/etc/news/server`

Contains the FQDN of the upstream NNTP server, or localhost if the local machine is an NNTP server.

Other global files may be added as required for cross-package news configuration.

11.8 Programs for the X Window System

11.8.1 Providing X support and package priorities

Programs that can be configured with support for the X Window System must be configured to do so and must declare any package dependencies necessary to satisfy their runtime requirements when using the X Window System. If such a package is of higher priority than the X packages on which it depends, it is required that either the X-specific components be split into a separate package, or that an alternative version of the package, which includes X support, be provided, or that the package's priority be lowered.

11.8.2 Packages providing an X server

Packages that provide an X server that, directly or indirectly, communicates with real input and display hardware should declare in their `Provides` control field that they provide the virtual package `xserver`.⁵

⁵ This implements current practice, and provides an actual policy for usage of the `xserver` virtual package which appears in the virtual packages list. In a nutshell, X servers that interface directly with the display and input hardware or via another subsystem (e.g., GGI) should provide `xserver`. Things like `Xvfb`, `Xnest`, and `Xprt` should not.

11.8.3 Packages providing a terminal emulator

Packages that provide a terminal emulator for the X Window System which meet the criteria listed below should declare in their `Provides` control field that they provide the virtual package `x-terminal-emulator`. They should also register themselves as an alternative for `/usr/bin/x-terminal-emulator`, with a priority of 20. That alternative should have a slave alternative for `/usr/share/man/man1/x-terminal-emulator.1.gz` pointing to the corresponding manual page.

To be an `x-terminal-emulator`, a program must:

- Be able to emulate a DEC VT100 terminal, or a compatible terminal.
- Support the command-line option `-e command`, which creates a new terminal window⁶ and runs the specified command. `<command>` may be multiple arguments, which form the argument list to the executed program. In other words, the behavior is as though the arguments were passed directly to `execvp`, bypassing the shell. (`xterm`'s behavior of falling back on using the shell if `-e` had a single argument and `exec` failed is permissible but not required.)
- Support the command-line option `-T title`, which creates a new terminal window with the window title `title`.

11.8.4 Packages providing a window manager

Packages that provide a window manager should declare in their `Provides` control field that they provide the virtual package `x-window-manager`. They should also register themselves as an alternative for `/usr/bin/x-window-manager`, with a priority calculated as follows:

- Start with a priority of 40.
- If the window manager complies with [The Window Manager Specification Project](#), written by the [Free Desktop Group](#), add 40 points.
- If the window manager permits the X session to be restarted using a *different* window manager (without killing the X server) in its default configuration, add 10 points; otherwise add none.

That alternative should have a slave alternative for `/usr/share/man/man1/x-window-manager.1.gz` pointing to the corresponding manual page.

11.8.5 Packages providing fonts

Packages that provide fonts for the X Window System⁷ must do a number of things to ensure that they are both available without modification of the X or font server configuration, and that they do not corrupt files used by other font packages to register information about themselves.

1. Fonts of any type supported by the X Window System must be in a separate binary package from any executables, libraries, or documentation (except that specific to the fonts shipped, such as their license information). If one or more of the fonts so packaged are necessary for proper operation of the package with which they are associated the font package may be Recommended; if the fonts merely provide an enhancement, a Suggests relationship may be used. Packages must not Depend on font packages.⁸
2. BDF fonts must be converted to PCF fonts with the `bdftopcf` utility (available in the `xfonts-utils` package, gzipped, and placed in a directory that corresponds to their resolution:
 - 100 dpi fonts must be placed in `/usr/share/fonts/X11/100dpi/`.

⁶ “New terminal window” does not necessarily mean a new top-level X window directly parented by the window manager; it could, if the terminal emulator application were so coded, be a new “view” in a multiple-document interface (MDI).

⁷ For the purposes of Debian Policy, a “font for the X Window System” is one which is accessed via X protocol requests. Fonts for the Linux console, for PostScript renderer, or any other purpose, do not fit this definition. Any tool which makes such fonts available to the X Window System, however, must abide by this font policy.

⁸ This is because an X client may be displayed by a remote X server, in which case X fonts are provided by the remote X server, not retrieved locally; the Debian package system is empowered to deal only with the local file system.

- 75 dpi fonts must be placed in `/usr/share/fonts/X11/75dpi/`.
 - Character-cell fonts, cursor fonts, and other low-resolution fonts must be placed in `/usr/share/fonts/X11/misc/`.
3. Type 1 fonts must be placed in `/usr/share/fonts/X11/Type1/`. If font metric files are available, they must be placed here as well.
 4. Subdirectories of `/usr/share/fonts/X11/` other than those listed above must be neither created nor used. (The `PEX`, `CID`, `Speedo`, and `cyrillic` directories are excepted for historical reasons, but installation of files into these directories remains discouraged.)
 5. Font packages may, instead of placing files directly in the X font directories listed above, provide symbolic links in that font directory pointing to the files' actual location in the filesystem. Such a location must comply with the FHS.
 6. Font packages should not contain both 75dpi and 100dpi versions of a font. If both are available, they should be provided in separate binary packages with `-75dpi` or `-100dpi` appended to the names of the packages containing the corresponding fonts.
 7. Fonts destined for the `misc` subdirectory should not be included in the same package as 75dpi or 100dpi fonts; instead, they should be provided in a separate package with `-misc` appended to its name.
 8. Font packages must not provide the files `fonts.dir`, `fonts.alias`, or `fonts.scale` in a font directory:
 - `fonts.dir` files must not be provided at all.
 - `fonts.alias` and `fonts.scale` files, if needed, should be provided in the directory `/etc/X11/fonts/fontdir/package.extension`, where `fontdir` is the name of the subdirectory of `/usr/share/fonts/X11/` where the package's corresponding fonts are stored (e.g., `75dpi` or `misc`), `package` is the name of the package that provides these fonts, and `extension` is either `scale` or `alias`, whichever corresponds to the file contents.
 9. Font packages must declare a dependency on `xfonts-utils` in their `Depends` or `Pre-Depends` control field.
 10. Font packages that provide one or more `fonts.scale` files as described above must invoke `update-fonts-scale` on each directory into which they installed fonts *before* invoking `update-fonts-dir` on that directory. This invocation must occur in both the `postinst` (for all arguments) and `postrm` (for all arguments except `upgrade`) scripts.
 11. Font packages that provide one or more `fonts.alias` files as described above must invoke `update-fonts-alias` on each directory into which they installed fonts. This invocation must occur in both the `postinst` (for all arguments) and `postrm` (for all arguments except `upgrade`) scripts.
 12. Font packages must invoke `update-fonts-dir` on each directory into which they installed fonts. This invocation must occur in both the `postinst` (for all arguments) and `postrm` (for all arguments except `upgrade`) scripts.
 13. Font packages must not provide alias names for the fonts they include which collide with alias names already in use by fonts already packaged.
 14. Font packages must not provide fonts with the same XLFD registry name as another font already packaged.

11.8.6 Application defaults files

Application defaults files must be installed in the directory `/etc/X11/app-defaults/` (use of a localized subdirectory of `/etc/X11/` as described in the *X Toolkit Intrinsics - C Language Interface* manual is also permitted). They must be registered as `conffiles` or handled as configuration files.

Customization of programs' X resources may also be supported with the provision of a file with the same name as that of the package placed in the `/etc/X11/Xresources/` directory, which must be registered as a `conffile` or handled as a configuration file.⁹

⁹ Note that this mechanism is not the same as using `app-defaults`; `app-defaults` are tied to the client binary on the local file system, whereas X

11.8.7 Installation directory issues

Historically, packages using the X Window System used a separate set of installation directories from other packages. This practice has been discontinued and packages using the X Window System should now generally be installed in the same directories as any other package. Specifically, packages must not install files under the `/usr/X11R6/` directory and the `/usr/X11R6/` directory hierarchy should be regarded as obsolete.

Include files previously installed under `/usr/X11R6/include/X11/` should be installed into `/usr/include/X11/`. For files previously installed into subdirectories of `/usr/X11R6/lib/X11/`, package maintainers should determine if subdirectories of `/usr/lib/` and `/usr/share/` can be used. If not, a subdirectory of `/usr/lib/X11/` should be used.

Configuration files for window, display, or session managers or other applications that are tightly integrated with the X Window System may be placed in a subdirectory of `/etc/X11/` corresponding to the package name. Other X Window System applications should use the `/etc/` directory unless otherwise mandated by policy (such as for *Application defaults files*).

11.9 Perl programs and modules

Perl programs and modules should follow the current Perl policy.

The Perl policy can be found in the `perl-policy` files in the `debian-policy` package. It is also available from the Debian web mirrors at <https://www.debian.org/doc/packaging-manuals/perl-policy/>.

11.10 Emacs lisp programs

Please refer to the “Debian Emacs Policy” for details of how to package emacs lisp programs.

The Emacs policy is available in `debian-emacs-policy.gz` of the `emacs-common` package. It is also available from the Debian web mirrors at <https://www.debian.org/doc/packaging-manuals/debian-emacs-policy>.

11.11 Games

The permissions on `/var/games` are mode `755`, owner `root` and group `root`.

Each game decides on its own security policy.

Games which require protected, privileged access to high-score files, saved games, etc., may be made `set-group-id` (mode `2755`) and owned by `root:games`, and use files and directories with appropriate permissions (`770 root:games`, for example). They must not be made `set-user-id`, as this causes security problems. (If an attacker can subvert any `set-user-id` game they can overwrite the executable of any other, causing other players of these games to run a Trojan horse program. With a `set-group-id` game the attacker only gets access to less important game data, and if they can get at the other players' accounts at all it will take considerably more effort.)

Some packages, for example some fortune cookie programs, are configured by the upstream authors to install with their data files or other static information made unreadable so that they can only be accessed through `set-id` programs provided. You should not do this in a Debian package: anyone can download the `.deb` file and read the data from it, so there is no point making the files unreadable. Not making the files unreadable also means that you don't have to make so many programs `set-id`, which reduces the risk of a security hole.

As described in the FHS, binaries of games should be installed in the directory `/usr/games`. This also applies to games that use the X Window System. Manual pages for games (X and non-X games) should be installed in `/usr/share/man/man6`.

resources are stored in the X server and affect all connecting clients.

DOCUMENTATION

12.1 Manual pages

You should install manual pages in `nroff` source form, in appropriate places under `/usr/share/man`. You should only use sections 1 to 9 (see the FHS for more details). You must not install a pre-formatted “cat page”.

Each program, utility, and function should have an associated manual page included in the same package or a dependency. It is suggested that all configuration files also have a manual page included as well. Manual pages for protocols and other auxiliary things are optional.

If no manual page is available, this is considered as a bug and should be reported to the Debian Bug Tracking System (the maintainer of the package is allowed to write this bug report themselves, if they so desire). Do not close the bug report until a proper man page is available.¹

You may forward a complaint about a missing man page to the upstream authors, and mark the bug as forwarded in the Debian bug tracking system. Even though the GNU Project do not in general consider the lack of a man page to be a bug, we do; if they tell you that they don't consider it a bug you should leave the bug in our bug tracking system open anyway.

Manual pages should be installed compressed using `gzip -9`.

If one man page needs to be accessible via several names it is better to use a symbolic link than the `.so` feature, but there is no need to fiddle with the relevant parts of the upstream source to change from `.so` to symlinks: don't do it unless it's easy. You should not create hard links in the manual page directories, nor put absolute filenames in `.so` directives. The filename in a `.so` in a man page should be relative to the base of the man page tree (usually `/usr/share/man`). If you do not create any links (whether symlinks, hard links, or `.so` directives) in the file system to the alternate names of the man page, then you should not rely on `man` finding your man page under those names based solely on the information in the man page's header.²

Manual pages in locale-specific subdirectories of `/usr/share/man` should use either UTF-8 or the usual legacy encoding for that language (normally the one corresponding to the shortest relevant locale name in `/usr/share/i18n/SUPPORTED`). For example, pages under `/usr/share/man/fr` should use either UTF-8 or ISO-8859-1.³

A country name (the `DE` in `de_DE`) should not be included in the subdirectory name unless it indicates a significant difference in the language, as this excludes speakers of the language in other countries.⁴

If a localized version of a manual page is provided, it should either be up-to-date or it should be obvious to the reader that it is outdated and the original manual page should be used instead. This can be done either by a note at the beginning of the manual page or by showing the missing or changed portions in the original language instead of the target language.

¹ It is not very hard to write a man page. See the [Man-Page-HOWTO](#), `man(7)`, the examples created by `dh_make`, the helper program `help2man`, or the directory `/usr/share/doc/man-db/examples`.

² Supporting this in `man` often requires unreasonable processing time to find a manual page or to report that none exists, and moves knowledge into `man`'s database that would be better left in the file system. This support is therefore deprecated and will cease to be present in the future.

³ `man` will automatically detect whether UTF-8 is in use. In future, all manual pages will be required to use UTF-8.

⁴ At the time of writing, Chinese and Portuguese are the main languages with such differences, so `pt_BR`, `zh_CN`, and `zh_TW` are all allowed.

Packages (other than manual page readers) must not require the existence of any files in `/usr/share/man/` in order to function. A package that optionally displays its own manual pages, such as to provide user help, must fail gracefully if the manual pages aren't installed.

12.2 Info documents

Info documents should be installed in `/usr/share/info`. They should be compressed with `gzip -9`.

The `install-info` program maintains a directory of installed info documents in `/usr/share/info/dir` for the use of info readers. This file must not be included in packages other than `install-info`.

`install-info` is automatically invoked when appropriate using `dpkg` triggers. Packages other than `install-info` *should not* invoke `install-info` directly and *should not* depend on, recommend, or suggest `install-info` for this purpose.

Info readers requiring the `/usr/share/info/dir` file should depend on `install-info`.

Info documents should contain section and directory entry information in the document for the use of `install-info`. The section should be specified via a line starting with `INFO-DIR-SECTION` followed by a space and the section of this info page. The directory entry or entries should be included between a `START-INFO-DIR-ENTRY` line and an `END-INFO-DIR-ENTRY` line. For example:

```
INFO-DIR-SECTION Individual utilities
START-INFO-DIR-ENTRY
* example: (example).           An example info directory entry.
END-INFO-DIR-ENTRY
```

To determine which section to use, you should look at `/usr/share/info/dir` on your system and choose the most relevant (or create a new section if none of the current sections are relevant).⁵

Packages (other than info readers) must not require the existence of any files in `/usr/share/info/` in order to function.

12.3 Additional documentation

Any additional documentation that comes with the package may be installed at the discretion of the package maintainer. It is often a good idea to include text information files (`READMEs`, `FAQs`, and so forth) that come with the source package in the binary package. However, you don't need to install the instructions for building and installing the package, of course!

Plain text documentation should be compressed with `gzip -9` unless it is small.

If a package comes with large amounts of documentation that many users of the package will not require, you should create a separate binary package to contain it so that it does not take up disk space on the machines of users who do not need or want it installed. As a special case of this rule, shared library documentation of any appreciable size should always be packaged with the library development package (*Development files*) or in a separate documentation package, since shared libraries are frequently installed as dependencies of other packages by users who have little interest in documentation of the library itself. The documentation package for the package package is conventionally named `package-doc` (or `package-doc-language-code` if there are separate documentation packages for multiple languages).

If package is a build tool, development tool, command-line tool, or library development package, package (or `package-dev` in the case of a library development package) already provides documentation in `man`, `info`, or plain text format, and

⁵ Normally, info documents are generated from Texinfo source. To include this information in the generated info document, if it is absent, add commands like:

```
@dircategory Individual utilities
@direntry
* example: (example).  An example info directory entry.
@end direntry
```

to the Texinfo source of the document and ensure that the info documents are rebuilt from source during the package build.

package-doc provides HTML or other formats, package should declare at most a `Suggests` on package-doc. Otherwise, package should declare at most a `Recommends` on package-doc.

Additional documentation included in the package should be installed under `/usr/share/doc/package`. If the documentation is packaged separately, as package-doc for example, it may be installed under either that path or into the documentation directory for the separate documentation package (`/usr/share/doc/package-doc` in this example). However, installing the documentation into the documentation directory of the main package is encouraged since it is independent of the packaging method and will be easier for users to find.

Any separate package providing documentation must still install standard documentation files in its own `/usr/share/doc` directory as specified in the rest of this policy. See, for example, *Copyright information* and *Changelog files and release notes*.

Packages must not require the existence of any files in `/usr/share/doc/` in order to function.⁶ Any files that are used or read by programs but are also useful as stand alone documentation should be installed elsewhere, such as under `/usr/share/package/`, and then included via symbolic links in `/usr/share/doc/package`.

`/usr/share/doc/package` is permitted to be a symbolic link to another directory in `/usr/share/doc` only if the two packages both come from the same source and the first package `Depends` on the second. Otherwise, `/usr/share/doc/package` must not be a symbolic link.⁷

12.4 Preferred documentation formats

The unification of Debian documentation is being carried out via HTML.

If the package comes with extensive documentation in a markup format that can be converted to various other formats you should if possible ship HTML versions in a binary package.⁸ The documentation must be installed as specified in *Additional documentation*.

Other formats such as PostScript may be provided at the package maintainer's discretion.

12.5 Copyright information

Every package must be accompanied by a verbatim copy of its distribution license(s) in the file `/usr/share/doc/PACKAGE/copyright`. This file must neither be compressed nor be a symbolic link.

A verbatim copy of the package's copyright information is often required to be present in `/usr/share/doc/PACKAGE/copyright`, too; see *Copyright considerations*.

In addition, the copyright file must say where the upstream sources (if any) were obtained, and should include a name or contact address for the upstream authors. This can be the name of an individual or an organization, an email address, a web forum or bugtracker, or any other means to unambiguously identify who to contact to participate in the development of the upstream source code.

Packages in the *contrib* or *non-free* archive areas should state in the copyright file that the package is not part of the Debian distribution and briefly explain why.

A copy of the file which will be installed in `/usr/share/doc/PACKAGE/copyright` should be in `debian/copyright` in the source package.

`/usr/share/doc/package` is permitted to be a symbolic link to another directory in `/usr/share/doc` only if the two packages both come from the same source and the first package `Depends` on the second. Otherwise, `/usr/share/`

⁶ The system administrator should be able to delete files in `/usr/share/doc/` without causing any programs to break.

⁷ Please note that this does not override the section on changelog files below, so the file `/usr/share/doc/package/changelog.Debian.gz` must refer to the changelog for the current version of package in question. In practice, this means that the sources of the target and the destination of the symlink must be the same (same source package and version).

⁸ Rationale: The important thing here is that HTML documentation should be available from *some* binary package.

`doc/package` must not be a symbolic link. These rules are important because `copyright` files must be extractable by mechanical means.

Packages distributed under the Apache license (version 2.0), the Artistic license, the Creative Commons CC0-1.0 license, the GNU GPL (versions 1, 2, or 3), the GNU LGPL (versions 2, 2.1, or 3), the GNU FDL (versions 1.2 or 1.3), and the Mozilla Public License (version 1.1 or 2.0) should refer to the corresponding files under `/usr/share/common-licenses`,⁹ rather than quoting them in the `copyright` file.

You should not use the `copyright` file as a general `README` file. If your package has such a file it should be installed in `/usr/share/doc/package/README` or `README.Debian` or some other appropriate place.

All `copyright` files must be encoded in UTF-8.

12.5.1 Machine-readable copyright information

A specification for a standard, machine-readable format for `debian/copyright` files is maintained as part of the `debian-policy` package. This document is in the `copyright-format` files in the `debian-policy` package. It is also available from the Debian web mirrors at <https://www.debian.org/doc/packaging-manuals/copyright-format/1.0/>.

Use of this format is optional.

12.6 Examples

Any examples (configurations, source files, whatever), should be installed in a directory `/usr/share/doc/package/examples`. These files should not be referenced by any program: they're there for the benefit of the system administrator and users as documentation only. Architecture-specific example files should be installed in a directory `/usr/lib/package/examples` with symbolic links to them from `/usr/share/doc/package/examples`, or the latter directory itself may be a symbolic link to the former.

If the purpose of a package is to provide examples, then the example files may be installed into `/usr/share/doc/package`.

12.7 Changelog files and release notes

Packages that are not Debian-native must contain a compressed copy of the `debian/changelog` file from the Debian source tree in `/usr/share/doc/package` with the name `changelog.Debian.gz`.

If an upstream release notes file is available, containing a summary of changes between upstream releases intended for end users of the package and often called `NEWS`, it should be accessible as `/usr/share/doc/package/NEWS.gz`. An older practice of installing the upstream release notes as `/usr/share/doc/package/changelog.gz` is permitted but deprecated.

If there is an upstream `changelog` available, it may be made available as `/usr/share/doc/package/changelog.gz`.

If either of these files are distributed in HTML, they should be made available at `/usr/share/doc/package/NEWS.html.gz` and `/usr/share/doc/package/changelog.html.gz` respectively, and plain text versions `NEWS.gz` and `changelog.gz` should be generated from them, using, for example, `lynx -dump -nolist`.

⁹ In particular, `/usr/share/common-licenses/Apache-2.0`, `/usr/share/common-licenses/Artistic`, `/usr/share/common-licenses/CC0-1.0`, `/usr/share/common-licenses/GPL-1`, `/usr/share/common-licenses/GPL-2`, `/usr/share/common-licenses/GPL-3`, `/usr/share/common-licenses/LGPL-2`, `/usr/share/common-licenses/LGPL-2.1`, `/usr/share/common-licenses/LGPL-3`, `/usr/share/common-licenses/GFDL-1.2`, `/usr/share/common-licenses/GFDL-1.3`, `/usr/share/common-licenses/MPL-1.1`, and `/usr/share/common-licenses/MPL-2.0` respectively. The University of California BSD license is also included in base-files as `/usr/share/common-licenses/BSD`, but given the brevity of this license, its specificity to code whose copyright is held by the Regents of the University of California, and the frequency of minor wording changes, its text should be included in the `copyright` file rather than referencing this file.

If the upstream release notes or changelog do not already conform to this naming convention, then this may be achieved either by renaming the files, or by adding a symbolic link, at the maintainer's discretion.¹⁰

All of these files should be installed compressed using `gzip -9`, as they will become large with time even if they start out small.

If the package has only one file which is used both as the Debian changelog and the upstream release notes or changelog, because there is no separate upstream maintainer, then that file should usually be installed as `/usr/share/doc/package/NEWS.gz` or `/usr/share/doc/package/changelog.gz` (depending on whether the file is release notes or a changelog); if there is a separate upstream maintainer, but no upstream release notes or changelog, then the Debian changelog should still be called `changelog.Debian.gz`.

For details about the format and contents of the Debian changelog file, please see *Debian changelog: [debian/changelog](#)*.

¹⁰ Rationale: People should not have to look in places for upstream changelogs merely because they are given different names or are distributed in HTML format.

INTRODUCTION AND SCOPE OF THESE APPENDICES

These appendices, except the final three, are taken essentially verbatim from the now-deprecated Packaging Manual, version 3.2.1.0. They are the chapters which are likely to be of use to package maintainers and which have not already been included in the policy document itself. Most of these sections are very likely not relevant to policy; they should be treated as documentation for the packaging system. Please note that these appendices are included for convenience, and for historical reasons: they used to be part of policy package, and they have not yet been incorporated into `dpkg` documentation. However, they still have value, and hence they are presented here.

They have not yet been checked to ensure that they are compatible with the contents of policy, and if there are any contradictions, the version in the main policy document takes precedence. The remaining chapters of the old Packaging Manual have also not been read in detail to ensure that there are not parts which have been left out. Both of these will be done in due course.

Certain parts of the Packaging manual were integrated into the Policy Manual proper, and removed from the appendices. Links have been placed from the old locations to the new ones.

`dpkg` is a suite of programs for creating binary package files and installing and removing them on Unix systems.¹

The binary packages are designed for the management of installed executable programs (usually compiled binaries) and their associated data, though source code examples and documentation are provided as part of some packages.

This manual describes the technical aspects of creating Debian binary packages (`.deb` files). It documents the behavior of the package management programs `dpkg`, `dselect` et al. and the way they interact with packages.

This manual does not go into detail about the options and usage of the package building and installation tools. It should therefore be read in conjunction with those programs' man pages.

The utility programs which are provided with `dpkg` not described in detail here, are documented in their man pages.

It is assumed that the reader is reasonably familiar with the `dpkg` System Administrators' manual. Unfortunately this manual does not yet exist.

The Debian version of the FSF's GNU hello program is provided as an example for people wishing to create Debian packages. However, while the examples are helpful, they do not replace the need to read and follow the Policy and Programmer's Manual.

¹ `dpkg` is targeted primarily at Debian, but may work on or be ported to other systems.

BINARY PACKAGES (FROM OLD PACKAGING MANUAL)

See *deb (5)* and *Binary package metadata files*.

14.1 Creating package files - `dpkg-deb`

All manipulation of binary package files is done by `dpkg-deb`; it's the only program that has knowledge of the format. (`dpkg-deb` may be invoked by calling `dpkg`, as `dpkg` will spot that the options requested are appropriate to `dpkg-deb` and invoke that instead with the same arguments.)

In order to create a binary package, you must make a directory tree which contains all the files and directories you want to have in the file system data part of the package. In Debian-format source packages, this directory is usually either `debian/tmp` or `debian/pkg`, relative to the top of the package's source tree.

They should have the locations (relative to the root of the directory tree you're constructing) ownerships and permissions which you want them to have on the system when they are installed.

With current versions of `dpkg` the uid/username and gid/groupname mappings for the users and groups being used should be the same on the system where the package is built and the one where it is installed.

You need to add one special directory to the root of the miniature file system tree you're creating: `DEBIAN`. It should contain the control information files, notably the binary package control file (see *The binary package control file: control*).

The `DEBIAN` directory will not appear in the file system archive of the package, and so won't be installed by `dpkg` when the package is unpacked.

When you've prepared the package, you should invoke:

```
dpkg --build directory
```

This will build the package in `directory.deb`. (`dpkg` knows that `--build` is a `dpkg-deb` option, so it invokes `dpkg-deb` with the same arguments to build the package.)

See the *dpkg-deb (8)* man page for details of how to examine the contents of this newly-created file. You may find the output of following commands enlightening:

```
dpkg-deb --info filename.deb
dpkg-deb --contents filename.deb
dpkg --contents filename.deb
```

To view the copyright file for a package you could use this command:

```
dpkg --fsys-tarfile filename.deb | tar xOf - --wildcards */copyright | pager
```

14.2 Binary package metadata files

The package metadata portion of a binary package is a collection of files with names known to `dpkg`. It will treat the contents of these files specially - some of them contain information used by `dpkg` when installing or removing the package; others are scripts which the package maintainer wants `dpkg` to run.

It is possible to put other files in the package metadata archive member, but this is not generally a good idea (though they will largely be ignored).

Here is a brief list of the package metadata files supported by `dpkg` and a summary of what they're used for.

control

This is the key description file used by `dpkg`. It specifies the package's name and version, gives its description for the user, states its relationships with other packages, and so forth. See *Debian source package template control files – debian/control* and *Debian binary package control files – DEBIAN/control*.

It is usually generated automatically from information in the source package by the `dpkg-gencontrol` program, and with assistance from `dpkg-shlibdeps`. See *Source packages (from old Packaging Manual)*.

postinst, preinst, postrm, prerm

These are executable files (usually scripts) which `dpkg` runs during installation, upgrade and removal of packages. They allow the package to deal with matters which are particular to that package or require more complicated processing than that provided by `dpkg`. Details of when and how they are called are in *Package maintainer scripts and installation procedure*.

It is very important to make these scripts idempotent. See *Maintainer scripts idempotency*.

The maintainer scripts are not guaranteed to run with a controlling terminal and may not be able to interact with the user. See *Controlling terminal for maintainer scripts*.

conffiles This file contains a list of configuration files which

are to be handled automatically by `dpkg` (see *Configuration file handling (from old Packaging Manual)*). Note that not necessarily every configuration file should be listed here.

shlibs

This file contains a list of the shared libraries supplied by the package, with dependency details for each. This is used by `dpkg-shlibdeps` when it determines what dependencies are required in a package control file. The `shlibs` file format is described on *The shlibs File Format*.

14.3 The binary package control file: `control`

The most important package metadata file used by `dpkg` when it installs a package is `control`. It contains all the package's "vital statistics".

The binary package control files of packages built from Debian sources are made by a special tool, `dpkg-gencontrol`, which reads `debian/control` and `debian/changelog` to find the information it needs. See *Source packages (from old Packaging Manual)* for more details.

The fields in binary package control files are listed in *Debian binary package control files – DEBIAN/control*.

A description of the syntax of control files and the purpose of the fields is available in *Control files and their fields*.

14.4 Time Stamps

See *Time Stamps*.

SOURCE PACKAGES (FROM OLD PACKAGING MANUAL)

The Debian binary packages in the distribution are generated from Debian sources, which are in a special format to assist the easy and automatic building of binaries.

15.1 Tools for processing source packages

Various tools are provided for manipulating source packages; they pack and unpack sources and help build of binary packages and help manage the distribution of new versions.

They are introduced and typical uses described here; see *dpkg-source(1)* for full documentation about their arguments and operation.

For examples of how to construct a Debian source package, and how to use those utilities that are used by Debian source packages, please see the `hello` example package.

15.1.1 `dpkg-source` - packs and unpacks Debian source packages

This program is frequently used by hand, and is also called from package-independent automated building scripts such as `dpkg-buildpackage`.

To unpack a package it is typically invoked with

```
dpkg-source -x .../path/to/filename.dsc
```

with the `filename.tar.gz` and `filename.diff.gz` (if applicable) in the same directory. It unpacks into `package-version`, and if applicable `package-version.orig`, in the current directory.

To create a packed source archive it is typically invoked:

```
dpkg-source -b package-version
```

This will create the `.dsc`, `.tar.gz` and `.diff.gz` (if appropriate) in the current directory. `dpkg-source` does not clean the source tree first - this must be done separately if it is required.

See also *Source packages as archives*.

15.1.2 `dpkg-buildpackage` - overall package-building control script

See *dpkg-buildpackage(1)*.

15.1.3 `dpkg-gencontrol` - generates binary package control files

This program is usually called from `debian/rules` (see *section_title*) in the top level of the source tree.

This is usually done just before the files and directories in the temporary directory tree where the package is being built have their permissions and ownerships set and the package is constructed using `dpkg-deb`.¹

`dpkg-gencontrol` must be called after all the files which are to go into the package have been placed in the temporary build directory, so that its calculation of the installed size of a package is correct.

It is also necessary for `dpkg-gencontrol` to be run after `dpkg-shlibdeps` so that the variable substitutions created by `dpkg-shlibdeps` in `debian/substvars` are available.

For a package which generates only one binary package, and which builds it in `debian/tmp` relative to the top of the source package, it is usually sufficient to call `dpkg-gencontrol`.

Sources which build several binaries will typically need something like:

```
dpkg-gencontrol -Pdebian/pkg -ppackage
```

The `-P` tells `dpkg-gencontrol` that the package is being built in a non-default directory, and the `-p` tells it which binary package's control file should be generated.

`dpkg-gencontrol` also adds information to the list of files in `debian/files`, for the benefit of (for example) a future invocation of `dpkg-genchanges`.

15.1.4 `dpkg-shlibdeps` - calculates shared library dependencies

See *dpkg-shlibdeps(1)*.

15.1.5 `dpkg-distaddfile` - adds a file to `debian/files`

Some packages' uploads need to include files other than the source and binary package files.

`dpkg-distaddfile` adds a file to the `debian/files` file so that it will be included in the `.changes` file when `dpkg-genchanges` is run.

It is usually invoked from the `binary` target of `debian/rules`:

```
dpkg-distaddfile filename section priority
```

The filename is relative to the directory where `dpkg-genchanges` will expect to find it - this is usually the directory above the top level of the source tree. The `debian/rules` target should put the file there just before or just after calling `dpkg-distaddfile`.

The section and priority are passed unchanged into the resulting `.changes` file.

15.1.6 `dpkg-genchanges` - generates a `.changes` upload control file

See *dpkg-genchanges(1)*.

15.1.7 `dpkg-parsechangelog` - produces parsed representation of a changelog

See *dpkg-parsechangelog(1)*.

¹ This is so that the control file which is produced has the right permissions

15.1.8 `dpkg-architecture` - information about the build and host system

See *`dpkg-architecture(1)`*.

15.2 The Debian package source tree

The source archive scheme described later is intended to allow a Debian package source tree with some associated package metadata to be reproduced and transported easily. The Debian package source tree is a version of the original program with certain files added for the benefit of the packaging process, and with any other changes required made to the rest of the source code and installation scripts.

The extra files created for Debian are in the subdirectory `debian` of the top level of the Debian package source tree. They are described below.

15.2.1 `debian/rules` - the main building script

See *Main building script: `debian/rules`*.

15.2.2 `debian/substvars` and variable substitutions

See *Variable substitutions: `debian/substvars`*.

15.2.3 `debian/files`

See *Generated files list: `debian/files`*.

15.2.4 `debian/tmp`

This is the default temporary location for the construction of binary packages by the `binary` target. The directory `tmp` serves as the root of the file system tree as it is being constructed (for example, by using the package's upstream makefiles install targets and redirecting the output there), and it also contains the `DEBIAN` subdirectory. See *Creating package files - `dpkg-deb`*.

This is only a default and can be easily overridden. Most packaging tools no longer use `debian/tmp`, instead preferring `debian/pkg` for the common case of a source package building only one binary package. Such tools usually only use `debian/tmp` as a temporary staging area for built files and do not construct packages from it.

If several binary packages are generated from the same source tree, it is usual to use a separate `debian/pkg` directory for each binary package as the temporary construction locations.

Whatever temporary directories are created and used by the `binary` target must of course be removed by the `clean` target.

15.3 Source packages as archives

As it exists on the FTP site, a Debian source package consists of three related files. You must have the right versions of all three to be able to use them.

Debian source control file - `.dsc`

This file is a control file used by `dpkg-source` to extract a source package. See *Debian source package control files - `.dsc`*.

Original source archive - `package_upstream-version.orig.tar.gz`

This is a compressed (with `gzip -9`) `tar` file containing the source code from the upstream authors of the program.

Debian package diff - `package_upstream_version-revision.diff.gz`

This is a unified context diff (`diff -u`) giving the changes which are required to turn the original source into the Debian source. These changes may only include editing and creating plain files. The permissions of files, the targets of symbolic links and the characteristics of special files or pipes may not be changed and no files may be removed or renamed.

All the directories in the diff must exist, except the `debian` subdirectory of the top of the source tree, which will be created by `dpkg-source` if necessary when unpacking.

The `dpkg-source` program will automatically make the `debian/rules` file executable (see below).

If there is no original source code - for example, if the package is specially prepared for Debian or the Debian maintainer is the same as the upstream maintainer - the format is slightly different: then there is no diff, and the tarfile is named `package_version.tar.gz`, and preferably contains a directory named `package-version`.

15.4 Unpacking a Debian source package without `dpkg-source`

`dpkg-source -x` is the recommended way to unpack a Debian source package. However, if it is not available it is possible to unpack a Debian source archive as follows:

1. Untar the tarfile, which will create a `.orig` directory.
2. Rename the `.orig` directory to `package-version`.
3. Create the subdirectory `debian` at the top of the source tree.
4. Apply the diff using `patch -p0`.
5. Untar the tarfile again if you want a copy of the original source code alongside the Debian version.

It is not possible to generate a valid Debian source archive without using `dpkg-source`. In particular, attempting to use `diff` directly to generate the `.diff.gz` file will not work.

15.4.1 Restrictions on objects in source packages

The source package may not contain any hard links,²³ device special files, sockets or setuid or setgid files.⁴

The source packaging tools manage the changes between the original and Debian source using `diff` and `patch`. Turning the original source tree as included in the `.orig.tar.gz` into the Debian package source must not involve any changes which cannot be handled by these tools. Problematic changes which cause `dpkg-source` to halt with an error when building the source package are:

- Adding or removing symbolic links, sockets or pipes.
- Changing the targets of symbolic links.
- Creating directories, other than `debian`.
- Changes to the contents of binary files.

Changes which cause `dpkg-source` to print a warning but continue anyway are:

- Removing files, directories or symlinks.⁵
- Changed text files which are missing the usual final newline (either in the original or the modified source tree).

Changes which are not represented, but which are not detected by `dpkg-source`, are:

² This is not currently detected when building source packages, but only when extracting them.

³ Hard links may be permitted at some point in the future, but would require a fair amount of work.

⁴ Setgid directories are allowed.

⁵ Renaming a file is not treated specially - it is seen as the removal of the old file (which generates a warning, but is otherwise ignored), and the creation of the new one.

- Changing the permissions of files (other than `debian/rules`) and directories.

The `debian` directory and `debian/rules` are handled specially by `dpkg-source` - before applying the changes it will create the `debian` directory, and afterwards it will make `debian/rules` world-executable.

CONTROL FILES AND THEIR FIELDS (FROM OLD PACKAGING MANUAL)

Many of the tools in the `dpkg` suite manipulate data in a common format, known as control files. Binary and source packages have control data as do the `.changes` files which control the installation of uploaded files, and `dpkg`'s internal databases are in a similar format.

16.1 Syntax of control files

See *Syntax of control files*.

It is important to note that there are several fields which are optional as far as `dpkg` and the related tools are concerned, but which must appear in every Debian package, or whose omission may cause problems.

16.2 List of fields

See *List of fields*.

This section now contains only the fields that didn't belong to the Policy manual.

16.2.1 Filename

This field in `Packages` files give the filename(s) of (the parts of) a package in the distribution directories, relative to the root of the Debian hierarchy. If the package has been split into several parts the parts are all listed in order, separated by spaces.

16.2.2 Size and MD5sum

These fields in `Packages` files give the size (in bytes, expressed in decimal) and MD5 checksum of the file(s) which make(s) up a binary package in the distribution. If the package is split into several parts the values for the parts are listed in order, separated by spaces.

16.2.3 Status

This field in `dpkg`'s status file records whether the user wants a package installed, removed or left alone, whether it is broken (requiring re-installation) or not and what its current state on the system is. Each of these pieces of information is a single word.

16.2.4 Config-Version

If a package is not installed or not configured, this field in `dpkg`'s status file records the last version of the package which was successfully configured.

16.2.5 Conffiles

This field in `dpkg`'s status file contains information about the automatically-managed configuration files held by a package. This field should *not* appear anywhere in a package!

16.2.6 Obsolete fields

These are still recognized by `dpkg` but should not appear anywhere any more.

Revision; Package-Revision; Package_Revision

The Debian revision part of the package version was at one point in a separate control field. This field went through several names.

Recommended

Old name for `Recommends`.

Optional

Old name for `Suggests`.

Class

Old name for `Priority`.

CONFIGURATION FILE HANDLING (FROM OLD PACKAGING MANUAL)

`dpkg` can do a certain amount of automatic handling of package configuration files.

Whether this mechanism is appropriate depends on a number of factors, but basically there are two approaches to any particular configuration file.

The easy method is to ship a best-effort configuration in the package, and use `dpkg`'s `conffile` mechanism to handle updates. If the user is unlikely to want to edit the file, but you need them to be able to without losing their changes, and a new package with a changed version of the file is only released infrequently, this is a good approach.

The hard method is to build the configuration file from scratch in the `postinst` script, and to take the responsibility for fixing any mistakes made in earlier versions of the package automatically. This will be appropriate if the file is likely to need to be different on each system.

17.1 Automatic handling of configuration files by `dpkg`

A binary package may contain a package metadata file called `conffiles`. This file should be a list of filenames of configuration files needing automatic handling, separated by newlines. The filenames should be absolute pathnames, and the files referred to should actually exist in the package.

When a package is upgraded `dpkg` will process the configuration files during the configuration stage, shortly before it runs the package's `postinst` script,

For each file it checks to see whether the version of the file included in the package is the same as the one that was included in the last version of the package (the one that is being upgraded from); it also compares the version currently installed on the system with the one shipped with the last version.

If neither the user nor the package maintainer has changed the file, it is left alone. If one or the other has changed their version, then the changed version is preferred - i.e., if the user edits their file, but the package maintainer doesn't ship a different version, the user's changes will stay, silently, but if the maintainer ships a new version and the user hasn't edited it the new version will be installed (with an informative message). If both have changed their version the user is prompted about the problem and must resolve the differences themselves.

The comparisons are done by calculating the MD5 message digests of the files, and storing the MD5 of the file as it was included in the most recent version of the package.

When a package is installed for the first time `dpkg` will install the file that comes with it, unless that would mean overwriting a file already on the file system.

However, note that `dpkg` will *not* replace a `conffile` that was removed by the user (or by a script). This is necessary because with some programs a missing file produces an effect hard or impossible to achieve in another way, so that a missing file needs to be kept that way if the user did it.

Note that a package should *not* modify a `dpkg`-handled `conffile` in its maintainer scripts. Doing this will lead to `dpkg` giving the user confusing and possibly dangerous options for `conffile` update when the package is upgraded.

17.2 Fully-featured maintainer script configuration handling

For files which contain site-specific information such as the hostname and networking details and so forth, it is better to create the file in the package's `postinst` script.

This will typically involve examining the state of the rest of the system to determine values and other information, and may involve prompting the user for some information which can't be obtained some other way.

When using this method there are a couple of important issues which should be considered:

If you discover a bug in the program which generates the configuration file, or if the format of the file changes from one version to the next, you will have to arrange for the `postinst` script to do something sensible - usually this will mean editing the installed configuration file to remove the problem or change the syntax. You will have to do this very carefully, since the user may have changed the file, perhaps to fix the very problem that your script is trying to deal with - you will have to detect these situations and deal with them correctly.

If you do go down this route it's probably a good idea to make the program that generates the configuration file(s) a separate program in `/usr/sbin`, by convention called `packageconfig` and then run that if appropriate from the post-installation script. The `packageconfig` program should not unquestioningly overwrite an existing configuration - if its mode of operation is geared towards setting up a package for the first time (rather than any arbitrary reconfiguration later) you should have it check whether the configuration already exists, and require a `--force` flag to overwrite it.

ALTERNATIVE VERSIONS OF AN INTERFACE - `update-alternatives` (FROM OLD PACKAGING MANUAL)

When several packages all provide different versions of the same program or file it is useful to have the system select a default, but to allow the system administrator to change it and have their decisions respected.

For example, there are several versions of the `vi` editor, and there is no reason to prevent all of them from being installed at once, each under their own name (`nvi`, `vim` or whatever). Nevertheless it is desirable to have the name `vi` refer to something, at least by default.

If all the packages involved cooperate, this can be done with `update-alternatives`.

Each package provides its own version under its own name, and calls `update-alternatives` in its `postinst` to register its version (and again in its `prerm` to deregister it).

See the `update-alternatives(8)` man page for details.

If `update-alternatives` does not seem appropriate you may wish to consider using `diversions` instead.

Do not use alternatives for `systemd` configuration files. See [Binary packages](#) for more information.

DIVERSIONS - OVERRIDING A PACKAGE'S VERSION OF A FILE (FROM OLD PACKAGING MANUAL)

It is possible to have `dpkg` not overwrite a file when it reinstalls the package it belongs to, and to have it put the file from the package somewhere else instead.

This can be used locally to override a package's version of a file, or by one package to override another's version (or provide a wrapper for it).

Before deciding to use a diversion, read *Alternative versions of an interface - update-alternatives (from old Packaging Manual)* to see if you really want a diversion rather than several alternative versions of a program.

There is a diversion list, which is read by `dpkg`, and updated by a special program `dpkg-divert`. Please see *dpkg-divert (8)* for full details of its operation.

When a package wishes to divert a file from another, it should call `dpkg-divert` in its `preinst` to add the diversion and rename the existing file. For example, supposing that a `smailwrapper` package wishes to install a wrapper around `/usr/sbin/smail`:

```
dpkg-divert --package smailwrapper --add --rename \  
--divert /usr/sbin/smail.real /usr/sbin/smail
```

The `--package smailwrapper` ensures that `smailwrapper`'s copy of `/usr/sbin/smail` can bypass the diversion and get installed as the true version. It's safe to add the diversion unconditionally on upgrades since it will be left unchanged if it already exists, but `dpkg-divert` will display a message. To suppress that message, make the command conditional on the version from which the package is being upgraded:

```
if [ upgrade != "$1" ] || dpkg --compare-versions "$2" lt 1.0-2; then  
    dpkg-divert --package smailwrapper --add --rename \  
        --divert /usr/sbin/smail.real /usr/sbin/smail  
fi
```

where `1.0-2` is the version at which the diversion was first added to the package. Running the command during abort-upgrade is pointless but harmless.

The `postrm` has to do the reverse:

```
if [ remove = "$1" -o abort-install = "$1" -o disappear = "$1" ]; then  
    dpkg-divert --package smailwrapper --remove --rename \  
        --divert /usr/sbin/smail.real /usr/sbin/smail  
fi
```

If the diversion was added at a particular version, the `postrm` should also handle the failure case of upgrading from an older version (unless the older version is so old that direct upgrades are no longer supported):

```
if [ abort-upgrade = "$1" ] && dpkg --compare-versions "$2" lt 1.0-2; then
    dpkg-divert --package smailwrapper --remove --rename \
        --divert /usr/sbin/smail.real /usr/sbin/smail
fi
```

where 1.0-2 is the version at which the diversion was first added to the package. The postrm should not remove the diversion on upgrades both because there's no reason to remove the diversion only to immediately re-add it and since the postrm of the old package is run after unpacking so the removal of the diversion will fail.

Do not attempt to divert a file which is vitally important for the system's operation - when using `dpkg-divert` there is a time, after it has been diverted but before `dpkg` has installed the new version, when the file does not exist.

Do not attempt to divert a conffile, as `dpkg` does not handle it well.

Do not use diversions for files that have their own native override mechanisms, such as `systemd` unit files. See [Binary packages](#) for more information.

DEBIAN POLICY CHANGES PROCESS

20.1 Introduction

To introduce a change in the current Debian Policy, the change proposal has to go through a certain process.¹

20.2 Change Goals

- The change should be technically correct, and consistent with the rest of the policy document. This means no legislating the value of π . This also means that the proposed solution be known to work; iterative design processes do not belong in policy.
- The change should not be too disruptive; if very many packages become instantly buggy, then instead there should be a transition plan. Exceptions should be rare (only if the current state is really untenable), and probably blessed by the TC.
- The change has to be reviewed in depth, in the open, where any one may contribute; a publicly accessible, archived, open mailing list.
- Proposal should be addressed in a timely fashion.
- Any domain experts should be consulted, since not every policy mailing list subscriber is an expert on everything, including policy maintainers.
- The goal is rough consensus on the change, which should not be hard if the matter is technical. Technical issues where there is no agreement should be referred to the TC; non-technical issues should be referred to the whole developer body, and perhaps general resolutions lie down that path.
- Package maintainers whose packages may be impacted should have access to policy change proposals, even if they do not subscribe to policy mailing lists (policy gazette?).

20.3 Current Process

Each suggested change goes through different states. These states are denoted through either usertags of the debian-policy@packages.debian.org user or, for `moreinfo`, `patch`, `pending`, and `wontfix`, regular tags.

Current list of bugs

The Policy delegates are responsible for managing the tags on bugs and will update tags as new bugs are submitted or as activity happens on bugs. All Debian Developers should feel free to add the `seconded` tag as described below. Other tags should be changed with the coordination of the Policy Team.

¹ This process was originally developed by Margarita Manterola, Clint Adams, Russ Allbery and Manoj Srivastava. In 2017, Sean Whitton deprecated the 'issue' usertag and added use of the 'moreinfo' tag, after discussions at DebConf17.

20.3.1 State A: More information required

The Policy delegates are unable to determine whether the bug is really a Policy matter, or judge that there are missing details that would prevent a fruitful discussion (and may result in a confused and unhelpful discussion).

Policy delegates ask the original submitter to provide the missing details. Others are asked to refrain from discussing whatever they take the issue to be, limiting their postings to attempts to supply the missing details.

TAG: `moreinfo`

What needs to happen next: Submitter (or someone else) provides the requested information within 30 days, or the bug is closed.

The majority of bugs will skip this stage.

20.3.2 State B: Discussion

Discuss remedy. Alternate proposals. Discussion guided by delegates. There should be a clear time limit to this stage, but as yet we have not set one.

TAG: `discussion`

What needs to happen next: Reach a conclusion and consensus in the discussion and make a final proposal for what should be changed (if anything), moving to the proposal tag.

20.3.3 State C: Proposal

A final proposal has emerged from the discussion, and there is a rough consensus on how to proceed to resolve the issue.

TAG: `proposal`

What needs to happen next: Provided that the rough consensus persists, develop a patch against the current Policy document with specific wording of the change. Often this is done in conjunction with the proposal, in which case one may skip this step and move directly to patch tag.

20.3.4 State D: Wording proposed

A patch against the Policy document reflecting the consensus has been created and is waiting for formal seconds. The standard patch tag is used for this state, since it's essentially equivalent to the standard meaning of that tag.

TAG: `patch`

What needs to happen next: The proposal needs to be reviewed and seconded. Any Debian developer who agrees with the change and the conclusion of rough consensus from the discussion should say so in the bug log by seconding the proposal.

20.3.5 State E: Seconded

The proposal is signed off on by N Debian Developers. To start with, we're going with N=3, meaning that if three Debian Developers agree, not just with the proposal but with the conclusion that it reflects consensus and addresses the original issue – it is considered eligible for inclusion in the next version of Policy. Since Policy is partly a technical project governance method, one must be a Debian Developer to formally second, although review and discussion is welcome from anyone. Once this tag has been applied, the bug is waiting for a Policy team member to apply the patch to the package repository.

TAG: `seconded`

What needs to happen next: A Policy maintainer does the final review and confirmation, and then applies the patch for the next Policy release.

This tag is not used very much because normally a Policy maintainer applies the patch and moves the proposal to the next state once enough seconds are reached.

20.3.6 State F: Accepted

Change accepted, will be in next upload. The standard pending tag is used for this state since it matches the regular meaning of pending.

TAG: pending

What needs to happen next: The bug is now in the waiting queue for the next Policy release, and there's nothing left to do except for upload a new version of Policy.

20.3.7 State G: Reject

Rejected proposals. The standard wontfix is used for this state. Normally, bugs in this state will not remain open (excepting **stalled**); instead, a Policy team member will close them with an explanation. The submitter may then appeal to the tech-ctte if they so desire. Alternately, issues appealed to the tech-ctte may remain open with this tag while that appeal proceeds.

TAG: wontfix

We may use one of the following tags here. It's not clear whether we need more tags for this stage.

dubious

Not a policy matter

ctte

Referred to the Technical Committee (tech-ctte)

devel

Referred to the developer body

delegate

Rejected by a Policy delegate

obsolete

Consensus on a proposal was not forthcoming, and the bug is to be closed. Those wishing to restart discussion should open a new bug, but only if they have a concrete new change proposal.

stalled

Consensus on a proposal was not forthcoming. However, the bug should be kept open, as a form of documentation, and to minimise the number of duplicate filings.

What may need to happen next: The bug should be closed once a final resolution is reached (excepting **stalled**), or retagged to an appropriate state if that final resolution reverses the decision to reject the proposal.

20.4 Other Tags

All Policy bugs are additionally categorized by class of bug.

The normative tag is used for bugs that make normative changes to Policy, meaning that the dictates of Policy will change in some fashion as part of the resolution of the bug if the proposal is accepted. The full process is followed for such bugs.

TAG: normative

The informative tag is used for bugs about wording issues, typos, informative footnotes, or other changes that do not affect the formal dictates of Policy, just the presentation. The same tags are used for these bugs for convenience, but the Policy maintainers may make informative changes without following the full process. Informative bugs fall under their discretion.

TAG: informative

The packaging tag is used for bugs about the packaging and build process of the debian-policy Debian package. These bugs do not follow the normal process and will not have the other tags except for pending and wontfix (used with their normal meanings).

TAG: packaging

MAINTAINER SCRIPT FLOWCHARTS

The flowcharts¹ included in this appendix use the following conventions:

- maintainer scripts and their arguments are within boxes;
- actions carried out external to the scripts are in italics; and
- the `dpkg` status of the package at the end of the run are in bold type.

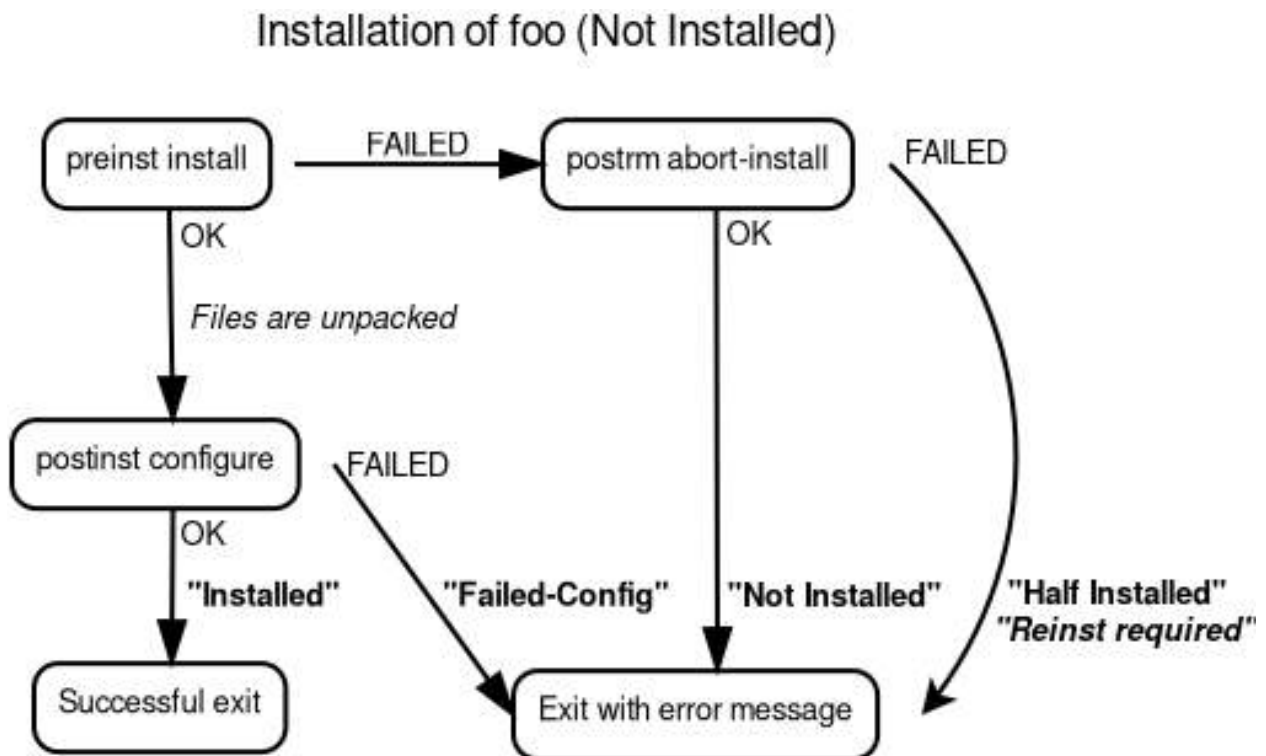


Fig. 1: Installing a package that was not previously installed

¹ These flowcharts were originally created by Margarita Manterola for the Debian Women project wiki.

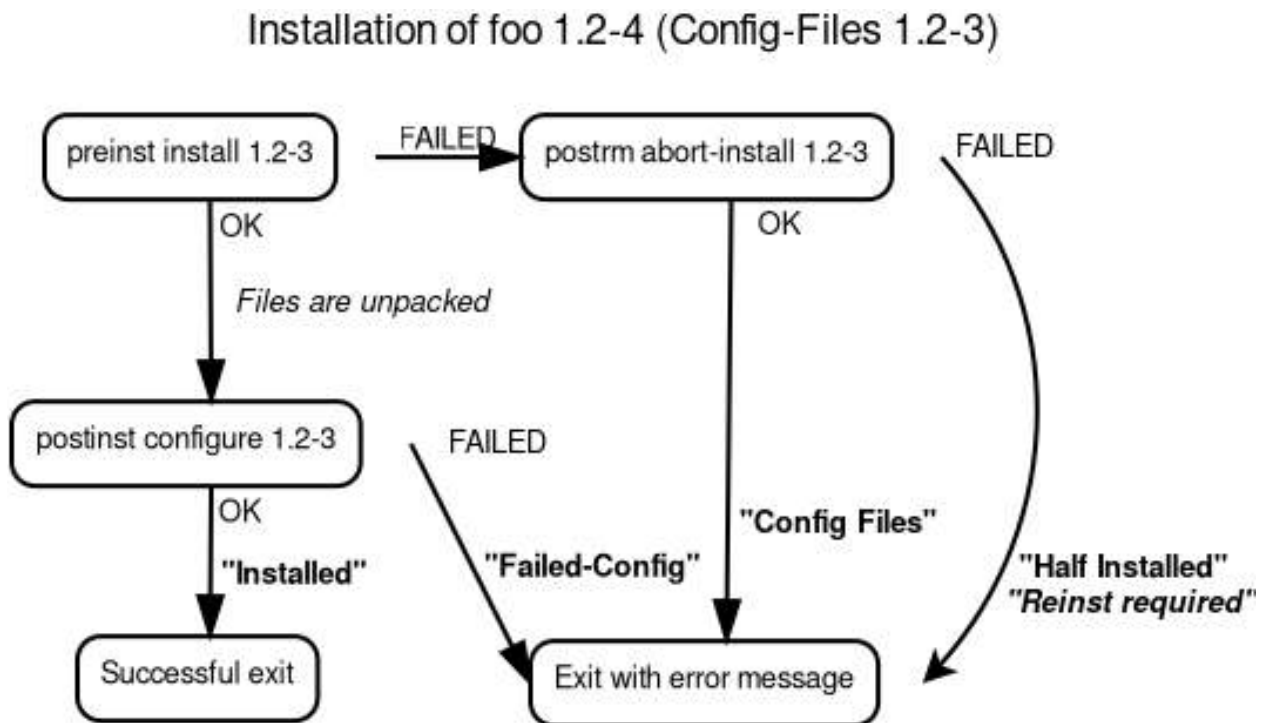


Fig. 2: Installing a package that was previously removed, but not purged

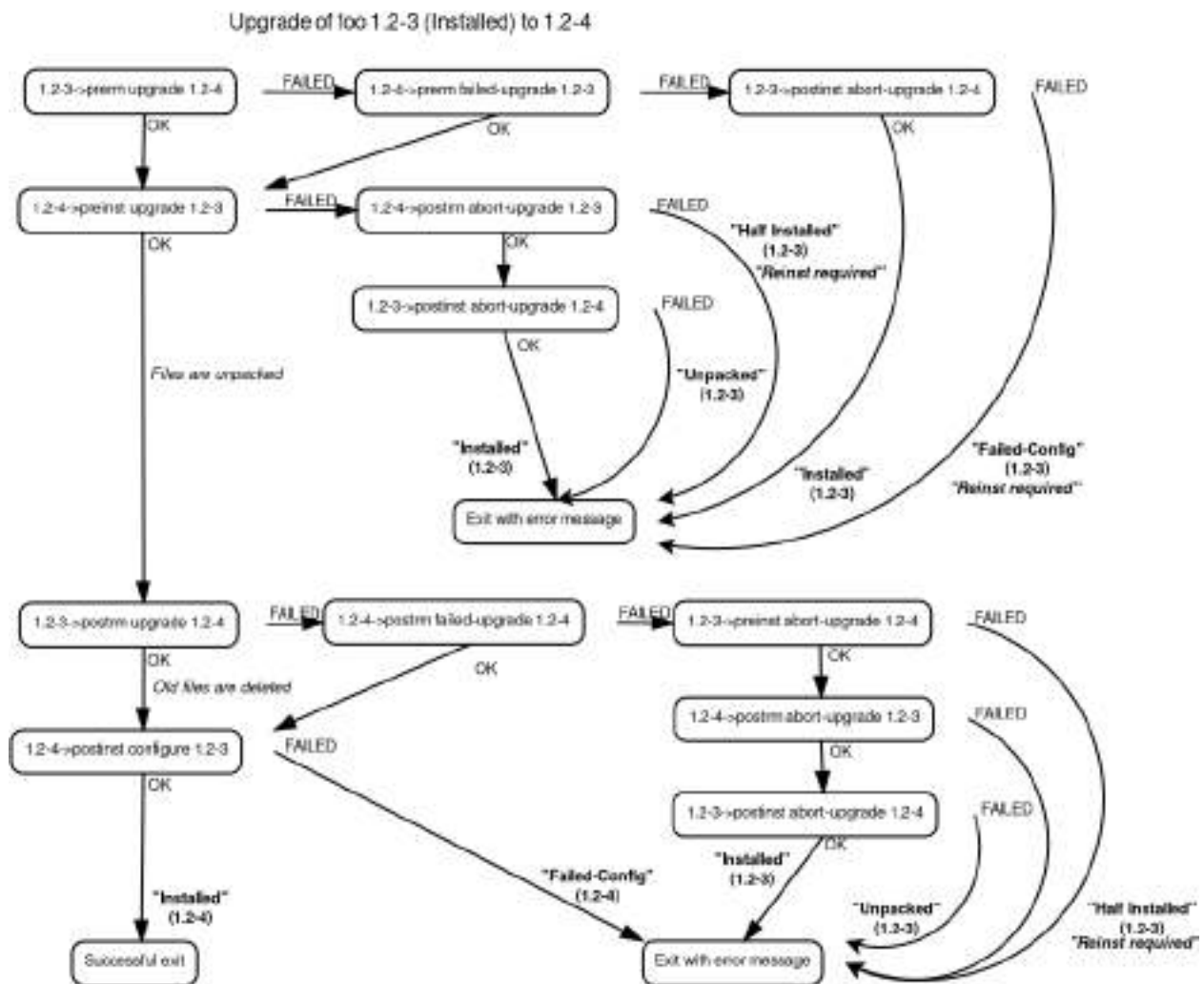


Fig. 3: Upgrading a package

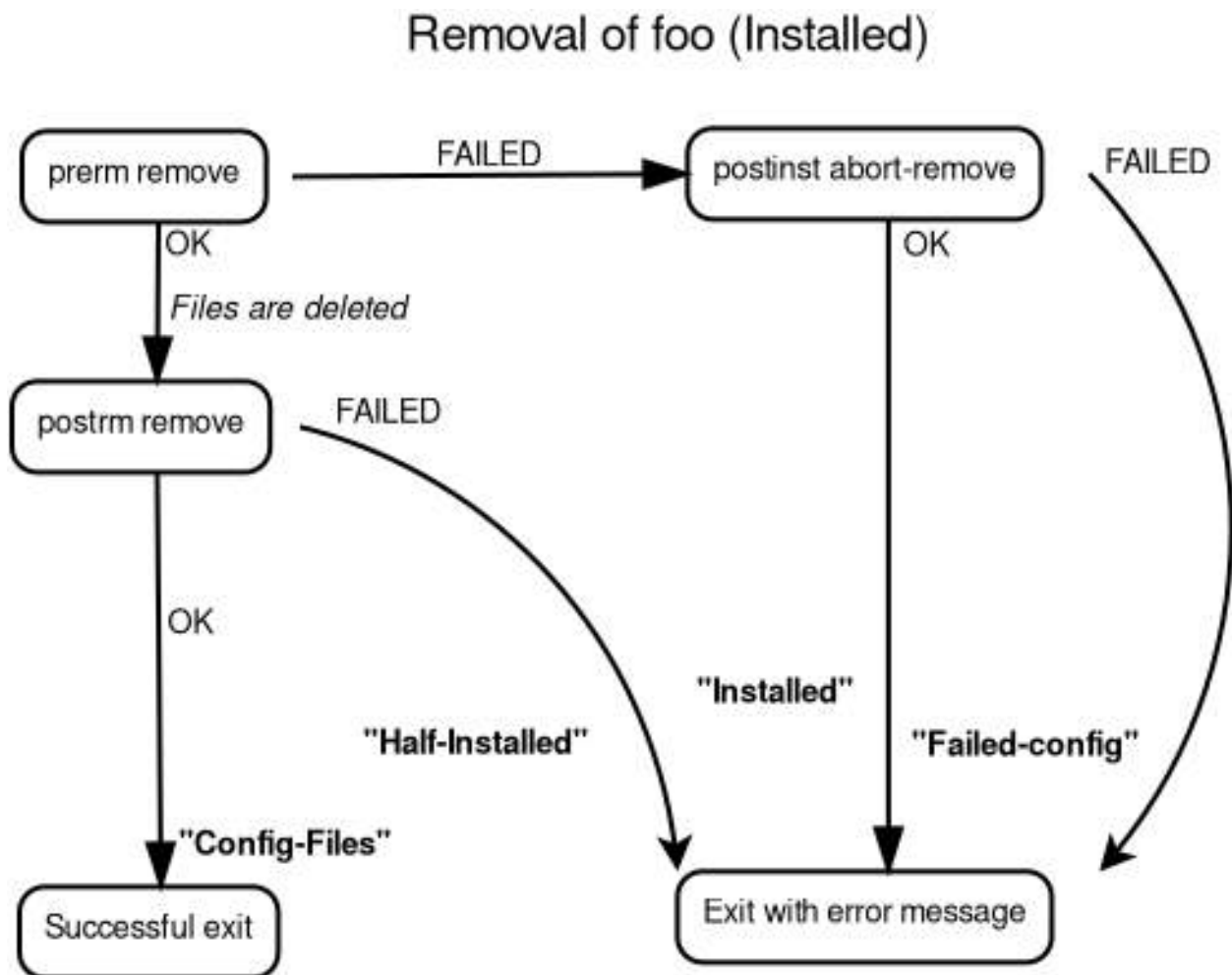


Fig. 4: Removing a package

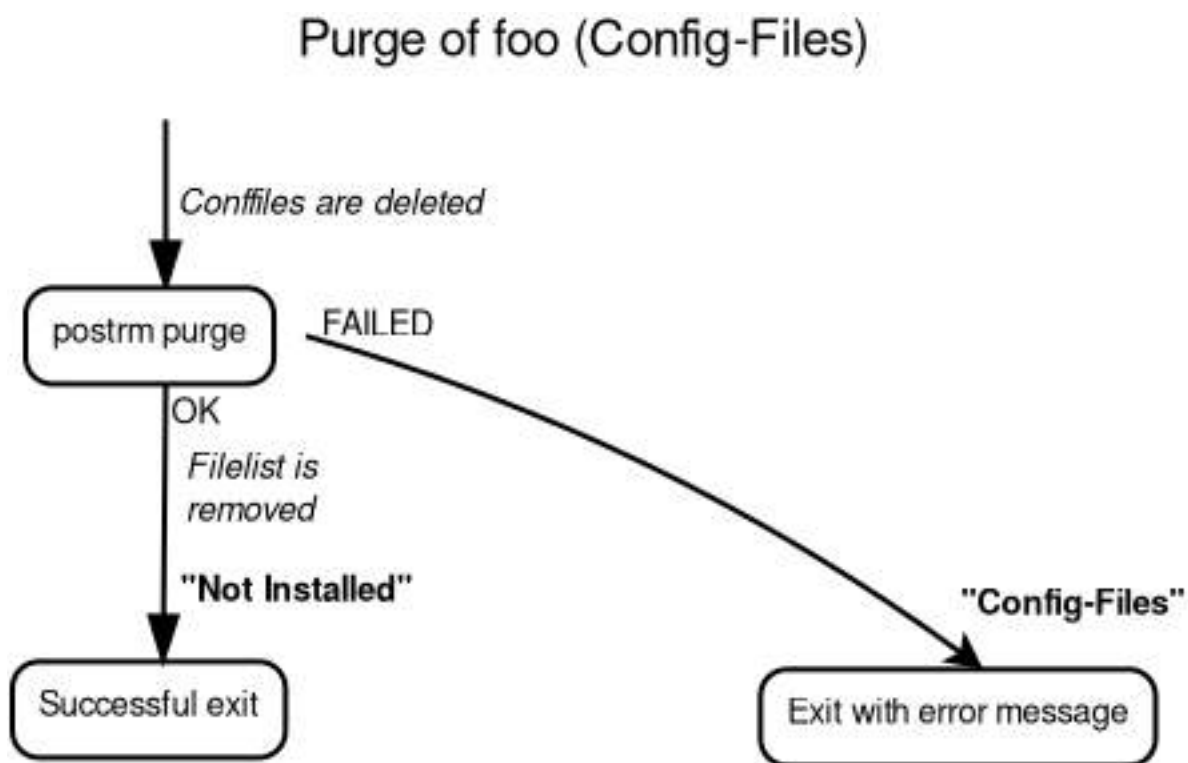


Fig. 5: Purging a package previously removed

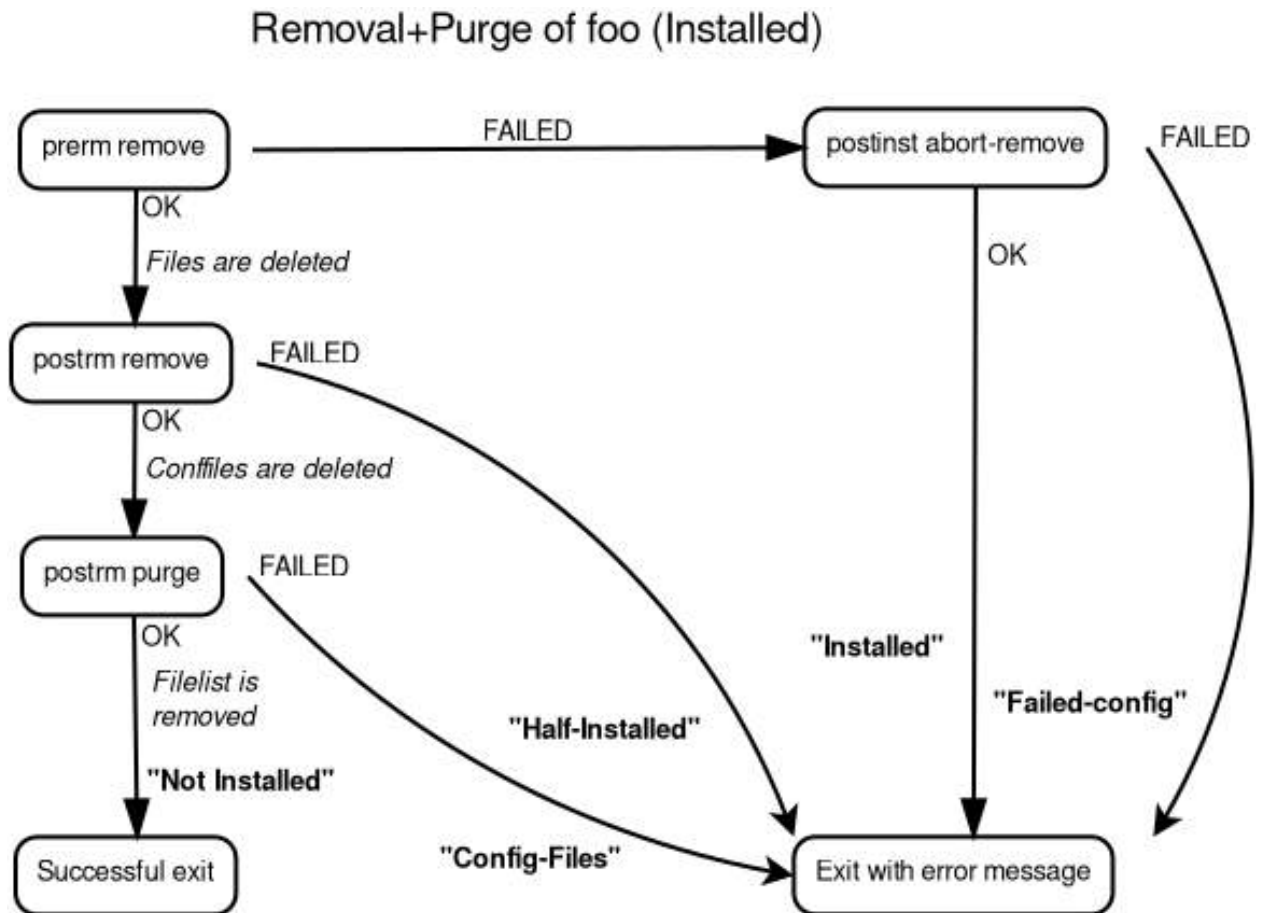


Fig. 6: Removing and purging a package

UPGRADING CHECKLIST

22.1 About the checklist

The checklist below has been created to simplify the upgrading process of old packages. This list is not official or normative. It only provides an indication of what has changed and whether you are likely to need to make changes to your package in light of this. If you have doubts about a certain topic, if you need more details, or if you think some other package does not comply with policy, please refer to the Policy Manual itself.

All of the changes from version 3.0.0 onwards indicate which section of the Policy Manual discusses the issue. The section numbering should still be accurate for changes back to the 2.5.0 release. Before that point, the sections listed here probably no longer correspond to sections in the modern Policy Manual.

Here is how the check list works: Check which policy version your package was checked against last (indicated in the `Standards-Version` field of the source package). Then move upwards until the top and check which of the items on the list might concern your package. Note which sections of policy discuss this, and then check out the Policy Manual for details. Once you've made all necessary changes to match the current rules, update the value of `Standards-Version` to the current Policy Manual version.

If an item in the list is followed by the name of a Lintian tag in square brackets, it indicates that the policy requirement is covered by that Lintian tag. The lack of such an annotation does not mean that no Lintian tag exists to cover the requirement. Our coverage of these annotations is quite incomplete, and patches to this checklist are very welcome.

The sections in this checklist match the values for the `Standards-Version` control field in omitting the minor patch version, except in the two anomalous historical cases where normative requirements were changed in a minor patch release.

22.2 Version 4.7.3

Released December, 2025.

5.6.6

It is no longer recommended to specify the `Priority` field in source package control files. `dpkg` now defaults it to `optional`.

5.6.32 & 5.6.33

New sections documenting the `Git-Tag-Tagger` and `Git-Tag-Info` fields in Debian source control and `.changes` files.

22.3 Version 4.7.2

Released February, 2025.

10.1

A change made the previous release of Policy is relaxed:

Packages that already install programs to `/usr/games`, where another package installs a program of the same name with different functionality to a different directory on the default `PATH`, may continue to do so.

Packages must not install any newly conflicting programs to `/usr/games`, and packages already doing so are encouraged to move these programs to non-conflicting names.

22.4 Version 4.7.1

Released February, 2025.

10.1

Two different packages must not install programs with different functionality to the same filenames, even names under different directories, when the directories are on the default `PATH`.

10.1

Packages must not install files to directories like `/bin`, `/lib`, `/lib*` and `/sbin` which are now symbolic links to corresponding directories under `/usr`. Packages may assume that `/bin`, `/lib` and `/sbin` are always symlinks, and that files under `/usr/bin`, `/usr/lib` and `/usr/sbin` are accessible via these aliases.

10.9

Packages should not require the existence of any files under `/usr/share/locale` in order to function in a `C` or `C.UTF-8` locale.

12.1

Packages (other than manpage readers) must not require the existence of any files under `/usr/share/man`.

12.2

Packages (other than info readers) must not require the existence of any files in `/usr/share/info`.

22.5 Version 4.7.0

Released April, 2024.

2.2.1

Document that source packages in the *main* archive area may build binary packages in the *contrib* archive area, although this is discouraged unless the source package is inconvenient to split. This does not relax the requirement that source packages in *main* must not have build dependencies outside of *main*.

2.2.2

The `non-free-firmware` archive area has been added.

3.9

Maintainer scripts should use native overriding mechanisms instead of `dpkg-divert`, wherever possible. Maintainer scripts must not divert configuration files used by `systemd` components.

Maintainer scripts must not use the alternatives system for `systemd` configuration files.

4.8

Hard links are permitted in source packages.

4.9

For packages in *contrib*, and for packages in *non-free* with `Autobuild: yes`, required targets in `d/rules` are no longer permitted to attempt network access. Previously, only packages in *main* had this restriction.

5.6.13

The `Description` field is not present in `.changes` files if no binary packages are being uploaded.

5.6.19

The `Binary` field is not present in `.changes` files if no binary packages are being uploaded.

6.3

Packages that automatically start or stop system services must include `systemd` units unless the service is only intended for use on systems running alternative init systems. Previously, `systemd` also supported init scripts, but that support is being removed.

22.6 Version 4.6.2

Released December, 2022.

3.8

Essential packages are only required to provide their core functionality when unconfigured if they had previously been configured at least once.

6.5 & 6.6

The new package version is provided as an additional argument following the old package version to several `preinst`, `prerm`, and `postrm` maintainer script actions.

11.8.4

When computing the priority for alternatives for `/usr/bin/x-window-manager`, start with a priority of 40, not 20, and don't increase the priority based on support for the (obsolete) Debian menu system.

22.7 Version 4.6.1

Released May, 2022.

9.1.1

Restore permission for packages for non-64-bit architectures to install files to `/usr/lib64/`.

9.7.2 & 9.7.3

The text of these sections has been updated to reflect the package split of `bin:mime-support` into `bin:media-types` and `bin:mailcap`.

22.8 Version 4.6.0

Released August, 2021.

9.1.1

No package is allowed to install files in `/usr/lib64/`. Previously, this prohibition only applied to packages for 64-bit architectures.

12.1

Manual pages may be included in dependencies, not only in the packages containing the things they document.

22.9 Version 4.5.1

Released November, 2020.

2.3, 4.5 & 12.5

The copyright information for files in a package must be copied verbatim into `/usr/share/doc/PACKAGE/copyright` when all of the following hold:

1. the distribution license for those files requires that copyright information be included in all copies and/or binary distributions;
2. the files are shipped in the binary package, either in source or compiled form; and

3. the form in which the files are present in the binary package does not include a plain text version of their copyright notices.

Note that there is no change to the requirement to copy all licensing information into `/usr/share/doc/PACKAGE/copyright`.

(Previously, it was always required for all copyright information to be copied into `/usr/share/doc/PACKAGE/copyright`.)

4.17

Packages must not contain a non-default series file. That is, `dpkg`'s vendor-specific patch series feature must not be used for packages in the Debian archive.

(previously a “should not”)

22.10 Version 4.5.0

Released January, 2020.

9.2.1

When maintainers choose a new hardcoded or dynamically generated username for packages to use, they should start this username with an underscore.

9.3.1

Packages that include system services should include `systemd` service units to start or stop those services.

Including an init script is encouraged if there is no `systemd` unit, and optional if there is (previously, it was recommended).

In the common case that a package includes a single system service, the service unit should have the same name as the package plus the “.service” extension. If an init script is included, it should have the same name as the `systemd` unit.

9.3.2

It is encouraged for init scripts to support the `status` argument (previously, it was recommended).

9.3.3

Use of `update-rc.d` is required if the package includes an init script (previously, Policy said in one place that it was required, and in another said that it was recommended).

22.11 Version 4.4.1

Released September, 2019.

5.6.26

A package control file must not have more than one `Vcs-<type>` field.

If the package is maintained in multiple version control systems, the maintainer should specify the one that they would prefer other people to use as the basis for proposing changes to the package.

9.10 & 11.5

doc-base registration is now optional, from being recommended.

9.12

Document `/run/reboot-required` mechanism.

copyright-format

State some syntactical restrictions on the `Files:` field. Wildcards are required to match the contents of directories, and the space character separates patterns and cannot be escaped.

22.12 Version 4.4.0

Released July, 2019.

4.9

The recommended way to implement the build process of a Debian package, in the absence of a good reason to use a different approach, is the `dh` tool. This recommendation includes the contents of the `debian/rules` building script.

Some examples of good reasons to use a different approach are given. The recommendation to use `dh` does not always apply, and use of `dh` is not required.

5.6.26

Permit `-b` in Vcs-Hg as well as Vcs-Git.

7.5

Document versioned Provides.

virtual

New `logind` and `default-logind` virtual packages for a package providing `logind` API (via D-Bus and `sd-login(3)`), and for Debian's preferred implementation, respectively.

22.13 Version 4.3.0

Released December, 2018.

2.3 & 4.5

In cases where a package's distribution license explicitly permits its copyright information to be excluded from distributions of binaries built from the source, a verbatim copy of the package's copyright information should normally still be included in the copyright file, but it need not be if creating and maintaining a copy of that information involves significant time and effort.

4.9

Required targets must not write outside of the unpacked source package tree, except for `TMPDIR`, `/tmp` and `/var/tmp`.

4.17

Packages should not contain a non-default series file. That is, `dpkg`'s vendor-specific patch series feature should not be used for packages in the Debian archive.

10.1

Binaries should be stripped using `strip --strip-unneeded --remove-section=.comment --remove-section=.note` (as `dh_strip` already does).

10.1

It is no longer suggested nor recommended to use `install -s` to strip binaries, because it gets several things wrong.

10.2

When stripping shared libraries with `strip(1)`, you should additionally pass `--remove-section=.comment --remove-section=.note` (as `dh_strip` already does).

virtual

New `dbus-session-bus` and `default-dbus-session-bus` virtual packages for a package providing the D-Bus session bus, and for Debian's preferred D-Bus implementation, respectively.

22.14 Version 4.2.1

Released August, 2018.

10.4 & perl

The requirement that the shebang at the top of Perl command scripts be `#!/usr/bin/perl` is relaxed from a ‘must’ to a ‘should’.

22.15 Version 4.2.0

Released August, 2018.

4.9

The package build should be as verbose as reasonably possible. This means that `debian/rules` should pass to the commands it invokes options that cause them to produce verbose output.

4.9

Required targets may attempt network access, via the loopback interface, to services on the build host that have been started by the build.

4.9.1

New `terse` tag that can appear in `DEB_BUILD_OPTIONS` to make a package build less verbose.

5.2 & 5.4

The Standards-Version field is now mandatory, not just recommended.

12.7

Upstream release notes, when available, should be installed as `/usr/share/doc/package/NEWS.gz`. Upstream changelogs may be made available as `/usr/share/doc/package/changelog.gz`.

This is a relaxation of older Policy which said that the upstream changelog should be made accessible at this path. Now it is up to maintainer discretion whether it is useful to install it.

The practice of installing the upstream release notes as `/usr/share/doc/package/changelog.gz` is permitted but deprecated.

22.16 Version 4.1.5

Released July, 2018.

4.9.2

Document how `debian/rules` and the `Rules-Requires-Root` field interact.

5.6.12

You should not change a package’s epoch, even in experimental, without getting consensus on `debian-devel` first.

5.6.12.1

Epochs should not be used for the purpose of rolling back the version of a package. Use the `+really` convention.

5.6.31

Document the `Rules-Requires-Root` field.

9.1.1

Update Debian’s version of the Filesystem Hierarchy Standard from 2.3 to 3.0, and update the list of exceptions. Only a tiny minority of packages, if any, should be made buggy by this change.

9.3.2 & 10.4

Update version of POSIX standard for shell scripts from SUSv3 to POSIX.1-2017 (also known as SUSv4 in some contexts).

22.17 Version 4.1.4

Released April, 2018.

3.2.2

The part of the version number after the epoch must not be reused for a version of the package with different contents, even after the version of the package previously using that part of the version number is no longer present in any archive suites.

3.2.2

For non-native packages, the upstream version must not be reused for different upstream source code, so that for each source package name and upstream version number there exists exactly one original source archive contents.

4.9

The `get-orig-source` rules target has been removed. Packages should transition to `debian/watch` and use `uscan` where possible.

9.1.2

If `/etc/staff-group-for-usr-local` does not exist, `/usr/local` and all subdirectories created by packages should have permissions `0755` and be owned by `root:root`. If the file exists, the old permissions of `2775` and ownership of `root:staff` should remain.

22.18 Version 4.1.3

Released December, 2017.

5.6.26

URLs given in `VCS-*` headers should use a scheme that provides confidentiality (`https`, for example) if the VCS repository supports it. [`vcs-field-uses-insecure-uri`]

7.8

`Built-Using` should be used exactly when there are license or DFSG requirements to retain full source code in the archive. Previously, the description of the field implied it was needed in other cases too.

9.1.1

`libc` may also install files in `/lib64`.

9.3.3.1

If a package's daemon should not be autostarted unless the local administrator has explicitly requested it, the package's `postinst` should use the new `defaults-disabled` option of `update-rc.d`.

The old method of including `DISABLED=yes` in the package's `/etc/default` file should not be used.

11.4

Clarify that programs may invoke either `/usr/bin/editor` and `/usr/bin/pager` directly, or use `editor` and `pager` and rely on `PATH`.

12.5

The Creative Commons CC0 1.0 Universal license (CC0-1.0) is now included in `/usr/share/common-licenses` and does not need to be copied verbatim in the package `copyright` file.

22.19 Version 4.1.2

Released November, 2017.

3.1.1

Binary packages that have potentially offensive content should have the suffix `-offensive`. This replaces an older

convention to use `-off`. As before, whether the contents of a package needs this content warning is a matter of maintainer discretion.

10.2

Private shared object files should be installed in subdirectories of `/usr/lib` or `/usr/lib/triplet`. This change permits private shared object files to take advantage of multiarch, and also removes the implication that it is permissible to install private shared object files directly into `/usr/lib/triplet`.

10.4

The shebang at the top of Perl command scripts must be `#!/usr/bin/perl`. (Previously, this was a ‘should’ rather than a ‘must’.)

22.20 Version 4.1.1

Released September, 2017.

4.4

`debian/changelog` must exist in source packages.

9.2.3

The canonical non-existent home directory is `/nonexistent`.

22.21 Version 4.1.0

Released August, 2017.

2.2.1

Non-default alternative dependencies on non-free packages are permitted for packages in `main`.

4.11

If upstream provides OpenPGP signatures, including the upstream signing key as `debian/upstream/signing-key.asc` in the source package and using the `pgpsigurlmangle` option in `debian/watch` configuration to indicate how to find the upstream signature for new releases is recommended.

4.15

Packages should build reproducibly when certain factors are held constant; see 4.15 for the list.

4.15

Packages are recommended to build reproducibly even when build paths and most environment variables are allowed to vary.

9.1.1

Only the dynamic linker may install files to `/lib64/`.

No package for a 64 bit architecture may install files to `/usr/lib64/` or any subdirectory.

11.8.3

The required behaviour of `x-terminal-emulator -e` has been clarified, and updated to replace a false claim about the behaviour of `xterm`.

Programs must support `-e command` where `command` may include multiple arguments, which must be executed as if the arguments were passed to `execvp` directly, bypassing the shell.

If this execution fails and `-e` has a single argument, `xterm`’s fallback behaviour of passing `command` to the shell is permitted but not required.

22.22 Version 4.0.1

Released August, 2017.

2.5

Priorities are now used only for controlling which packages are part of a minimal or standard Debian installation and should be selected based on functionality provided directly to users (so nearly all shared libraries should have a priority of `optional`). Packages may now depend on packages with a lower priority.

The `extra` priority has been deprecated and should be treated as equivalent to `optional`. All `extra` priorities should be changed to `optional`. Packages with a priority of `optional` may conflict with each other (but packages that both have a priority of `standard` or higher still may not conflict).

5.6.30

New section documenting the `Testsuite` field in Debian source control files.

8.1.1

Shared libraries must now invoke `ldconfig` by means of triggers, instead of maintscripts.

9.3.3

Packages are recommended to use debhelper tools instead of invoking `update-rc.d` and `invoke-rc.d` directly.

9.3.3

Policy's description of how the local system administrator may modify the runlevels at which a daemon is started and stopped, and how init scripts may depend on other init scripts, have been removed. These are now handled by LSB headers.

9.4

Policy's specification of the console messages that should be emitted by `init.d` scripts has been removed. This is now defined by LSB, for `sysvinit`, and is not expected to be followed by other init systems.

9.6

Packages installing a Free Desktop entry must not also install a Debian menu system entry.

9.9

The prohibition against depending on environment variables for reasonable defaults is only for programs on the system `PATH` and only for custom environment variable settings (not, say, a sane `PATH`).

22.23 Version 4.0.0

Released May, 2017.

4.3

`config.sub` and `config.guess` should be updated at build time or replaced with the versions from `autotools-dev`.

4.9

New `TARGET` set of `dpkg-architecture` variables and new `DEB_*_ARCH_BITS` and `DEB_*_ARCH_ENDIAN` variables.

4.9.1

New `DEB_BUILD_OPTIONS` tag, `nodoc`, which says to suppress documentation generation (but continue to build all binary packages, even documentation packages, just let them be mostly empty).

5.2

Automatically-generated debug packages do not need to have a corresponding stanza in `debian/control`. (This is existing practice; this Policy update is just clearer about it.)

5.6.12

Colons are not permitted in upstream version numbers.

7.7

New `Build-Depends-Arch` and `Build-Conflicts-Arch` fields are now supported.

8.4

The recommended package name for shared library development files is now `libraryname-dev` or `librarynameapiversion-dev`, not `librarynamesoversion-dev`.

9.1.1

The stable release of Debian supports `/run`, so packages may now assume that it exists and do not need any special dependency on a version of `initscripts`.

9.3.2

New optional `try-restart` standard init script argument, which (if supported) should restart the service if it is already running and otherwise just report success.

9.3.2

Support for the `status` init script argument is recommended.

9.3.3.2

Packages must not call `/etc/init.d` scripts directly even as a fallback, and instead must always use `invoke-rc.d` (which is essential and shouldn't require any conditional).

9.11.1

Instructions for `upstart` integration removed since `upstart` is no longer maintained in Debian.

10.1

Packages may not install files in both `/path` and `/usr/path`, and must manage any backward-compatibility symlinks so that they don't break if `/path` and `/usr/path` are the same directory.

10.6

Packages should assume device files in `/dev` are dynamically managed and don't have to be created by the package. Packages other than those whose purpose is to manage `/dev` must not create or remove files there when a dynamic management facility is in use. Named pipes and device files outside of `/dev` should normally be created on demand via init scripts, systemd units, or similar mechanisms, but may be created and removed in maintainer scripts if they must be created during package installation.

10.9

Checking with the `base-passwd` maintainer is no longer required (or desirable) when creating a new dynamic user or group in a package.

12.3

Dependencies on `*-doc` packages should be at most `Recommends` (Suggests if they only include documentation in supplemental formats).

12.5

The Mozilla Public License 1.1 and 2.0 (MPL-1.1 and MPL-2.0) are now included in `/usr/share/common-licenses` and do not need to be copied verbatim in the package `copyright` file.

copyright-format

The `https` form of the `copyright-format` URL is now allowed and preferred in the `Format` field.

perl

The Perl search path now includes `multiarch` directories. The vendor directory for architecture-specific modules is now versioned to support `multiarch`.

virtual

New `adventure` virtual package for implementations of the classic Colossal Cave Adventure game.

virtual

New `httpd-wsgi3` virtual package for Python 3 WSGI-capable HTTP servers. The existing `httpd-wsgi` virtual package is for Python 2 WSGI-capable HTTP servers.

virtual

New `virtual-mysql-client`, `virtual-mysql-client-core`, `virtual-mysql-server`, `virtual-mysql-server-core`, and `virtual-mysql-testsuite` virtual packages for MySQL-compatible software.

22.24 Version 3.9.8

Released April, 2016.

9.6

The menu system is deprecated in favor of the FreeDesktop menu standard. New requirements set for FreeDesktop menu entries.

9.7

New instructions for registering media type handlers with the FreeDesktop system, which automatically synchronizes with mailcap and therefore replaces mailcap registration for packages using desktop entries.

22.25 Version 3.9.7

Released February, 2016.

10.5

Symbolic links must not traverse above the root directory.

9.2.2

32bit UIDs in the range 65536-4294967293 are reserved for dynamically allocated user accounts.

5.1

Empty field values in control files are only permitted in the `debian/control` file of a source package.

4.9

`debian/rules`: required targets must not attempt network access.

12.3

recommend to ship additional documentation for package `pkg` in a separate package `pkg-doc` and install it into `/usr/share/doc/pkg`.

22.26 Version 3.9.6

Released September, 2014.

9.1

The FHS is relaxed to allow a subdirectory of `/usr/lib` to hold a mixture of architecture-independent and architecture-dependent files, though directories entirely composed of architecture-independent files should be located in `/usr/share`.

9.1

The FHS requirement for `/usr/local/lib64` to exist if `/lib64` or `/usr/lib64` exists is removed.

9.1

An FHS exception has been granted for multiarch include files, permitting header files to instead be installed to `/usr/include/triplet`.

10.1

Binaries must not be statically linked with the GNU C library, see policy for exceptions.

4.4

It is clarified that signature appearing in `debian/changelog` should be the details of the person who prepared this release of the package.

11.5

The default web document root is now `/var/www/html`

virtual

`java1-runtime` and `java2-runtime` are removed, `javaN-runtime` and `javaN-runtime-headless` are added for all `N` between 5 and 9.

virtual

Added `httpd-wsgi` for WSGI capable HTTP servers.

perl

Perl packages should use the `%Config` hash to locate module paths instead of hardcoding paths in `@INC`.

perl

Perl binary modules and any modules installed into `$Config{vendorarch}` must depend on the relevant `perlapi-*` package.

22.27 Version 3.9.5

Released October, 2013.

5.1

Control data fields must not start with the hyphen character (`-`), to avoid potential confusions when parsing clear-signed control data files that were not properly unescaped.

5.4, 5.6.24

`Checksums-Sha1` and `Checksums-Sha256` are now mandatory in `.dsc` files.

5.6.25, 5.8.1

The `DM-Upload-Allowed` field is obsolete. Permissions are now granted via *`dak-commands`* files.

5.6.27

New section documenting the `Package-List` field in Debian source control files.

5.6.28

New section documenting the `Package-Type` field in source package control files.

5.6.29

New section documenting the `Dgit` field in Debian source control files.

9.1.1.8

The exception to the FHS for the `/selinux` was removed.

10.7.3

Packages should remove all obsolete configuration files without local changes during upgrades. The `dpkg-maintscript-helper` tool, available from the `dpkg` package since *Wheezy*, can help with this.

10.10

The name of the files and directories installed by binary packages must be encoded in UTF-8 and should be restricted to ASCII when possible. In the system `PATH`, they must be restricted to ASCII.

11.5.2

Stop recommending to serve HTML documents from `/usr/share/doc/package`.

12.2

Packages distributing Info documents should use `install-info`'s trigger, and do not need anymore to depend on `dpkg` (`>= 1.15.4`) | `install-info`.

debconf

The `escape` capability is now documented.

virtual

`mp3-decoder` and `mp3-encoder` are removed.

22.28 Version 3.9.4

Released August, 2012.

2.4

New *tasks* archive section.

4.9

`build-arch` and `build-indep` are now mandatory targets in `debian/rules`.

5.6.26

New section documenting the `Vcs-*` fields, which are already in widespread use. Note the mechanism for specifying the Git branch used for packaging in the `Vcs-Git` field.

7.1

The deprecated relations `<` and `>` now must not be used.

7.8

New `Built-Using` field, which must be used to document the source packages for any binaries that are incorporated into this package at build time. This is used to ensure that the archive meets license requirements for providing source for all binaries.

8.6

Policy for dependencies between shared libraries and other packages has been largely rewritten to document the `symbols` system and more clearly document handling of shared library ABI changes. `symbols` files are now recommended over `shlibs` files in most situations. All maintainers of shared library packages should review the entirety of this section.

9.1.1

Packages must not assume the `/run` directory exists or is usable without a dependency on `initscripts` (`>= 2.88dsf-13.3`) until the stable release of Debian supports `/run`.

9.7

Packages including MIME configuration can now rely on triggers and do not need to call `update-mime`.

9.11

New section documenting general requirements for alternate init systems and specific requirements for integrating with `upstart`.

12.5

All copyright files must be encoded in UTF-8.

22.29 Version 3.9.3

Released February, 2012.

2.4

New archive sections *education*, *introspection*, and *metapackages* added.

5.6.8

The `Architecture` field in `*.dsc` files may now contain the value `any all` for source packages building both architecture-independent and architecture-dependent packages.

7.1

If a dependency is restricted to particular architectures, the list of architectures must be non-empty.

9.1.1

`/run` is allowed as an exception to the FHS and replaces `/var/run`. `/run/lock` replaces `/var/lock`. The FHS requirements for the older directories apply to these directories as well. Backward compatibility links will be maintained and packages need not switch to referencing `/run` directly yet. Files in `/run` should be stored in a temporary file system.

9.1.4

New section spelling out the requirements for packages that use files in `/run`, `/var/run`, or `/var/lock`. This generalizes information previously only in 9.3.2.

9.5

Cron job file names must not contain `.` or `+` or they will be ignored by cron. They should replace those characters with `_`. If a package provides multiple cron job files in the same directory, they should each start with the package name (possibly modified as above), `-`, and then some suitable suffix.

9.10

Packages using doc-base do not need to call `install-docs` anymore.

10.7.4

Packages that declare the same `conffile` may see left-over configuration files from each other even if they conflict.

11.8

The Policy rules around Motif libraries were just a special case of normal rules for non-free dependencies and were largely obsolete, so they have been removed.

12.5

`debian/copyright` is no longer required to list the Debian maintainers involved in the creation of the package (although note that the requirement to list copyright information is unchanged).

copyright-format

Version 1.0 of the “Machine-readable `debian/copyright` file” specification is included.

mime

This separate document has been retired and its (short) contents merged into Policy section 9.7. There are no changes to the requirements.

perl

Packages may declare an interest in the `perl-major-upgrade` trigger to be notified of major upgrades of perl.

virtual

`ttf-japanese-{mincho, gothic}` is renamed to `fonts-japanese-{mincho, gothic}`.

22.30 Version 3.9.2

Released April, 2011.

*

Multiple clarifications throughout Policy where “installed” was used and the more precise terms “unpacked” or “configured” were intended.

3.3

The maintainer address must accept mail from Debian role accounts and the BTS. At least one human must be listed with their personal email address in `Uploaders` if the maintainer is a shared email address. The duties of a maintainer are also clearer.

5

All control fields are now classified as simple, folded, or multiline, which governs whether their values must be a single line or may be continued across multiple lines and whether line breaks are significant.

5.1

Parsers are allowed to accept stanza separation lines containing whitespace, but control files should use completely empty lines. Ordering of stanzas is significant. Field names must be composed of printable ASCII characters except colon and must not begin with #.

5.6.25

The `DM-Upload-Allowed` field is now documented.

6.5

The system state maintainer scripts can rely upon during each possible invocation is now documented. In several less-common cases, this is stricter than Policy had previously documented. Packages with complex maintainer scripts should be reviewed in light of this new documentation.

7.2

The impact on system state when maintainer scripts that are part of a circular dependency are run is now documented. Circular dependencies are now a should not.

7.2

The system state when `postinst` and `prerm` scripts are run is now documented, and the documentation of the special case of dependency state for `postrm` scripts has been improved. `postrm` scripts are required to gracefully skip actions if their dependencies are not available.

9.1.1

GNU/Hurd systems are allowed `/hurd` and `/servers` directories in the root filesystem.

9.1.1

Packages installing to architecture-specific subdirectories of `/usr/lib` must use the value returned by `dpkg-architecture -qDEB_HOST_MULTIARCH`, not by `dpkg-architecture -qDEB_HOST_GNU_TYPE`; this is a path change on i386 architectures and a no-op for other architectures.

virtual

`mailx` is now a virtual package provided by packages that install `/usr/bin/mailx` and implement at least the POSIX-required interface.

22.31 Version 3.9.1

Released July, 2010.

3.2.1

Date-based version components should be given as the four-digit year, two-digit month, and then two-digit day, but may have embedded punctuation.

3.9

Maintainer scripts must pass `--package` to `dpkg-divert` when creating or removing diversions and must not use `--local`.

4.10

Only `dpkg-gencontrol` supports variable substitution. `dpkg-genchanges` (for `*.changes`) and `dpkg-source` (for `*.dsc`) do not.

7.1

Architecture restrictions and wildcards are also allowed in binary package relationships provided that the binary package is not architecture-independent.

7.4

`Conflicts` and `Breaks` should only be used when there are file conflicts or one package breaks the other, not just because two packages provide similar functionality but don't interfere.

8.1

The SONAME of a library should change whenever the ABI of the library changes in a way that isn't backward-compatible. It should not change if the library ABI changes are backward-compatible. Discourage bundling shared libraries together in one package.

8.4

Ada Library Information (`*.ali`) files must be installed read-only.

8.6.1, 8.6.2, 8.6.5

Packages should normally not include a `shlibs.local` file since we now have complete `shlibs` coverage.

8.6.3

The SONAME of a library may instead be of the form `name-major-version.so`.

10.2

Libtool `.la` files should not be installed for public libraries. If they're required (for `libltdl`, for instance), the `dependency_libs` setting should be emptied. Library packages historically including `.la` files must continue to include them (with `dependency_libs` emptied) until all libraries that depend on that library have removed or emptied their `.la` files.

10.2

Libraries no longer need to be built with `-D_REENTRANT`, which was an obsolete LinuxThreads requirement. Instead, say explicitly that libraries should be built with threading support and to be thread-safe if the library supports this.

10.4

`/bin/sh` scripts may assume that `kill` supports an argument of `-signal`, that `kill` and `trap` support the numeric signals listed in the XSI extension, and that signal 13 (SIGPIPE) can be trapped with `trap`.

10.8

Use of `/etc/logrotate.d/package` for `logrotate` rules is now recommended.

10.9

Control information files should be owned by `root:root` and either mode 644 or mode 755.

11.4, 11.8.3, 11.8.4

Packages providing alternatives for `editor`, `pager`, `x-terminal-emulator`, or `x-window-manager` should also provide a slave alternative for the corresponding manual page.

11.5

Cgi-bin executable files may be installed in subdirectories of `/usr/lib/cgi-bin` and web servers should serve out executables in those subdirectories.

12.5

The GPL version 1 is now included in `common-licenses` and should be referenced from there instead of included in the `copyright` file.

22.32 Version 3.9.0

Released June, 2010.

4.4, 5.6.15

The required format for the date in a changelog entry and in the `Date` control field is now precisely specified.

5.1

A control stanza must not contain more than one instance of a particular field name.

5.4, 5.5, 5.6.24

The `Checksums-Sha1` and `Checksums-Sha256` fields in `*.dsc` and `*.changes` files are now documented and recommended.

5.5, 5.6.16

The `Format` field of `.changes` files is now 1.8. The `Format` field syntax for source package `.dsc` files allows a subtype in parentheses, and it is used for a different purpose than the `Format` field for `.changes` files.

5.6.2

The syntax of the `Maintainer` field is now `must` rather than `should`.

5.6.3

The comma separating entries in `Uploaders` is now `must` rather than `should`.

5.6.8, 7.1, 11.1.1

Architecture wildcards may be used in addition to specific architectures in `debian/control` and `*.dsc` Architecture fields, and in architecture restrictions in build relationships.

6.3

Maintainer scripts are no longer guaranteed to run with a controlling terminal and must be able to fall back to noninteractive behavior (`debconf` handles this). Maintainer scripts may abort if there is no controlling terminal and no reasonable default for a high-priority question, but should avoid this if possible.

7.3, 7.6.1

`Breaks` should be used with `Replaces` for moving files between packages.

7.4

`Breaks` should normally be used instead of `Conflicts` for transient issues and moving files between packages. New documentation of when each should be used.

7.5

Use `Conflicts` with `Provides` if only one provider of a virtual facility can be installed at a time.

8.4

All shared library development files are no longer required to be in the `-dev` package, only be available when the `-dev` package is installed. This allows the `-dev` package to be split as long as it depends on the additional packages.

9.2.2

The UID range of user accounts is extended to 1000-59999.

9.3.2, 10.4

`init.d` scripts are a possible exception from the normal requirement to use `set -e` in each shell script.

12.5

The UCB BSD license was removed from the list of licenses that should be referenced from `/usr/share/common-licenses/BSD`. It should instead be included directly in `debian/copyright`, although it will still be in `common-licenses` for the time being.

debconf

`SETTITLE` is now documented (it has been supported for some time). `SETTITLE` is like `TITLE` but takes a template instead of a string to allow translation.

perl

`perl-base` now provides `perlapi-abiname` instead of a package based solely on the Perl version. Perl packages must now depend on `perlapi-${Config{debian_abi}}`, falling back on `${Config{version}}` if `${Config{debian_abi}}` is not set.

perl

Packages using `Makefile.PL` should use `DESTDIR` rather than `PREFIX` to install into the package staging area. `PREFIX` only worked due to a Debian-local patch.

22.33 Version 3.8.4

Released January, 2010.

9.1.1

An FHS exception has been granted for multiarch libraries. Permitting files to instead be installed to `/lib/triplet` and `/usr/lib/triplet` directories.

10.6

Packages may not contain named pipes and should instead create them in `postinst` and remove them in `prerm` or `postrm`.

9.1.1

`/sys` and `/selinux` directories are explicitly allowed as an exception to the FHS.

22.34 Version 3.8.3

Released August, 2009.

4.9

`DEB_*_ARCH_CPU` and `DEB_*_ARCH_OS` variables are now documented and recommended over GNU-style variables for that information.

5.6.8

Source package Architecture fields may contain *all* in combination with other architectures. Clarify when *all* and *any* may be used in different versions of the field.

5.6.14

The Debian archive software does not support uploading to multiple distributions with one `*.changes` file.

5.6.19

The Binary field may span multiple lines.

10.2

Shared library packages are no longer allowed to install libraries in a non-standard location and modify `ld.so.conf`. Packages should either be installed in a standard library directory or packages using them should be built with `RPATH`.

11.8.7

Installation directories for X programs have been clarified. Packages are no longer required to pre-depend on `x11-common` before installing into `/usr/include/X11` and `/usr/lib/X11`.

12.1

Manual pages are no longer required to contain only characters representable in the legacy encoding for that language.

12.1

Localized man pages should either be kept up-to-date with the original version or warn that they're not up-to-date, either with warning text or by showing missing or changed portions in the original language.

12.2

`install-info` is now handled via triggers so packages no longer need to invoke it in maintainer scripts. Info documents should now have directory sections and entries in the document. Packages containing info documents should add a dependency to support partial upgrades.

perl

The requirement for Perl modules to have a versioned `Depend` and `Build-Depend` on `perl >= 5.6.0-16` has been removed.

22.35 Version 3.8.2

Released June, 2009.

2.4

The list of archive sections has been significantly expanded. See [this debian-devel-announce message](#) for the list of new sections and rules for how to categorize packages.

3.9.1

All packages must use debconf or equivalent for user prompting, though essential packages or their dependencies may also fall back on other methods.

5.6.1

The requirements for source package names are now explicitly spelled out.

9.1

Legacy XFree86 servers no longer get a special exception from the FHS permitting `/etc/X11/XF86Config-4`.

9.1.3

Removed obsolete dependency requirements for packages that use `/var/mail`.

11.8.5

Speedo fonts are now deprecated. The X backend was disabled starting in lenny.

12.5

The GNU Free Documentation License version 1.3 is included in common-licenses and should be referenced from there.

22.36 Version 3.8.1

Released March, 2009.

3.8

Care should be taken when adding functionality to essential and such additions create an obligation to support that functionality in essential forever unless significant work is done.

4.4

Changelog files must be encoded in UTF-8.

4.4

Some format requirements for changelog files are now “must” instead of “should.”

4.4.1

Alternative changelog formats have been removed. Debian only supports one changelog format for the Debian Archive.

4.9.1

New `nocheck` option for `DEB_BUILD_OPTIONS` indicating any build-time test suite provided by the package should not be run.

5.1

All control files must be encoded in UTF-8.

5.2

`debian/control` allows comment lines starting with `#` with no preceding whitespace.

9.3

Init scripts ending in `.sh` are not handled specially. They are not sourced and are not guaranteed to be run by `/bin/sh` regardless of the `#!` line. This brings Policy in line with the long-standing behavior of the init system in Debian.

9.3.2

The start action of an init script must exit successfully and not start the daemon again if it's already running.

9.3.2

`/var/run` and `/var/lock` may be mounted as temporary filesystems, and init scripts must therefore create any necessary subdirectories dynamically.

10.4

`/bin/sh` scripts may assume that `local` can take multiple variable arguments and supports assignment.

11.6

User mailboxes may be mode 600 and owned by the user rather than mode 660, owned by user, and group mail.

22.37 Version 3.8.0

Released June, 2008.

2.4, 3.7

The base section has been removed. `contrib` and `non-free` have been removed from the section list; they are only categories. The base system is now defined by priority.

4.9

If `dpkg-source -x` doesn't provide the source that will be compiled, a `debian/rules` patch target is recommended and should do whatever else is necessary.

4.9.1, 10.1

Standardized the format of `DEB_BUILD_OPTIONS`. Specified permitted characters for tags, required that tags be whitespace-separated, allowed packages to assume non-conflicting tags, and required unknown flags be ignored.

4.9.1

Added `parallel=n` to the standardized `DEB_BUILD_OPTIONS` tags, indicating that a package should be built using up to `n` parallel processes if the package supports it

4.13

Debian packages should not use convenience copies of code from other packages unless the included package is explicitly intended to be used that way.

4.14

If `dpkg-source -x` doesn't produce source ready for editing and building with `dpkg-buildpackage`, packages should include a `debian/README.source` file explaining how to generate the patched source, add a new modification, and remove an existing modification. This file may also be used to document packaging a new upstream release and any other complexity of the Debian build process.

5.6.3

The Uploaders field in `debian/control` may be wrapped.

5.6.12

An empty Debian revision is equivalent to a Debian revision of 0 in a version number.

5.6.23

New Homepage field for upstream web sites.

6.5, 6.6, 7

The Breaks field declares that this package breaks another and prevents installation of the breaking package unless the package named in Breaks is deconfigured first. This field should not be used until the `dpkg` in Debian stable supports it.

8.1, 8.2

Clarify which files should go into a shared library package, into a separate package, or into the `-dev` package. Suggest `-tools` instead of `-runtime` for runtime support programs, since that naming is more common in Debian.

9.5

Files in `/etc/cron.{hourly,daily,weekly,monthly}` must be configuration files (upgraded from should). Mention the hourly directory.

11.8.6

Packages providing `/etc/X11/Xresources` files need not conflict with `xbase` (`<< 3.3.2.3a-2`), which is long-obsolete.

12.1

Manual pages in locale-specific directories should use either the legacy encoding for that directory or UTF-8. Country names should not be included in locale-specific manual page directories unless indicating a significant difference in the language. All characters in the manual page source should be representable in the legacy encoding for a locale even if the man page is encoded in UTF-8.

12.5

The Apache 2.0 license is now in `common-licenses` and should be referenced rather than quoted in `debian/copyright`.

12.5

Packages in `contrib` and `non-free` should state in the copyright file that the package is not part of Debian GNU/Linux and briefly explain why.

debconf

Underscore (`_`) is allowed in `debconf` template names.

22.38 Version 3.7.3

Released December, 2007.

5.6.12

Package version numbers may contain tildes, which sort before anything, even the end of a part.

10.4

Scripts may assume that `/bin/sh` supports `local` (at a basic level) and that its `test` builtin (if any) supports `-a` and `-o` binary logical operators.

8.5

The substitution variable `${binary:Version}` should be used in place of `${Source-Version}` for dependencies between packages of the same library.

menu policy

Substantial reorganization and renaming of sections in the Debian menu structure. Packages with menu entries should be reviewed to see if the menu section has been renamed or if one of the new sections would be more appropriate.

5.6.1

The `Source` field in a `.changes` file may contain a version number in parentheses.

5.6.17

The acceptable values for the `Urgency` field are `low`, `medium`, `high`, `critical`, or `emergency`.

8.6

The `shlibs` file now allows an optional `type` field, indicating the type of package for which the line is valid. The only currently supported type is `udeb`, used with packages for the Debian Installer.

3.9.1

Packages following the Debian Configuration management specification must allow for translation of their messages by using a gettext-based system such as `po-debconf`.

12.5

GFDL 1.2, GPL 3, and LGPL 3 are now in common-licenses and should be referenced rather than quoted in debian/copyright.

22.39 Version 3.7.2.2

Released October, 2006.

This release broke the normal rule against introducing normative changes without changing the major patch level.

6.1

Maintainer scripts must not be world writeable (up from a should to a must)

22.40 Version 3.7.2

Released April, 2006.

11.5

Revert the cgi-lib change.

22.41 Version 3.7.1

Released April, 2006.

10.2

It is now possible to create shared libraries without relocatable code (using -fPIC) in certain exceptional cases, provided some procedures are followed, and for creating static libraries with relocatable code (again, using -fPIC). Discussion on debian-devel@lists.debian.org, getting a rough consensus, and documenting it in README.Debian constitute most of the process.

11.8.7

Packages should install any relevant files into the directories `/usr/include/X11/` and `/usr/lib/X11/`, but if they do so, they must pre-depend on `x11-common (>= 1:7.0.0)`

22.42 Version 3.7.0

Released April, 2006.

11.5

Packages shipping web server CGI files are expected to install them in `/usr/lib/cgi-lib/` directories. This location change perhaps should be documented in NEWS

11.5

Web server packages should include a standard scriptAlias of `cgi-lib` to `/usr/lib/cgi-lib`.

9.1.1

The version of FHS mandated by policy has been upped to 2.3. There should be no changes required for most packages, though new top level directories `/media`, `/srv`, etc. may be of interest.

5.1, 5.6.3

All fields, apart from the Uploaders field, in the control file are supposed to be a single logical line, which may be spread over multiple physical lines (newline followed by space is elided). However, any parser for the control file must allow the Uploaders field to be spread over multiple physical lines as well, to prepare for future changes.

10.4

When scripts are installed into a directory in the system PATH, the script name should not include an extension that denotes the scripting language currently used to implement it.

9.3.3.2

packages that invoke initscripts now must use invoke-rc.d to do so since it also pays attention to run levels and other local constraints.

11.8.5.2, 11.8.7, etc

We no longer use `/usr/X11R6`, since we have migrated away to using Xorg paths. This means, for one thing, fonts live in `/usr/share/fonts/X11/` now, and `/usr/X11R6` is gone.

22.43 Version 3.6.2

Released June, 2005.

Recommend doc-base, and not menu, for registering package documentation.

8.1

Run time support programs should live in subdirectories of `/usr/lib/` or `/usr/share`, and preferably the shared lib is named the same as the package name (to avoid name collisions).

11.5

It is recommended that HTTP servers provide an alias `/images` to allow packages to share image files with the web server

22.44 Version 3.6.1

Released August, 2003.

3.10.1

Prompting the user should be done using debconf. Non debconf user prompts are now deprecated.

22.45 Version 3.6.0

Released July, 2003.

Restructuring caused shifts in section numbers and bumping of the minor version number.

Many packaging manual appendices that were integrated into policy sections are now empty, and replaced with links to the Policy. In particular, the appendices that included the list of control fields were updated (new fields like Closes, Changed-By were added) and the list of fields for each of control, `.changes` and `.dsc` files is now in Policy, and they're marked mandatory, recommended or optional based on the current practice and the behavior of the deb-building tool-chain.

Elimination of needlessly deep section levels, primarily in the chapter Debian Archive, from which two new chapters were split out, Binary packages and Source packages. What remained was reordered properly, that is, some sects became sects etc.

Several sections that were redundant, crufty or simply not designed with any sort of vision, were rearranged according to the formula that everything should be either in the same place or properly interlinked. Some things remained split up between different chapters when they talked about different aspects of files: their content, their syntax, and their placement in the file system. In particular, see the new sections about changelog files.

menu policy

Added Games/Simulation and Apps/Education to menu sub-policy

C.2.2

Debian changelogs should be UTF-8 encoded.

10.2

shared libraries must be linked against all libraries that they use symbols from in the same way that binaries are.

7.6

build-depends-indep need not be satisfied during clean target.

22.46 Version 3.5.10

Released May, 2003.

11.8.3

packages providing the x-terminal-emulator virtual package ought to ensure that they interpret the command line exactly like xterm does.

11.8.4

Window managers compliant with the Window Manager Specification Project may add 40 points for ranking in the alternatives

22.47 Version 3.5.9

Released March, 2003.

3.4.2

The section describing the Description: package field once again has full details of the long description format.

4.2

Clarified that if a package has non-build-essential build-dependencies, it should have them listed in the Build-Depends and related fields (i.e. it's not merely optional).

9.3.2

When asked to restart a service that isn't already running, the init script should start the service.

12.6

If the purpose of a package is to provide examples, then the example files can be installed into `/usr/share/doc/package` (rather than `/usr/share/doc/package/examples`).

22.48 Version 3.5.8

Released November, 2002.

12.7

It is no longer necessary to keep a log of changes to the upstream sources in the copyright file. Instead, all such changes should be documented in the changelog file.

7.6

Build-Depends, Build-Conflicts, Build-Depends-Indep, and Build-Conflicts-Indep must also be satisfied when the clean target is called.

menu policy

A new Apps/Science menu section is available

debconf policy

debconf specification cleared up, various changes.

12.1

It is no longer recommended to create symlinks from nonexistent manual pages to `undocumented(7)`. Missing manual pages for programs are still a bug.

22.49 Version 3.5.7

Released August, 2002.

Packages no longer have to ask permission to call MAKEDEV in postinst, merely notifying the user ought to be enough.

2.2.4

cryptographic software may now be included in the main archive.

3.9

task packages are no longer permitted; tasks are now created by a special Tasks: field in the control file.

11.8.4

window managers that support netwm can now add 20 points when they add themselves as an alternative for `/usr/bin/x-window-manager`

10.1

The default compilation options have now changed, one should provide debugging symbols in all cases, and optionally step back optimization to `-O0`, depending on the `DEB_BUILD_OPTIONS` environment variable.

7.6, 4.8

Added mention of build-arch, build-indep, etc, in describing the relationships with Build-Depends, Build-Conflicts, Build-Depends-Indep, and Build-Conflicts-Indep. May need to review the new rules.

8

Changed rules on how, and when, to invoke `ldconfig` in maintainer scripts. Long rationale.

Added the last note in 3.5.6 upgrading checklist item regarding build rules, please see below

22.50 Version 3.5.6

Released July, 2001.

2.5

Emacs and TeX are no longer mandated by policy to be priority standard packages

11.5

Programs that access docs need to do so via `/usr/share/doc`, and not via `/usr/doc/` as was the policy previously

12.3

Putting documentation in `/usr/doc` versus `/usr/share/doc` is now a “serious” policy violation.

11.5

For web servers, one should not provide non-local access to the `/usr/share/doc` hierarchy. If one can’t provide access controls for the <http://localhost/doc/> directory, then it is preferred that one ask permission to expose that information during the install.

7

There are new rules for build-indep/build-arch targets and there is a new Build-Depend-Indep semantic.

22.51 Version 3.5.5

Released May, 2001.

12.1

Manpages should not rely on header information to have alternative manpage names available; it should only use symlinks or `.so` pages to do this

Clarified note in 3.5.3.0 upgrading checklist regarding examples and templates: this refers only to those examples used by scripts; see section 10.7.3 for the whole story

Included a new section 10.9.1 describing the use of dpkg-statoverride; this does not have the weight of policy

Clarify Standards-Version: you don't need to rebuild your packages just to change the Standards-Version!

10.2

Plugins are no longer bound by all the rules of shared libraries

X Windows related things:

11.8.1

Clarification of priority levels of X Window System related packages

11.8.3

Rules for defining x-terminal-emulator improved

11.8.5

X Font policy rewritten: you must read this if you provide fonts for the X Window System

11.8.6

Packages must not ship `/usr/X11R6/lib/X11/app-defaults/`

11.8.7

X-related packages should usually use the regular FHS locations; imake-using packages are exempted from this

11.8.8

OpenMotif linked binaries have the same rules as OSF/Motif-linked ones

22.52 Version 3.5.4

Released April, 2001.

11.6

The system-wide mail directory is now `/var/mail`, no longer `/var/spool/mail`. Any packages accessing the mail spool should access it via `/var/mail` and include a suitable Depends field;

11.9; perl-policy

The perl policy is now part of Debian policy proper. Perl programs and modules should follow the current Perl policy

22.53 Version 3.5.3

Released April, 2001.

7.1

Build-Depends arch syntax has been changed to be less ambiguous. This should not affect any current packages

10.7.3

Examples and templates files for use by scripts should now live in `/usr/share/<package>` or `/usr/lib/<package>`, with symbolic links from `/usr/share/doc/<package>/examples` as needed

22.54 Version 3.5.2

Released February, 2001.

11.8.6

X `app-defaults` directory has moved from `/usr/X11R6/lib/X11/app-defaults` to `/etc/X11/app-defaults`

22.55 Version 3.5.1

Released February, 2001.

8.1

`dpkg-shlibdeps` now uses `objdump`, so shared libraries have to be run through `dpkg-shlibdeps` as well as executables

22.56 Version 3.5.0

Released January, 2001.

11.8.5

Font packages for the X Window System must now declare a dependency on `xutils` (`>= 4.0.2`)

22.57 Version 3.2.1.1

Released January, 2001.

This release broke the normal rule against introducing normative changes without changing the major patch level.

9.3.2

Daemon startup scripts in `/etc/init.d/` should not contain modifiable parameters; these should be moved to a file in `/etc/default/`

12.3

Files in `/usr/share/doc` must not be referenced by any program. If such files are needed, they must be placed in `/usr/share/<package>/`, and symbolic links created as required in `/usr/share/doc/<package>/`

Much of the packaging manual has now been imported into the policy document

22.58 Version 3.2.1

Released August, 2000.

11.8.1

A package of priority standard or higher may provide two binaries, one compiled with support for the X Window System, and the other without

22.59 Version 3.2.0

Released August, 2000.

10.1

By default executables should not be built with the debugging option `-g`. Instead, it is recommended to support building the package with debugging information optionally.

12.8

Policy for packages where the upstream uses HTML changelog files has been expanded. In short, a plain text changelog file should always be generated for the upstream changes

Please note that the new release of the X window system (3.2) shall probably need sweeping changes in policy

Policy for packages providing the following X-based features has been codified:

11.8.2

X server (virtual package xserver)

11.8.3

X terminal emulator (virtual package x-terminal-emulator)

11.8.4

X window manager (virtual package x-window-manager, and `/usr/bin/x-window-manager` alternative, with priority calculation guidelines)

12.8.5

X fonts (this section has been written from scratch)

11.8.6

X application defaults

11.8.7

Policy for packages using the X Window System and FHS issues has been clarified;

11.7.3

No package may contain or make hard links to conffiles

8

Noted that newer dpkg versions do not require extreme care in always creating the shared lib before the symlink, so the unpack order be correct

22.60 Version 3.1.1

Released November, 1999.

7.1

Correction to semantics of architecture lists in Build-Depends etc. Should not affect many packages

22.61 Version 3.1.0

Released October, 1999.

defunct

`/usr/doc/<package>` has to be a symlink pointing to `/usr/share/doc/<package>`, to be maintained by `postinst` and `prerm` scripts.

7.1, 7.6

Introduced source dependencies (Build-Depends, etc.)

9.3.4

`/etc/rc.boot` has been deprecated in favour of `/etc/rcS.d`. (Packages should not be touching this directory, but should use `update-rc.d` instead)

9.3.3

`update-rc.d` is now the *only* allowable way of accessing the `/etc/rc?.d/[SK]??*` links. Any scripts which manipulate them directly must be changed to use `update-rc.d` instead. (This is because the `file-rc` package handles this information in an incompatible way.)

12.7

Architecture-specific examples go in `/usr/lib/<package>/examples` with symlinks from `/usr/share/doc/<package>/examples/*` or from `/usr/share/doc/<package>/examples` itself

9.1.1

Updated FHS to a 2.1 draft; this reverts `/var/state` to `/var/lib`

9.7; mime-policy

Added MIME sub-policy document

12.4

VISUAL is allowed as a (higher priority) alternative to EDITOR

11.6

Modified liblockfile description, which affects mailbox-accessing programs. Please see the policy document for details

12.7

If a package provides a changelog in HTML format, a text-only version should also be included. (Such a version may be prepared using `lynx -dump -nolist`.)

3.2.1

Description of how to handle version numbers based on dates added

22.62 Version 3.0.1

Released July, 1999.

10.2

Added the clarification that the `.la` files are essential for the packages using libtool's `libltdl` library, in which case the `.la` files must go in the run-time library package

22.63 Version 3.0.0

Released June, 1999.

9.1

Debian formally moves from the FSSTND to the FHS. This is a major change, and the implications of this move are probably not all known.

4.1

Only 3 digits of the Standards version need be included in control files, though all four digits are still permitted.

12.6

The location of the GPL has changed to `/usr/share/common-licenses`. This may require changing the copyright files to point to the correct location of the GPL and other major licenses

10.2

Packages that use libtool to create shared libraries must include the `.la` files in the `-dev` packages

10.8

Use `logrotate` to rotate log files

now 11.8

section 5.8 has been rewritten (Programs for the X Window System)

9.6; menu-policy

There is now an associated menu policy, in a separate document, that carries the full weight of Debian policy

11.3

Programs which need to modify the files `/var/run/utmp`, `/var/log/wtmp` and `/var/log/lastlog` must be installed setgid utmp

22.64 Version 2.5.0

Released October, 1998.

Please note that section numbers below this point may not match the current Policy Manual.

- Rearranged the manual to create a new Section 4, Files
 - Section 3.3 (“Files”) was moved to Section 4. The Sections that were Section 4 and Section 5 were moved down to become Section 5 and Section 6.
 - What was Section 5.5 (“Log files”) is now a subsection of the new Section 4 (“Files”), becoming section 4.8, placed after “Configuration files”, moving the Section 4.8 (“Permissions and owners”) to Section 4.9. All subsections of the old Section 5 after 5.5 were moved down to fill in the number gap.
- Modified the section about changelog files to accommodate upstream changelogs which were formatted as HTML. These upstream changelog files should now be accessible as `/usr/doc/package/changelog.html.gz`
- Symlinks are permissible to link the real, or upstream, changelog name to the Debian mandated name.
- Clarified that HTML documentation should be present in some package, though not necessarily the main binary package.
- Corrected all references to the location of the copyright files. The correct location is `/usr/doc/package/copyright`
- Ratified the architecture specification strings to cater to the HURD.

22.65 Version 2.4.1

Released April, 1998.

Updated section 3.3.5 Symbolic links

symbolic links within a toplevel directory should be relative, symbolic links between toplevel directories should be absolute (cf., Policy Weekly Issue#6, topic 2)

Updated section 4.9 Games

manpages for games should be installed in `/usr/man/man6` (cf., Policy Weekly Issue#6, topic 3)

Updated Chapter 12 Shared Libraries

ldconfig must be called in the postinst script if the package installs shared libraries (cf., Policy Weekly Issue #6, fixes:bug#20515)

22.66 Version 2.4.0

Released January, 1998

Updated section 3.3.4 Scripts

- `/bin/sh` may be any POSIX compatible shell
- scripts including bashisms have to specify `/bin/bash` as interpreter
- scripts which create files in world-writable directories (e.g., in `/tmp`) should use `tempfile` or `mktemp` for creating the directory

Updated section 3.3.5 Symbolic Links

symbolic links referencing compressed files must have the same file extension as the referenced file

Updated section 3.3.6 Device files

`/dev/tty*` serial devices should be used instead of `/dev/cu*`

Updated section 3.4.2 Writing the scripts in `/etc/init.d`

- all `/etc/init.d` scripts have to provide the following options: start, stop, restart, force-reload
- the reload option is optional and must never stop and restart the service

Updated section 3.5 Cron jobs

cron jobs that need to be executed more often than daily should be installed into `/etc/cron.d`

Updated section 3.7 Menus

removed section about how to register HTML docs to ``menu'` (the corresponding section in 4.4, Web servers and applications, has been removed in policy 2.2.0.0 already, so this one was obsolete)

New section 3.8 Keyboard configuration

details about how the backspace and delete keys should be handled

New section 3.9 Environment variables

no program must depend on environment variables to get a reasonable default configuration

New section 4.6 News system configuration

`/etc/news/organization` and `/etc/news/server` should be supported by all news servers and clients

Updated section 4.7 Programs for the X Window System

- programs requiring a non-free Motif library should be provided as `foo-smotif` and `foo-dmotif` package
- if `lesstif` works reliably for such program, it should be linked against `lesstif` and not against a non-free Motif library

Updated section 4.9 Games

games for X Windows have to be installed in `/usr/games`, just as non-X games

22.67 Version 2.3.0

Released September, 1997.

- new section ``4.2 Daemons'` including rules for `/etc/services`, `/etc/protocols`, `/etc/rpc`, and `/etc/inetd.conf`
- updated section about ``Configuration files'`: packages may not touch other packages' configuration files
- MUAs and MTAs have to use `liblockfile`

22.68 Version 2.2.0

Released July, 1997.

- added section 4.1 ``Architecture specification strings'`: use `<arch>-linux` where `<arch>` is one of the following: `i386`, `alpha`, `arm`, `m68k`, `powerpc`, `sparc`.
- detailed rules for `/usr/local`
- user ID's
- editor/pager policy
- cron jobs
- device files
- don't install shared libraries as executable
- `app-defaults` files may not be `conffiles`

22.69 Version 2.1.3

Released March, 1997.

- two programs with different functionality must not have the same name
- “Webstandard 3.0”
- “Standard for Console Messages”
- Libraries should be compiled with `-D_REENTRANT`
- Libraries should be stripped with `strip --strip-unneeded`

22.70 Version 2.1.2

Released November, 1996.

- Some changes WRT shared libraries

22.71 Version 2.1.1

Released September, 1996.

- No hard links in source packages
- Do not use `dpkg-divert` or `update-alternatives` without consultation
- Shared libraries must be installed stripped

22.72 Version 2.1.0

Released August, 1996.

- Upstream changelog must be installed too

LICENSE

Copyright © 1996, 1997, 1998 Ian Jackson and Christian Schwarz

These are the copyright dates of the original Policy manual. Since then, this manual has been updated by many others. No comprehensive collection of copyright notices for subsequent work exists.

This manual is free software; you may redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

A copy of the GNU General Public License is available as `/usr/share/common-licenses/GPL` in the Debian distribution or on the World Wide Web at <https://www.gnu.org/licenses/>.

INDEX

R

- reboot
 - signaling, 88
- reboot-required, 88
- reboot-required.pkgs, 88

S

- signaling
 - reboot, 88